

Zona Git y Documentación

Complementos a temario aula

None

None

Table of contents

1. Inicio	3
2. Intro	4
2.1 Git y Github	4
2.2 Git en local	10
2.3 Git, ramas y fusiones (merge)	19
2.4 Acceso con token (PAT)	26
3. Documentación	30
3.1 1. GitHub Pages	30
3.2 DNS	35
3.3 Markdown	43
3.4 mkdocs	45
4. Formatos	55
4.1 1. YAML, TOML y JSON	55
4.2 Linting	61
4.3 Ejemplos	63
5. Amplia	64
5.1 Git. Trabajo en grupos: teams	64
5.2 Git teams tarea	86
5.3 Git seguro	87
6. Utiles	102
6.1 Git y VSCode	102
6.2 1. Anexo dudas git	103
6.3 Borrador: uso GitHub como "Drive" en LMSGI	106

1. Inicio

Ayudas para el uso de Git y Github, etc.

2. Intro

2.1 Git y Github

2.1.1 1. Introducción al Control de Versiones

Lectura de los apartados 1.1, 1.2 y 1.3 de este enlace:

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress, is available here: <https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>

1. Acerca del Control de Versiones
2. Una breve historia de Git
3. Fundamentos de Git

2.1.2 2. Git: instalación

Es posible que `git` ya esté ya instalado en su máquina virtual. Si no:

```
$ sudo apt update
$ sudo apt install git
```

```
h1 {
  color:red;
}
```

2.1.3 3. GitHub

GitHub, Inc. is an Internet hosting service for software development and version control using Git. It provides the distributed version control of Git plus access control, bug tracking, software feature requests, task management, continuous integration, and wikis for every project.

Para la práctica debe tener cuenta creada en GitHub.

3.1. Creación de un repositorio en GitHub

Para la práctica se crea en GitHub un repositorio de nombre `test-md-clases` : - privado - con fichero README

2.1.4 4. Acceso GitHub desde local

GitHub ya **no** permite acceder desde línea de comandos con usuario y clave. En su lugar deberá usar el acceso ssh (o un *token* personal, esta segunda forma se explica en el anexo).

4.1. Acceso con ssh

Debe seguir los pasos indicados en [Connecting to GitHub with SSH](#). Es necesario crear claves SSH. Para ello puede seguir estos dos enlaces:

1. [Generación de una nueva clave SSH](#)
2. [Agregar una clave SSH nueva a tu cuenta de GitHub](#)

Vea paso a paso un ejemplo de creación de las claves:

```
$ ssh-keygen -t ed25519 -C "profesor@iesromerovargas.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/alu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alu/.ssh/id_ed25519
Your public key has been saved in /home/alu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:10h/UuiiNSGtpnb7brUoPdyu79mQSyBtsPiQGHlgicc profesor@iesromerovargas.com
```

```
The key's randomart image is:
+--[ED25519 256]--+
|  .                |
| + o   .   .       |
| . E o   o + . .   |
| . o o o B * .     |
|   . + S X = .     |
|       = * +. +     |
|       o +o ++.     |
|       . . . *.o=    |
|       . =*** .     |
+-----[SHA256]-----+
$ ls -al .ssh/*.pub
-rw-r--r-- 1 alu alu 108 feb 23 09:12 .ssh/id_ed25519.pub
$ cat .ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIbaMp/uhRc1wPz0AbQhooXxQN7IsUMfhqE9KtA0toXa profesor@iesromerovargas.com
$
```

Luego se [añade la clave SSH a GitHub](#)

Y finalmente puede probar la configuración con el comando `ssh -T git@github.com`

```
$ ssh -T git@github.com
Hi angioies! You've successfully authenticated, but GitHub does not provide shell access.
$
```

Si todo es correcto ahora puede clonar el repositorio de GitHub en local usando el comando `git clone`. La referencia del repositorio a usar la proporciona Github en el botón "Code"

```
$ git clone git@github.com:angioies/test-md-clase.git
Clonando en 'test-md-clase'...
Enter passphrase for key '/home/alu/.ssh/id_ed25519':
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
Recibiendo objetos: 100% (15/15), listo.
Resolviendo deltas: 100% (2/2), listo.
remote: Total 15 (delta 2), reused 9 (delta 1), pack-reused 0
$
```

Nota: Puede mantener en cache la autenticación usando `ssh-agent` como se indica en [este enlace](#).

2.1.5 5. Proceso básico desarrollo

Se muestran en este apartado los pasos básicos para trabajar en local y subir los cambios a GitHub.

- `git clone`: descargar el repositorio creado previamente en GitHub

```
$ git clone git@github.com:angoies/test-md-github.git
Clonando en 'test-md-github'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Recibiendo objetos: 100% (3/3), listo.
$ cd test-md-github/
~/test-md-github$ ls
README.md
~/test-md-github$
```

- Trabajo en local: modificar fichero del repositorio, por ejemplo `README.md`

```
~/test-md-github$ code README.md
~/test-md-github$
```

- `git status`: comprobar el estado del repositorio, del área de trabajo y de la zona de seguimiento:

```
~/test-md-github$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
    modificado:   README.md

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
~/test-md-github$
```

- `git add`: añadir a la zona de seguimiento los ficheros modificados que queremos añadir al repositorio

```
~/test-md-github$ git add README.md
~/test-md-github$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
(usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificado:   README.md

~/test-md-github$
```

- `git commit`: añadir los cambios en la zona de seguimiento al repositorio

```
~/test-md-github$ git commit -m "cambios en README para demo"
Author identity unknown

*** Por favor cuéntame quien eres.
Ejecuta

git config --global user.email "you@example.com"
git config --global user.name "Tu Nombre"

para configurar la identidad por defecto de tu cuenta.
Omite --global para configurar tu identidad solo en este repositorio.

fatal: no es posible auto-detectar la dirección de correo (se obtuvo 'alu@xdebian11.(none)')
~/test-md-github$
```

El commit **no se ha realizado**, `git` necesita que configuremos el nombre y correo para añadirlo a la información de confirmación. Esto se puede hacer en modo local (opción por defecto) sólo para el repositorio actual, o en modo global.

En este ejemplo lo hacemos en modo local:

```
~/test-md-github$ git config user.email "jgomez@iesromerovargas.com"
~/test-md-github$ git config user.name "Jose Gomez"
```

Y ahora si se ejecuta el commit:

```
~/test-md-github$ git commit -m "cambios en README para demo"
[main 64f7ed1] cambios en README para demo
1 file changed, 20 insertions(+)
~/test-md-github$
```

- `git push`: sincronizar el estado actual del repositorio local con el de GitHub

```
~/test-md-github$ git push
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
```

```
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 431 bytes | 431.00 KiB/s, listo.
Total 3 (delta 0), reusado 0 (delta 0), pack-reusado 0
To github.com:angoies/test-md-github.git
c9a4725..64f7ed1 main -> main
~/test-md-github$
```

* Verificar en la web de GitHub los cambios.

2.1.6 6. Enlaces

- [Git - la guía sencilla](#)
- Vídeos: [Git y GitHub para Poetas](#)

2.1.7 7. Anexos

7.1. Paso de repositorio HTTPS a SSH

Tenga en cuenta que si ya ha clonado el repositorio con HTTPS y desea usar SSH será necesario modificar la configuración de los remotos.

Para ello puede seguir estas indicaciones [Cambiar la URL del repositorio remoto](#)

```
$ git remote -v
> origin https://github.com/OWNER/REPOSITORY.git (fetch)
> origin https://github.com/OWNER/REPOSITORY.git (push)

$ git remote set-url origin git@github.com:OWNER/REPOSITORY.git

$ git remote -v
# Verify new remote URL
> origin git@github.com:OWNER/REPOSITORY.git (fetch)
> origin git@github.com:OWNER/REPOSITORY.git (push)
$
```

7.2. Acceso a GitHub con tokens

Otra forma de acceder a Github es usando su usuario y un PAT (*personal access token*)

En este enlace <https://github.blog/2020-12-15-token-authentication-requirements-for-git-operations/> lo más destacable:

Use of tokens offer a number of security benefits over password-based authentication:

1. **Unique** – tokens are specific to GitHub and can be generated per use or per device
2. **Revocable** – tokens can be individually revoked at any time without needing to update unaffected credentials
3. **Limited** – tokens can be narrowly scoped to allow only the access necessary for the use case
4. **Random** – tokens are not subject to the types of dictionary or brute force attempts that simpler passwords that you need to remember or enter regularly might be

Lo que nos interesa: **crear un token y utilizarlo en lugar de usuario/clave.**

- [Enlace directo a creación de tokens](#)
- [Instrucciones de cómo crear el token](#)

Para esta práctica basta generar un token "clásico" y darle todos los permisos al scope `repo`. Almacene el token en un lugar seguro, lo necesitará a continuación.

Para comprobar que todo es correcto intentar clonar el repositorio usando como *password* el token obtenido:

```
$ git clone https://github.com/angoies/test-md-clase.git
Clonando en 'test-md-clase'...
Username for 'https://github.com': angoies
Password for 'https://angoies@github.com':
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 15 (delta 2), reused 9 (delta 1), pack-reused 0
Recibiendo objetos: 100% (15/15), listo.
```

```
Resolviendo deltas: 100% (2/2), listo.
$
```

Cada vez que intente acceder a un repositorio le pedirá usuario y token. Para trabajar con más comodidad es posible almacenar usuario y token en cache usando el comando `git config --global credential.helper 'cache --timeout 3600'` (*timeout* especificado en segundos). Tras ejecutarlo le pedirá autenticarse una vez y esos datos se quedan en cache con ese tiempo de refresco.

```
$ cd test-md-clase/
~/test-md-clase$ git config --global credential.helper 'cache --timeout 3600'
~/test-md-clase$ git push
Username for 'https://github.com': angoes
Password for 'https://angoies@github.com':
Everything up-to-date
~/test-md-clase$ git push
Everything up-to-date
~/test-md-clase$
```

Es posible almacenar el token de forma permanente, pero tenga en cuenta que lo hará en texto plano en el fichero `~/.git-credentials`.

```
$ git config --local credential.helper store
$ git push http://example.com/repo.git
Username: <type your username>
Password: <type your password>

[several days later]
$ git push http://example.com/repo.git
[your credentials are used automatically]
```

Si quiere personalizar el fichero donde se almacena puede usar la opción `git config --global credential.helper "store --file ~/.my-credentials"`

2.2 Git en local

2.2.1 1. Trabajando en local

1.1. Creación del repositorio

Para este apartado se trabajará con un repositorio local sin necesidad de crearlo previamente en GitHub.

Primero se crea el directorio del proyecto

```
~$ mkdir practica01
~$ cd practica01/
~/practica01$
```

y se inicia el repositorio con el comando `git init`. Puesto que GitHub usa como rama por defecto `main` puede crear el repositorio usando ese nombre como rama principal con la opción `-b` (*branch*):

```
~/practica01$ git init -b main
Iniciado repositorio Git vacío en /home/alu/practica01/.git/
~/practica01$ git status
En la rama main

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)
~/practica01$
```

Si no lo ha hecho antes se debe configurar el nombre de usuario y correo que usará git. Puede comprobar si están ya definidos con el comando `git config`:

```
~$ git config -l --local
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
~$ git config -l --global
~$
```

En este caso no lo estaban y se definen sólo para el repositorio

```
~$ git config user.email "jgomez@iesromerovargas.com"
~$ git config user.name "Jose Gómez"
~$
```

1.2. Comienza la edición

Se crea y se edita por ejemplo el fichero `index.html`:

```
~/practica01$ code index.html
~/practica01$ cat index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Práctica 01</title>
</head>
<body>
  <h1>Ejemplo uso repo local</h1>
  <ul>
    <li> item 01
    <li> item 02
    <li> item 03
  </ul>
</body>
</html>
~/practica01$
```

Puede comprobar el estado del repositorio con `git status`

```
~/practica01$ git status
En la rama main

No hay commits todavía
```

```
Archivos sin seguimiento:
(usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
index.html

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
~/practica01$
```

El comando indica que el fichero `index.html` no está incluido en la zona de seguimiento (*staging area*). Por lo tanto en primer lugar se añade con `git add index.html` (o con `git add .` todos los archivos del directorio). Si ejecuta de nuevo el comando `git status` el mensaje ha cambiado e indica que hay cambios a la espera de ser confirmados (e incorporarlos al repositorio local):

```
~/practica01$ git add index.html
~/practica01$ git status
En la rama main

No hay commits todavía

Cambios a ser confirmados:
(usa "git rm --cached <archivo>..." para sacar del área de stage)
nuevo archivo: index.html

~/practica01$
```

Para confirmar los cambios se usa `git commit -m "mensaje"` con un texto descriptivo (sin la opción `-m` nos abriría el editor por defecto y nos permite editar un mensaje más completo).

```
~/practica01$ git commit -m "index inicial"
[main (commit-raiz) 2248e30] index inicial
1 file changed, 15 insertions(+)
create mode 100644 index.html
~/practica01$
```

Puede consultar el log `git log` para ver los commits realizado (que se identifican por un hash):

```
~/practica01$ git log --oneline
2248e30 (HEAD -> main) index inicial
~/practica01$
```

Se crea ahora un segundo fichero (`README.md`) y se repite el proceso de status, add, commit. Luego puede ver en el log los dos commits:

```
~/practica01$ echo "fichero pendiente completar" > README.md
~/practica01$ git status
En la rama main
Archivos sin seguimiento:
(usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
README.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
~/practica01$ git add README.md
~/practica01$ git commit -m "Se añade README del proyecto"
[main 223773f] Se añade README del proyecto
1 file changed, 1 insertion(+)
create mode 100644 README.md
~/practica01$ git log --oneline
223773f (HEAD -> main) Se añade README del proyecto
2248e30 index inicial
~/practica01$
```

Modifique `index.html` para cambiar la cabecera de nivel 1 y añadir una cabecera de nivel 2. Luego añadir al repositorio:

```
~/practica01$ git add index.html
~/practica01$ git commit -m "modifico h1, añado h2, borro párrafo"
[main 7845c43] modifico h1, añado h2, borro párrafo
1 file changed, 3 insertions(+), 1 deletion(-)
~/practica01$ git log --oneline
7845c43 (HEAD -> main) modifico h1, añado h2, borro párrafo
223773f Se añade README del proyecto
2248e30 index inicial
~/practica01$
```

Puede ver los cambios entre los dos últimos commits con `git show` o con `git diff` (o de forma más genérica con `git diff HEAD~1 HEAD`)

```
~/practica01$ git show
commit 7845c43c3e59100476538d637d8386412e7cbea9 (HEAD -> main)
Author: jgomez <jgomez@iesromerovargas.com>
Date: Fri Mar 24 17:50:25 2023 +0100

    modifico h1, añado h2, borro párrafo

diff --git a/index.html b/index.html
index ffb8be..c8d545e 100644
```

```

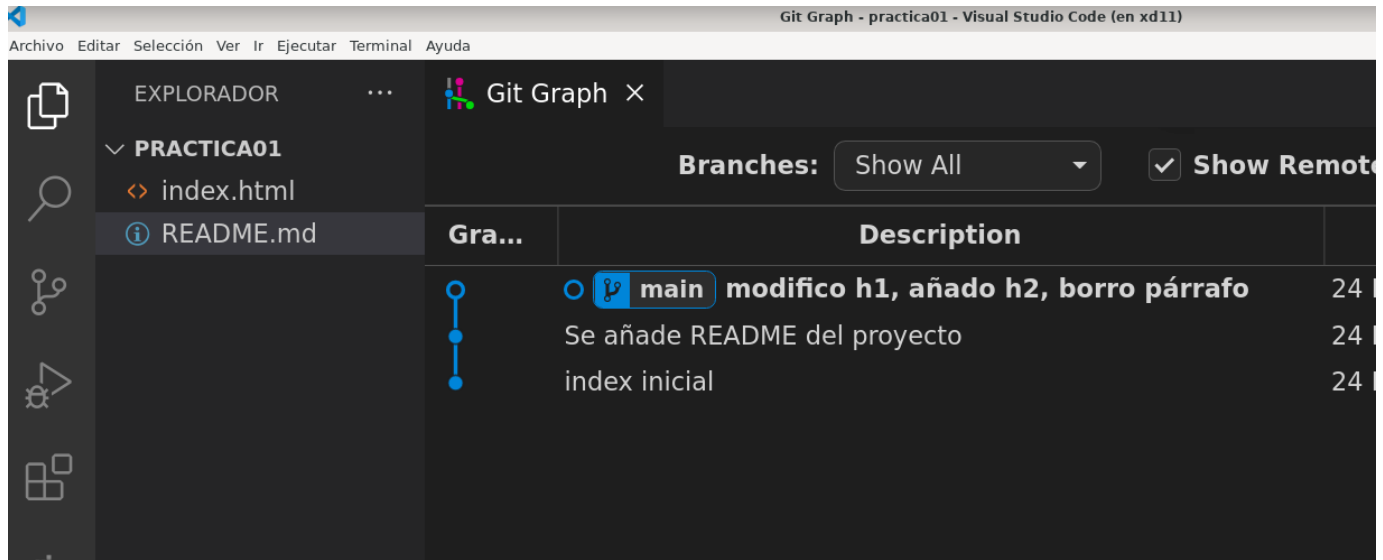
--- a/index.html
+++ b/index.html
@@ -8,7 +8,9 @@
</head>
<body>

-   <h1>Ejemplo uso repo local</h1>
+   <h1>Modifico cabecera de nivel 1</h1>
+   <h2>Añado cabecera nivel 2 y borro párrafo</h2>
+
+   <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Temporibus alias est error natus nostrum magni, quaerat sunt in pariatur ratione placeat
dolorum odio explicabo aliquam deleniti voluptas hic voluptates velit?</p>

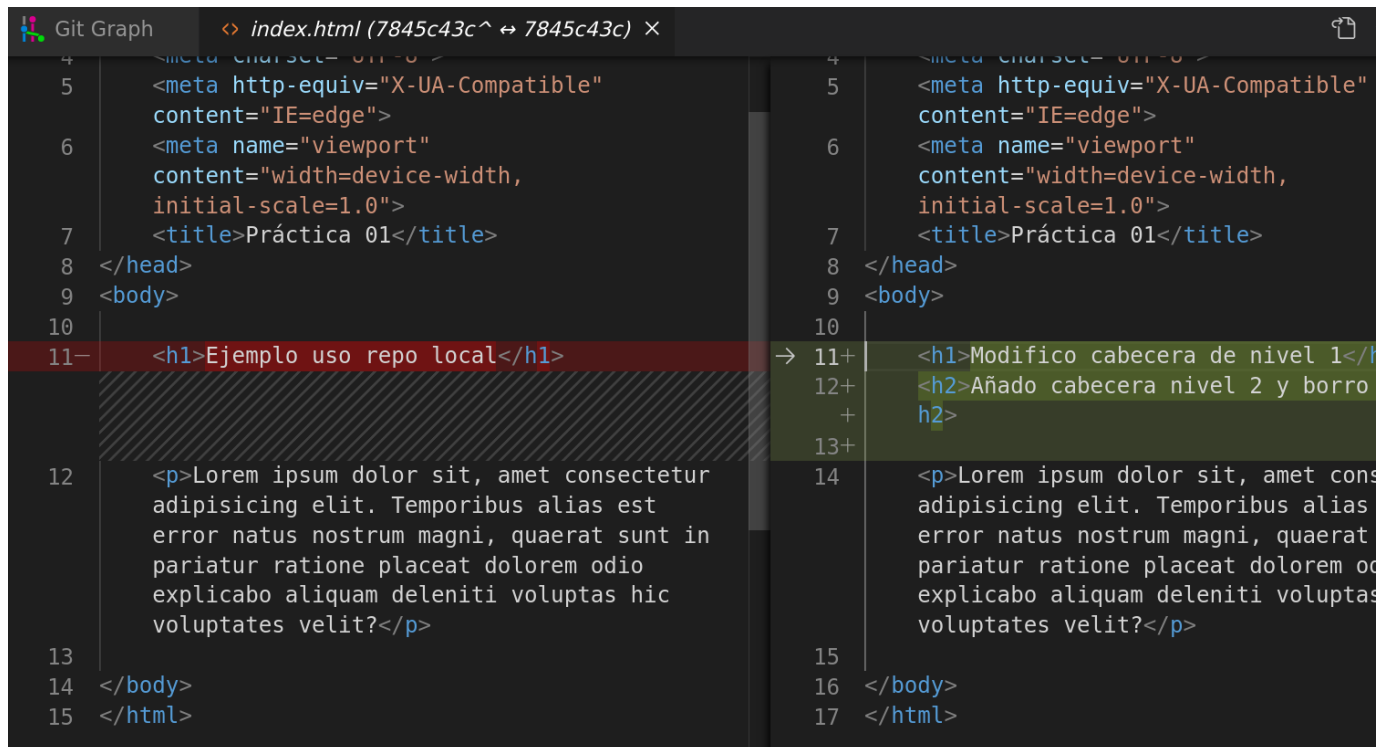
</body>
~/practica01$

```

O desde VsCode con el complemento *Git Graph* puede ver el log y los cambios:



Y las diferencias entre dos commits:



2.2.2 2. Deshacer cambios

Hay dos formas de **volver hacia atrás** en el repositorio usando el comando `git reset <hash>`:

- retorno *suave*
- retorno *duro*.

2.1. Modo suave

En el modo suave se cumple que:

- se sitúa el estado del repositorio en el commit que se le indique, y
- en el directorio de trabajo (no en la zona de seguimiento) tendrá los ficheros con las modificaciones pendientes tras ese commit

Si parte del ejemplo anterior con tres commits, uno inicial donde se añade `index.html` otro donde se añade el fichero `README` y otro donde se modifica `index.html`:

```
~/practica01$ git log --oneline
7845c43 (HEAD -> main) modifiko h1, añado h2, borro párrafo
223773f Se añade README del proyecto
2248e30 index inicial
~/practica01$
```

Si ejecuta un `git reset` indicando el segundo commit puede comprobar que

- el repositorio se sitúa en el commit del hash indicado.
- en el directorio de trabajo aparece el fichero `index.html` con modificaciones, pero no está en la zona de seguimiento:

```
~/practica01$ git reset 223773f
Cambios fuera del área de stage tras el reset:
M   index.html
~/practica01$ git status
En la rama main
Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
    modificado:   index.html

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
~/practica01$ cat index.html
<!DOCTYPE html>
...
  <h1>Modifiko cabecera de nivel 1</h1>
  <h2>Añado cabecera nivel 2 y borro párrafo</h2>
...
</body>
</html>
~/practica01$
```

2.2. Modo duro

En el modo duro (`git reset --hard <hash>`) el proceso es similar, pero no deja los ficheros modificados en la zona de trabajo.

Para comprobarlo puede hacer un reset duro al primer commit y: - el fichero `README.md` no está ni siquiera en el directorio de trabajo - y no hay cambios pendientes de seguimiento:

```
~/practica01$ git log --oneline
223773f (HEAD -> main) Se añade README del proyecto
2248e30 index inicial
~/practica01$ git reset --hard 2248e30
HEAD está ahora en 2248e30 index inicial
~/practica01$ ls
index.html
~/practica01$ git status
En la rama main
nada para hacer commit, el árbol de trabajo está limpio
~/practica01$
```

2.2.3 3. Trabajando con ramas

Se puede bifurcar el repositorio y crear una rama con `git branch`. Se pueden ver las ramas existentes con `git branch -l`, donde se marca la rama activa con un `*`.

Si parte del ejemplo previo:

```
~/practica01$ git log --oneline
2248e30 (HEAD -> main) index inicial
~/practica01$ ls -l
total 4
-rw-r--r-- 1 alu alu 533 mar 24 18:17 index.html
~/practica01$
```

Puede crear una rama de nombre `css` con `git branch css`, **moverse** a ella con `git switch css` (o `git checkout`):

```
~/practica01$ git branch css
~/practica01$ git switch css
Cambiado a rama 'css'
~/practica01$ git branch -l
* css
  main
~/practica01$
```

El comando `git status` también informa de la rama activa

```
~/practica01$ git status
En la rama css
nada para hacer commit, el árbol de trabajo está limpio
~/practica01$
```

Siga trabajando en la rama `css`. En el momento de la creación de la rama `css` el contenido de la rama creada es el mismo que la rama `main`. Observe que en `main` tiene un único commit, que es el mismo que en la rama creada:

```
~/practica01$ git log --oneline
2248e30 (HEAD -> css, main) index inicial
~/practica01$ git status
En la rama css
nada para hacer commit, el árbol de trabajo está limpio
~/practica01$
```

Cree ahora en la nueva rama el fichero `estilos.css` que se enlaza con el fichero `index.html`. Finalmente añada los dos ficheros al repositorio (estando en la rama `css`):

```
~/practica01$ code estilos.css index.html
~/practica01$ git add index.html estilos.css
~/practica01$ git commit -m "añado estilos CSS a index"
[css dc48b05] añadido estilos CSS a index
2 files changed, 4 insertions(+)
 create mode 100644 estilos.css
~/practica01$ git log --oneline
dc48b05 (HEAD -> css) añadido estilos CSS a index
2248e30 (main) index inicial
~/practica01$
~/practica01$ ls -l
total 8
-rw-r--r-- 1 alu alu 22 mar 24 19:37 estilos.css
-rw-r--r-- 1 alu alu 580 mar 24 19:37 index.html
~/practica01$
```

2.2.4 4. Diferencias

Si quiere ver las diferencia entre dos commit use `git diff <commit1> <commit2>`. Para ver sólo las de un fichero se indica al final del comando el fichero `git diff <commit1> <commit2> <fichero>`. Observe como se marcan la líneas añadidas con un `+`

```
~/practica01$ git diff 2248e30 dc48b05
diff --git a/estilos.css b/estilos.css
new file mode 100644
index 0000000..8f7a5ec
--- /dev/null
+++ b/estilos.css
@@ -0,0 +1,3 @@
+hl {
+  color:red;
+}
diff --git a/index.html b/index.html
index ffb8be..60d62a6 100644
```

```

--- a/index.html
+++ b/index.html
@@ -5,6 +5,7 @@
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Práctica 01</title>
+ <link rel="stylesheet" href="estilos.css">
</head>
<body>

~/practica01$

```

Si quiere ver las diferencias entre dos ramas use el comando `git diff rama1 rama2`. O para ver los cambios de un fichero `git diff rama1 rama2 fichero`

2.2.5 5. Fusionar (merge)

Si ahora cambia a la rama `main` vea que **en esa rama no existe el fichero `estilos.css` ni la modificación de `index.html`**:

```

~/practica01$ git switch main
Cambiado a rama 'main'
~/practica01$ ls -l
total 4
-rw-r--r-- 1 alu alu 533 mar 24 19:40 index.html
~/practica01$ git log --oneline
2248e30 (HEAD -> main) index inicial
~/practica01$

```

Lo normal es que si los cambios en la rama `css` están probados y se aceptan, los quiera "fusionar" a la rama principal. Esto se hace con el comando `git merge <rama_a_fusionar>`.

Primero cambie a la rama `main`

```

~/practica01$ git switch main
~/practica01$ git log --oneline
2248e30 (HEAD -> main) index inicial

```

Y ahora la fusión:

```

~/practica01$ git merge css
Actualizando 2248e30..dc48b05
Fast-forward
 estilos.css | 3 +++
 index.html | 1 +
 2 files changed, 4 insertions(+)
 create mode 100644 estilos.css
~/practica01$

```

Y si muestre los commits

```

~/practica01$ git log --oneline
dc48b05 (HEAD -> main, css) añadido estilos CSS a index
2248e30 index inicial
~/practica01$ ls -l
total 8
-rw-r--r-- 1 alu alu 22 mar 24 19:41 estilos.css
-rw-r--r-- 1 alu alu 580 mar 24 19:41 index.html
~/practica01$

```

2.2.6 6. Borrar una rama

La rama de trabajo `css` se puede mantener o puede borrarla (`git branch -d css`) si ya no es necesaria:

```

~/practica01$ git branch -d css
Eliminada la rama css (era dc48b05)..
~/practica01$ git branch -l
* main
~/practica01$

```

Tiene ejemplos de fusiones con ramas en el apartado [Git, ramas y fusiones](#).

2.2.7.7. Anexos

7.1. Conexión con repositorio remoto

El repositorio usado en este documento es **local**, no está conectado con ningún repositorio remoto.

Si necesita que el repositorio esté compartido en GitHub:

1. Acceda a GitHub
2. Use el botón de **Crear** un nuevo repositorio:
3. Por comodidad use el mismo nombre que el directorio del proyecto, `practica01`
4. Añada una descripción.
5. Decida si será publico o privado. Usamos privado
6. **Ignore las opciones de** crear ficheros `README`, `.gitignore` y elegir licencia, ya que cómo nos indica GitHub "**Skip this step if you're importing an existing repository**", que es nuestro caso.

Tras darle a crear aparece una pantalla con información de los pasos a seguir:

Quick setup — if you've done this kind of thing before

or

HTTPS

SSH

`git@github.com:angoies/practica01.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repo:

...or create a new repository on the command line

```
echo "# practica01" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:angoies/practica01.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin git@github.com:angoies/practica01.git
git branch -M main
git push -u origin main
```

De las tres opciones la que aplica es la segunda (ya tiene el repositorio en local). Indica cómo añadir el repositorio previamente creado en CLI a GitHub. Son tres pasos a ejecutar en el directorio del repositorio en local:

1. Añadir a la configuración el repositorio remoto de GitHub con el nombre `origin` con el comando `git remote add`
2. Renombrar la rama actual del repositorio local a `main` con `git branch -m`
3. Subir el código del repositorio al remoto `origin` con `git push`

Si en CLI comprueba la configuración local del repositorio remoto con `git remote -v` (antes de ejecutar los tres comandos anteriores) observará que está vacía. Añada el remoto y compruebe:

```
~/practica01$ git remote -v
~/practica01$ git remote add origin git@github.com:angoies/practica01.git
~/practica01$ git remote -v
origin  git@github.com:angoies/practica01.git (fetch)
origin  git@github.com:angoies/practica01.git (push)
~/practica01$
```

El segundo paso de las instrucciones no es necesario porque nuestra rama principal en local ya se llama `main`. Si no fuese el caso habría que renombrar la rama principal a `main` con el comando `git branch -M` (equivale a `git branch --move --force`):

```
~/git-module$ git branch -M main
~/git-module$ git branch --list
* main
~/git-module$
```

Ahora la rama en local tiene el mismo nombre que la rama en GitHub, y puede ejecutar el comando `git push -u origin main` como recomendaban en GitHub para subir el contenido del repositorio local al repositorio remoto GitHub.

```
~/practica01$ git push -u origin main
Enter passphrase for key '/home/alu/.ssh/id_ed25519':
Enumerando objetos: 7, listo.
Contando objetos: 100% (7/7), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (5/5), listo.
Escribiendo objetos: 100% (7/7), 895 bytes | 895.00 KiB/s, listo.
Total 7 (delta 1), reusado 0 (delta 0), pack-reusado 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:angoies/practica01.git
 * [new branch]      main -> main
Rama 'main' configurada para hacer seguimiento a la rama remota 'main' de 'origin'.
~/practica01$
```

Observe la última línea: indica que se ha asociado la rama local `main` con la rama remota `main` de `origin`. El comando `git push -u origin main` hace dos cosas: - Configura `main` como rama de *upstream* de `origin` - Hace un push del código en `main` a `origin`

En siguientes sincronizaciones de `main` bastaría hacer `git push`.

Si comprueba en GitHub verá que se ha creado la rama `main` y que tiene en ella los ficheros del repositorio local, la información de los `commit` realizados, etc:

[Code](#)
[Issues](#)
[Pull requests](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security](#)

main
1 branch
0 tags
Go to file
Add file
Code

angoies
añado estilos CSS a index
dc48b05 13 hours ago
2 commits

estilos.css	añado estilos CSS a index	13 hours ago
index.html	añado estilos CSS a index	13 hours ago

Add a README with an overview of your project.
Add a README

7.2. master o main

Nota: si inici un repositorio sólo con `git init` la rama creada será por defecto `master`:

```
~/git-module$ git init
ayuda: Using 'master' as the name for the initial branch. This default branch name
ayuda: is subject to change. To configure the initial branch name to use in all
ayuda: of your new repositories, which will suppress this warning, call:
ayuda:
ayuda:   git config --global init.defaultBranch <name>
ayuda:
ayuda: Names commonly chosen instead of 'master' are 'main', 'trunk' and
ayuda: 'development'. The just-created branch can be renamed via this command:
ayuda:
ayuda:   git branch -m <name>
Iniciado repositorio Git vacío en /home/alu/git-module/.git/
~/git-module$
```

Observe que nos avisa de que la rama creada por defecto se llama `master` y que puede ser renombrada, etc. Puesto que GitHub usa como rama por defecto `main` es mejor crear el repositorio usando ese nombre como rama principal con la opción `-b` (branch) con `git init -b main`.

En el caso de que el repositorio se cree en GitHub y se clone en local la rama principal ya tiene como nombre `main`. En este enlace tiene una explicación de esta pequeña *movida* con el uso del termino **master** y GitHub: <https://platzi.com/blog/cambios-en-github-master-main/>

2.3 Git, ramas y fusiones (merge)

Se muestra una secuencia de ejemplos para mostrar las distintas situaciones posibles en fusiones de ramas.

2.3.1 1. Crear un repositorio local

```
~$ mkdir merge
~$ cd merge/
~/merge$ git init -b main
Iniciado repositorio Git vacío en /home/alu/merge/.git/
~/merge$
```

2.3.2 2. Fusión tipo *fast forward*

Añade un fichero en el repositorio creado y añade un primer commit en la rama `main`.

```
~/merge$ echo "# README" > README.md
~/merge$ git add README.md
~/merge$ git commit -m "inicio repo vacío con README"
[main (commit-raíz) 5f953ae] inicio repo vacío con README
1 file changed, 1 insertion(+)
create mode 100644 README.md
~/merge$
```

Crear una nueva rama de nombre `web`, cambiar a ella y crear en ella un fichero de nombre `index.html` con una cabecera de nivel h1:

```
~/merge$ git branch web
~/merge$ git switch web
Cambiado a rama 'web'
~/merge$ code index.html
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>index.html</title>
</head>
<body>
  <h1>Página web en repositorio</h1>
</body>
</html>
```

Añadir el fichero al repositorio (desde la rama `web`):

```
~/merge$ git status
En la rama web
Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
    index.html

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
~/merge$ git add index.html
~/merge$ git commit -m "añadimos index.html en rama web"
[web b81acff] añadimos index.html en rama web
1 file changed, 13 insertions(+)
create mode 100644 index.html
~/merge$
```

En la rama `web` ahora hay dos commits:

```
~/merge$ git log --oneline
b81acff (HEAD -> web) añadimos index.html en rama web
5f953ae (main) inicio repo vacío con README
~/merge$
```

Si ahora se mueve de la rama `web` a `main` puede comprobar que el único commit coincide con el primero de la rama `web`. La rama `main` "no ha avanzado" desde que creamos la rama `web` (y al listar el directorio en `main` no aparece el fichero `index.html`):

```
~/merge$ git switch main
Cambiado a rama 'main'
~/merge$ git log --oneline
```

```
5f953ae (HEAD -> main) inicio repo vacío con README
~/merge$ ls -a
.  ..  .git  README.md
~/merge$
```

Ahora se procede a fusionar el código de la rama `web` con la rama `main`. Esta fusión en la que `main` no ha avanzado respecto a otra rama es la más sencilla o directa.

```
~/merge$ git merge web
Actualizando 5f953ae..b81acff
Fast-forward
 index.html | 13 ++++++
 1 file changed, 13 insertions(+)
 create mode 100644 index.html
~/merge$ git log --oneline
b81acff (HEAD -> main, web) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Se observa que **ni siquiera hace falta un nuevo commit**, simplemente `main` ahora "apunta" al último commit de la rama `web`.

2.3.3 3. Fusión recursiva

Se prueba ahora una fusión usando una nueva rama de nombre `estilos`.

Los pasos a dar son:

- en la rama `estilos`
- añadir un nuevo fichero `estilos.css`.
- modificar `index.html` para que enlace ese fichero de estilos.
- en la rama `main`
- modificar el fichero `index.html` para añadir una cabecera h2.

Observe que **en dos ramas distintas modificamos el mismo fichero `index.html` pero en líneas distintas**.

Empiece con la rama de nombre `estilos`:

```
~/merge$ git branch estilos
~/merge$ git switch estilos
Cambiado a rama 'estilos'
~/merge$ ls
index.html  README.md
~/merge$
```

Se crea en esta rama el fichero `estilos.css` y se modifica `index.html` para enlazarlo con la hoja de estilos:

```
~/merge$ echo "h1 {color:blue;}" > estilos.css
~/merge$ code index.html
~/merge$ git diff index.html
diff --git a/index.html b/index.html
index 4e63d8a..fddc6b0 100644
--- a/index.html
+++ b/index.html
@@ -5,6 +5,7 @@
 <meta http-equiv="X-UA-Compatible" content="IE=edge">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>index.html</title>
+ <link rel="stylesheet" href="estilos.css">
 </head>
 <body>
 <h1>Página web en repositorio</h1>
~/merge$
```

Y se actualiza el repositorio en la rama `estilos`

```
~/merge$ git add index.html estilos.css
~/merge$ git commit -m "update index, add estilos"
[estilos 87f9f8b] update index, add estilos
 2 files changed, 2 insertions(+)
 create mode 100644 estilos.css
~/merge$ git log --oneline
87f9f8b (HEAD -> estilos) update index, add estilos
b81acff (web, main) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Ahora cambie a la rama `main` y modifique el fichero `index.html` añadiendo una cabecera de nivel 2:

```
~/merge$ git switch main
Cambiado a rama 'main'
~/merge$ ls
index.html  README.md
~/merge$ code index.html
~/merge$ git diff index.html
diff --git a/index.html b/index.html
index 4e63d8a..83b2e9b 100644
--- a/index.html
+++ b/index.html
@@ -8,6 +8,7 @@
</head>
<body>
  <h1>Página web en repositorio</h1>
+  <h2>añadido desde rama main</h2>

</body>
</html>
~/merge$
```

Y por último actualice el repositorio en la rama `main`:

```
~/merge$ git status
En la rama main
Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
(usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
modificado:   index.html

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
~/merge$ git add index.html
~/merge$ git commit -m "update index con h2 en main"
[main 1e39070] update index con h2 en main
 1 file changed, 1 insertion(+)
~/merge$ git log --oneline
1e39070 (HEAD -> main) update index con h2 en main
b81acff (web) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Ahora se procede a la fusión de la rama `estilos` con `main`. Recuerde que se ha trabajado en las dos ramas. Debe estar posicionado en la rama `main`. El comando `git merge` ahora lleva un mensaje de confirmación ya que como verá se crea un nuevo commit:

```
~/merge$ git merge estilos -m "fusiono estilos con main"
Auto-fusionando index.html
Merge made by the 'recursive' strategy.
 estilos.css | 1 +
 index.html  | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 estilos.css
~/merge$
```

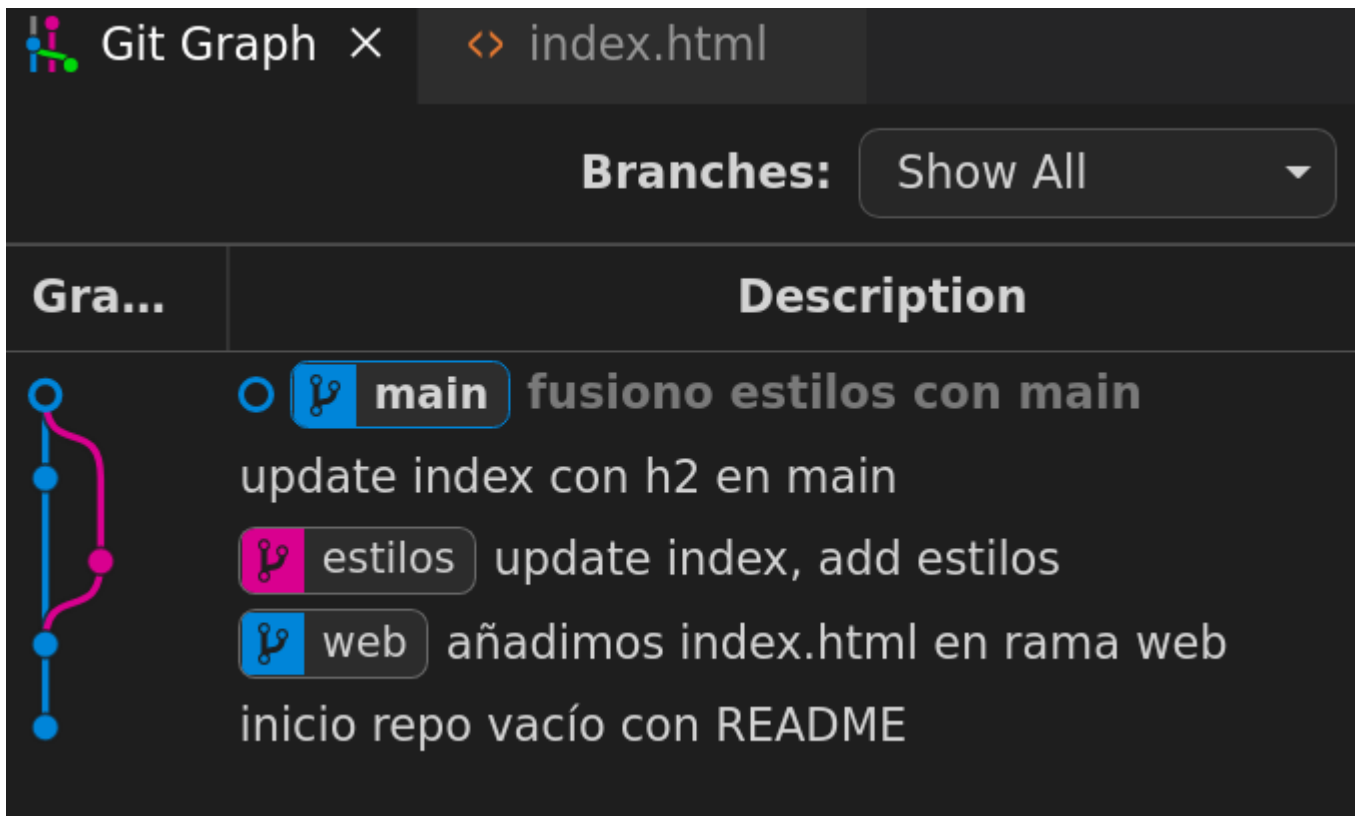
Y observe el nuevo punto de confirmación

```
~/merge$ git log --oneline
e00d1c4 (HEAD -> main) fusiono estilos con main
1e39070 update index con h2 en main
87f9f8b (estilos) update index, add estilos
b81acff (web) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Vea que la fusión se realiza sin problemas ya que los cambios afectan a distintas partes del fichero `index.html`. Observe que en el primer ejemplo de fusión se usó como estrategia **fast-forward** y en este **recursive**.

En el gráfico de ramas puede observar:

- al hacer la primera fusión la rama `web` se unifica con `main` en un único hilo (y no crea nuevos commits)
- al hacer esta segunda fusión la rama `estilos` se bifurca y se reúne posteriormente con `main` tras generarse en la fusión un nuevo commit.



en Vscode puede obtener esta imagen en el control de versiones ya incorporado o usando una extensión como *Git Graph*

2.3.4 4. Fusión con colisión a resolver

Por último se analiza un ejemplo de fusión con una colisión que hay que resolver manualmente debido a que en la rama `main` y en una nueva rama **se modifica la misma línea del fichero** `index.html`.

Para probarlo se crea una nueva rama `modificaH1` que modifica la cabecera h1 de `index.html`

```
~/merge$ git branch modificaH1
~/merge$ git switch modificaH1
Cambiado a rama 'modificaH1'
~/merge$ nano index.html
```

Para ver las diferencias del fichero que estamos editando respecto a la versión en el repositorio `git diff <fichero>`:

```
~/merge$ git diff index.html
diff --git a/index.html b/index.html
index cb9580d..8ce3391 100644
--- a/index.html
+++ b/index.html
@@ -8,8 +8,8 @@
<link rel="stylesheet" href="estilos.css">
</head>
<body>
- <h1>Página web en repositorio</h1>
+ <h1>Página web en repositorio: añadido desde rama modificarH1</h1>
  <h2>añadido desde rama main</h2>

</body>
</html>
\ No newline at end of file
+</html>
~/merge$
```

Se añade al repositorio en esta rama nueva:

```
~/merge$ git add index.html
~/merge$ git commit -m "update h1"
[modificaH1 a3d1091] update h1
1 file changed, 2 insertions(+), 2 deletions(-)
~/merge$
```

Se muestran los commits:

```
~/merge$ git log --oneline
a3d1091 (HEAD -> modificaH1) update h1
e00d1c4 (main) fusiono estilos con main
1e39070 update index con h2 en main
87f9f8b (estilos) update index, add estilos
b81acff (web) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Ahora regrese a la rama `main` y modifique la misma línea del fichero `index.html`:

```
~/merge$ git switch main
Cambiado a rama 'main'
~/merge$ nano index.html
~/merge$ git diff index.html
diff --git a/index.html b/index.html
index cb9580d..a58f89f 100644
--- a/index.html
+++ b/index.html
@@ -8,8 +8,8 @@
     <link rel="stylesheet" href="estilos.css">
 </head>
 <body>
-   <h1>Página web en repositorio</h1>
+   <h1>UPDATE EN MAIN: Página web en repositorio</h1>
     <h2>añadido desde rama main</h2>

 </body>
</html>
+</html>
~/merge$
```

Actualice el repositorio en la rama `main`:

```
~/merge$ git add index.html
~/merge$ git commit -m "update h1"
[main 978e6d6] update h1
 1 file changed, 2 insertions(+), 2 deletions(-)
~/merge$ git log --oneline
978e6d6 (HEAD -> main) update h1
e00d1c4 fusiono estilos con main
1e39070 update index con h2 en main
87f9f8b (estilos) update index, add estilos
b81acff (web) añadimos index.html en rama web
5f953ae inicio repo vacío con README
~/merge$
```

Si intenta ahora la fusión indicará que **hay un conflicto de fusión** en `index.html` y que debe solucionarlo y luego hacer un commit:

```
~/merge$ git merge modificaH1
Auto-fusionando index.html
CONFLICTO (contenido): Conflicto de fusión en index.html
Fusión automática falló; arregle los conflictos y luego realice un commit con el resultado.
~/merge$
```

Si consulta el estado es este momento

```
~/merge$ git status
En la rama main
Tienes rutas no fusionadas.
(arregla los conflictos y corre "git commit"
(usa "git merge --abort" para abortar la fusion)

Rutas no fusionadas:
(usa "git add <archivo>..." para marcar una resolución)
ambos modificados:   index.html

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
~/merge$
```

indica que hay ramas no fusionadas, y que en el fichero `index.html` están reflejados los conflictos.

Si visualiza el contenido del fichero que colisiona se han marcado las líneas en conflicto con un formato que:

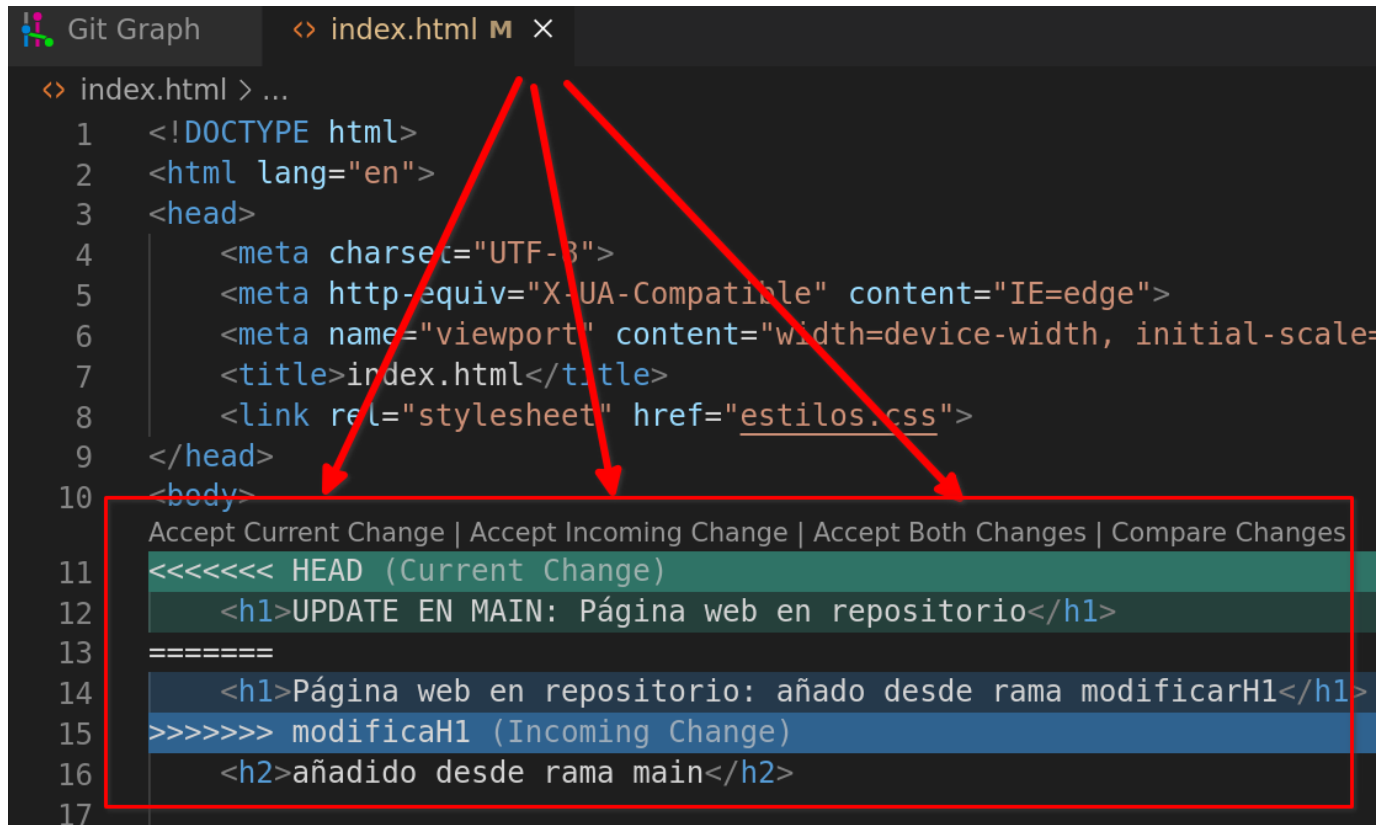
- delimita la zona de conflicto con `<<<<<<` y `>>>>>>`,
- y las dos zonas en conflicto separadas por `=====`.

Este es el aspecto del fichero `index.html` en VsCode:

```
~/merge$ cat index.html
<!DOCTYPE html>
<html lang="en">
...
</head>
<body>
<<<<<< HEAD
  <h1>UPDATE EN MAIN: Página web en repositorio</h1>
=====
  <h1>Página web en repositorio: añadido desde rama modificarH1</h1>
>>>>>> modificaH1
  <h2>añadido desde rama main</h2>

</body>
</html>
~/merge$
```

De hecho el editor VSCode facilita el trabajo de edición al mostrarnos opciones para aceptar la versión superior, la inferior o ambos cambios:



Tiene dos opciones:

1. Editar el fichero, guardar los cambios con la versión correcta, y hacer un `add` y `commit`. Ya estaría terminada la fusión
2. Otra opción es parar la fusión y volver a la situación previa con `git merge --abort`

Si opta por la primera:

```
~/merge$ nano index.html
~/merge$ git add index.html
~/merge$ git commit -m "fusion modificah1 con colision"
[main 4d3bb41] fusion modificah1 con colision
~/merge$ git log --online
4d3bb41 (HEAD -> main) fusion modificah1 con colision
978e6d6 update h1
a3d1091 (modificah1) update h1
e00d1c4 fusiono estilos con main
1e39070 update index con h2 en main
87f9f8b (estilos) update index, add estilos
b81acff (web) añadimos index.html en rama web
5f953ae inicio repo vacio con README
~/merge$
```

Una vez fusionado la gráfica de ramas queda:


Git Graph
×

Branches:

Show All

Gra...	Description
 main	fusion modificah1 con colision update h1
 modificaH1	update h1 fusiono estilos con main
 estilos	update index, add estilos
 web	añadimos index.html en rama web
	inicio repo vacío con README

Recuerde que es posible comprobar las diferencias entre dos ramas **antes de hacer la fusión** con el comando `git diff ramaA ramaB`

```
~/merge$ git diff main modificaH1
diff --git a/index.html b/index.html
index a58f89f..8ce3391 100644
--- a/index.html
+++ b/index.html
@@ -8,7 +8,7 @@
<link rel="stylesheet" href="estilos.css">
</head>
<body>
- <h1>UPDATE EN MAIN: Página web en repositorio</h1>
+ <h1>Página web en repositorio: añadido desde rama modificarH1</h1>
  <h2>añadido desde rama main</h2>
</body>
~/merge$
```

2.3.5 5. Enlaces

- Tutorial [Sobre git merge](#) en web de Atlassian.

2.4 Acceso con token (PAT)

En algunos casos es posible que no tenga acceso por SSH. En esa situación no podrá usar certificados SSH.

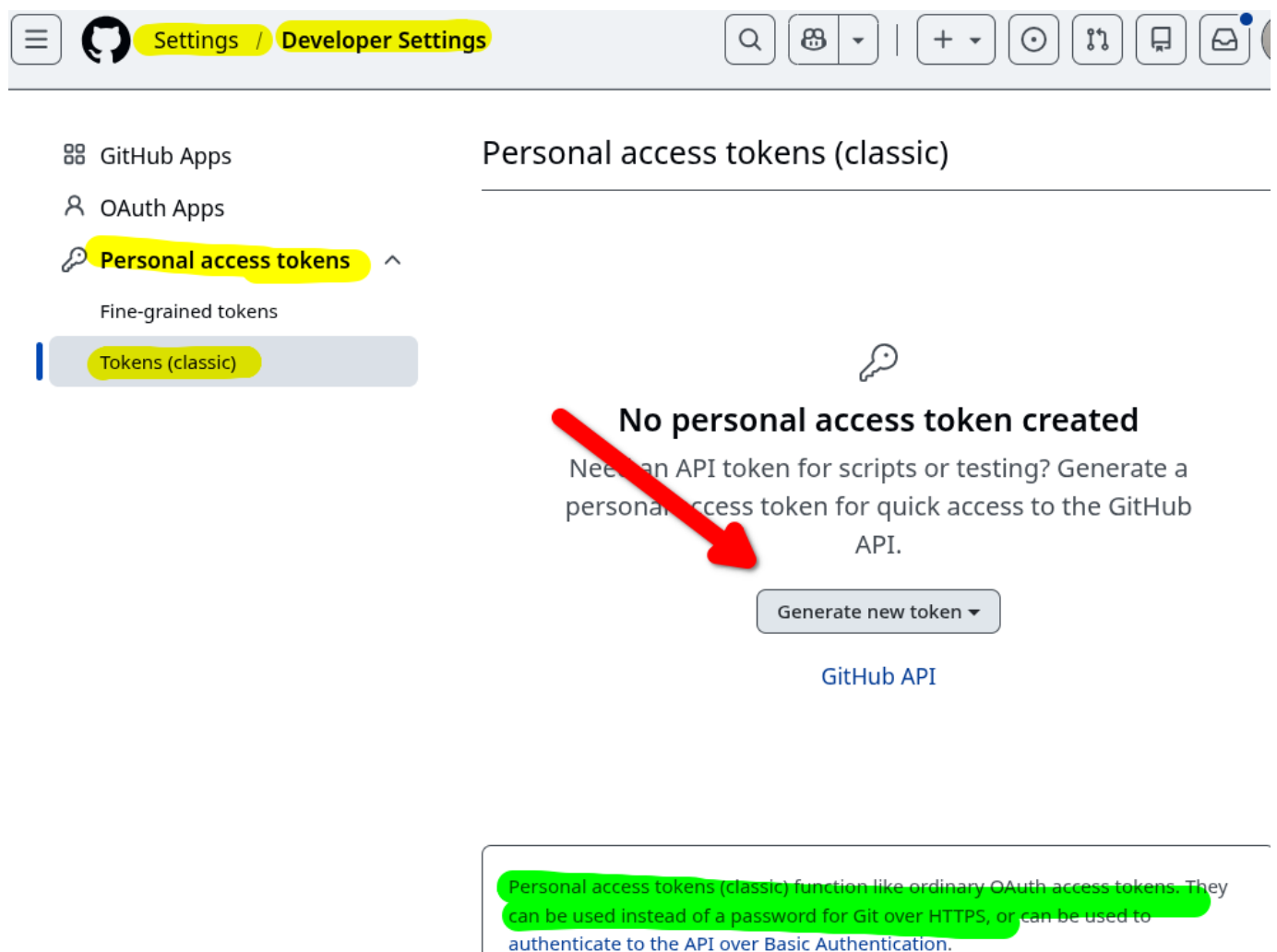
En su lugar puede usar el acceso a GitHub usando HTTP e identificándose con:

- su usuario
- y un *Personal Acces Token* (PAT), que sustituye a la contraseña.

Tenga en cuenta que al clonar el repositorio debe usar la opción HTTPS, donde la URL tiene un formato similar a `https://github.com/user/nombrerepo.git`.

2.4.1 1. Creación

El PAT se crea a en los *Settings* del usuario, *Developer Settings* y puede usar el tipo *Tokens (classic)*.



Personal access tokens (classic)

No personal access token created

Need an API token for scripts or testing? Generate a personal access token for quick access to the GitHub API.

[Generate new token](#)

[GitHub API](#)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

En los `scopes` y para el uso en estos ejemplos basta activar `repo` y `workflow` (en realidad `workflow` incluye `repo`).

GitHub Apps

OAuth Apps

Personal access tokens ^

Fine-grained tokens

Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access token. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

25-demo

What's this token for?

Expiration

30 days (Apr 19, 2026)

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☒ workflow	Update GitHub Action workflows

Ya sólo queda anotar y usar el PAT como clave cuando la solicite.

Some of the scopes you've selected are included in other scopes. Only the minimum set of necessary scopes has been saved. ✕

GitHub Apps

OAuth Apps

Personal access tokens ^

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)

Generate new token ▼

Tokens you have generated that can be used to access the [GitHub API](#).



Make sure to copy your personal access token now. You won't be able to see it again!



ghp_

[Redacted token]



Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

[Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)



© 2026 GitHub, Inc.

2.4.2.2. Uso de helpers

Para no tener que estar copiando y pegando el valor del PAT se usa el comando `git config --global credential.helper 'cache --timeout=segundos'`

Este comando permite no tener que estar escribiendo ni el nombre de usuario y contraseña (o token) cada vez que hace un git push o git pull, pero sin dejar sus claves grabadas en el disco duro.

Básicamente guarda las credenciales **en la memoria RAM por un rato y luego las olvida**. El tiempo por defecto son 900 segundos (15 minutos)

Por ejemplo si quiere que recuerde por una hora (3600 segundos), el comando sería:

```
git config --global credential.helper 'cache --timeout=3600'
```

El flag `--global` es para que se aplique a todos sus repositorios y no solo al que tenga abierto en la terminal.

Otra opción es almacenar en el disco duro de forma permanente usando como helper `store` en vez de `cache`:

```
git config --global credential.helper store
```

En ese caso las credenciales se almacenan en texto plano en el fichero `~/.git-credentials`.

2.4.3 3. SSH: acceder sin frase de paso

Se puede añadir la frase de paso SSH al agente SSH para que no la pida reiteradamente. Para ello:

1. Primero compruebe si está iniciado el agente ssh con el comando `ssh-add -l`

```
alu@tos:~$ ssh-add -l
Could not open a connection to your authentication agent.
alu@tos:~$
```

2. Si indica que no puede abrir la conexión es necesario iniciar el agente SSH. Si no puede pasar al paso 3.

```
alu@tos:~$ eval `ssh-agent -s`
Agent pid 916589
alu@tos:~$ ssh-add -l
The agent has no identities.
alu@tos:~$
```

3. Añadir la clave con `ssh-add -t <time> <file>` (si se omite `-t time` se añade de forma permanente). Si se omite `file` toma la clave por defecto. Puede comprobar que se ha añadido con `ssh-add -l`:

```
alu@tos:~$ ssh-add -t 2h
Enter passphrase for /home/alu/.ssh/id_ed25519:
Identity added: /home/alu/.ssh/id_ed25519 (clases.alu+01@gmail.com)
Lifetime set to 7200 seconds
alu@tos:~$ ssh-add -l
256 SHA256:W9Ka18JIVwQfuC68EUUp1wpv7acGwroNdzucFnG4wr4 clases.alu+01@gmail.com (ED25519)
alu@tos:~$
```

Ya puede usar la clave SSH durante ese periodo sin introducir la frase de paso. Se pueden borrar las claves del agente con `ssh-add -D`.

3. Documentación

3.1 1. GitHub Pages

GitHub Pages es un servicio de alojamiento de sitio estático que toma archivos HTML, CSS y JavaScript directamente desde un repositorio en GitHub, opcionalmente ejecuta los archivos a través de un proceso de compilación y publica un sitio web. Puede ver ejemplos de sitios de GitHub Pages en la colección de ejemplos de GitHub Pages.

Tiene dos opciones:

- página por repositorio: una URL similar a `username.github.io/repositorio`
- página por usuario: con una URL similar a `username.github.io`

En este caso se muestra el uso de la primera: la idea es que la documentación del proyecto vaya junto con el código en el mismo repositorio.

3.1.1 1.1. Configurar Pages en el repositorio

El primer paso es crear el repositorio. El segundo es configurar Pages, que están desactivadas por defecto.

Debe acceder a los *Settings* del repositorio, Pages, y seleccionar la rama que será el origen (source) de Pages:

Tiene varias opciones:

- La rama principal, `main`, y decidir si los ficheros estarán en la raíz de dicha rama o en el directorio `docs`.
- Crear una rama específica para Pages (en el apartado *Code*), por *costumbre* de nombre `gh-pages` y usarla como fuente.

Tenga en cuenta que la rama que use debe estar creada previamente. En este ejemplo se usa la rama `gh-pages` y en ella como directorio la raíz de esa rama, `root`.

The screenshot shows the GitHub repository settings for 'clases-alu-01 / test-ghpages'. The 'Pages' tab is selected in the left sidebar. The main content area shows 'GitHub Pages' settings. A blue notification box states: 'Your site is ready to be published at <https://clases-alu-01.github.io/test-ghpages/>'. Under the 'Source' section, it says 'Your GitHub Pages site is currently being built from the gh-pages branch.' and shows a dropdown menu set to 'Branch: gh-pages' and a directory dropdown set to '/ (root)'. There is a 'Save' button next to the directory dropdown. Under the 'Theme Chooser' section, it says 'Select a theme to publish your site with a Jekyll theme.' and there is a 'Choose a theme' button.

Observe como se muestra la URL de publicación de la página, algo similar a `https://usuario.github.io/nombre_repo/`.

Si accede con el navegador se muestra una página inicia, tomada del fichero `README.md` que hay en la rama: Pages por defecto busca un `index.html`, un `index.md` y si no los hay `README.md`.



Por lo tanto el procedimiento habitual para publicar sería:

1. Configurar Pages en GitHub
2. Hacer clone del repositorio en local.
3. Trabajar en local en la rama `gh-pages` y crear los ficheros HTML, CSS, añadir imágenes ,etc.
4. Hacer add y commit de los ficheros.
5. Hacer `push` al repositorio en GitHub
6. Comprobar con el navegador (recuerde que los cambios pueden tardar un poco en reflejarse).

3.1.2 1.2. Ejemplo del proceso

1.2.1. Clonar

Primer paso, clonar:

```
clases@vm:~/tmp$ git clone https://github.com/clases-alu-01/test-ghpages.git
Clonando en 'test-ghpages'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Desempaquetando objetos: 100% (3/3), 635 bytes | 635.00 KiB/s, listo.
clases@vm:~/tmp$ cd test-ghpages/
clases@vm:~/tmp/test-ghpages$
```

Al listar se observa la rama `main` y no la rama `gh-pages`. Puede listar con el parámetro `-r` para ver las ramas remotas. Cambiamos a la rama `gh-pages` con `git switch`:

```
clases@vm:~/tmp/test-ghpages$ git branch -l
* main
clases@vm:~/tmp/test-ghpages$ git branch -l -r
origin/HEAD -> origin/main
origin/gh-pages
origin/main
clases@vm:~/tmp/test-ghpages$ git switch gh-pages
Rama 'gh-pages' configurada para hacer seguimiento a la rama remota 'gh-pages' de 'origin'.
Cambiado a nueva rama 'gh-pages'
clases@vm:~/tmp/test-ghpages$
```

1.2.2. Editar la web

Ya puede trabajar en local con su editor favorito. Cree una página `index.html` con texto y enlazarla con un fichero de estilos, `estilos.css`. Puede comprobar que no están en el `staged` con `git status`:

```
clases@vm:~/tmp/test-ghpages$ code .
clases@vm:~/tmp/test-ghpages$ ls
estilos.css  index.html  README.md
clases@vm:~/tmp/test-ghpages$ git status
En la rama gh-pages
Tu rama está actualizada con 'origin/gh-pages'.

Archivos sin seguimiento:
(usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
    estilos.css
    index.html

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
clases@vm:~/tmp/test-ghpages$
```

Añada a la zona de seguimiento y haga `commit`:

```
clases@vm:~/tmp/test-ghpages$ git add estilos.css index.html
clases@vm:~/tmp/test-ghpages$ git commit -m "Primera versión web"
[gh-pages f037a5e] Primera versión web
 2 files changed, 26 insertions(+)
 create mode 100644 estilos.css
 create mode 100644 index.html
clases@vm:~/tmp/test-ghpages$
```

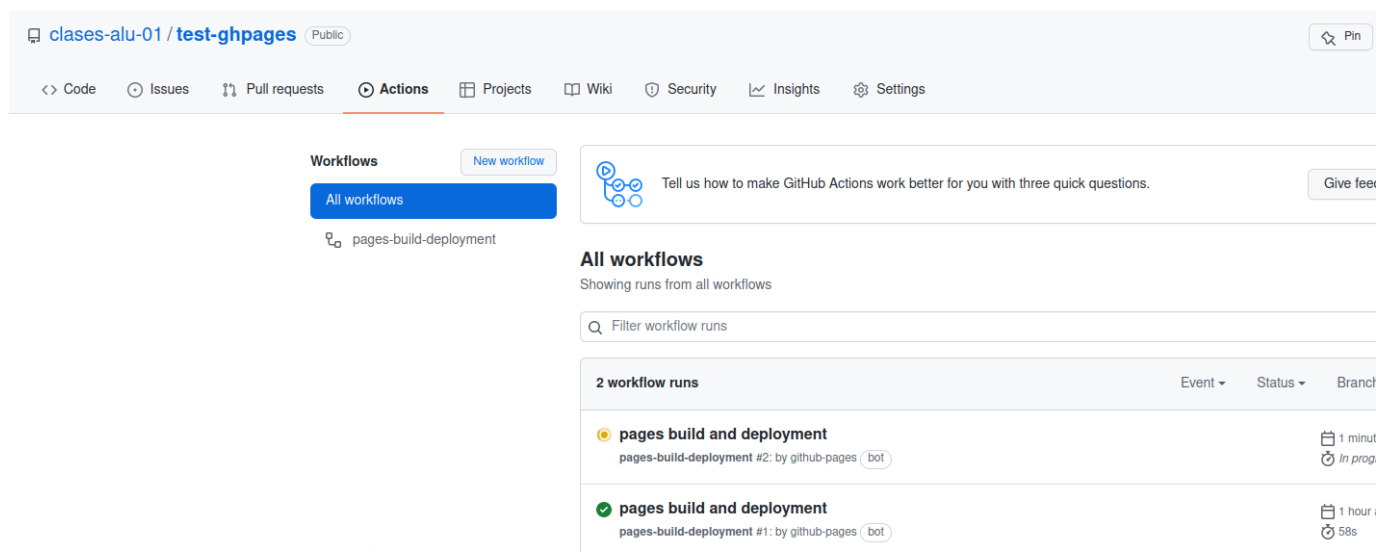
1.2.3. Publicar

Y finalmente se suben los cambios a GitHub con `git push`:

```
clases@vm:~/tmp/test-ghpages$ git push origin gh-pages
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 8 hilos
Comprimiendo objetos: 100% (4/4), listo.
Escribiendo objetos: 100% (4/4), 1.01 KiB | 1.01 MiB/s, listo.
Total 4 (delta 0), reusado 0 (delta 0)
To https://github.com/clases-alu-01/test-ghpages.git
 44c952d..f037a5e  gh-pages -> gh-pages
clases@vm:~/tmp/test-ghpages$
```

1.2.4. Probar

Tras hacer el push debe esperar a que se haga el despliegue (que se realiza con **Actions**), puede ver el progreso en dicha pestaña:



The screenshot shows the GitHub repository interface for 'clases-alu-01 / test-ghpages'. The 'Actions' tab is active, displaying a list of workflows. The workflow 'pages build and deployment' is highlighted, showing its status as 'In progress'.

Workflows

- [All workflows](#)
- [pages-build-deployment](#)

All workflows

Showing runs from all workflows

Filter workflow runs

2 workflow runs	Event	Status	Branch
pages build and deployment pages-build-deployment #2: by github-pages (bot)		In progress	
pages build and deployment pages-build-deployment #1: by github-pages (bot)		Completed	

Y una vez terminado ver el resultado en el navegador:



Ampliación: si quiere ver los detalles de cómo se ha construido el sitio web puede acceder a la pestaña Actions, hacer clic sobre el workflow más reciente (con el título *pages build and deployment*) y acceder al *build* donde se pueden consultar los detalles (en la figura 09 se muestra el paso "Build Page with Jekyll").

- [figuras/gh_pages_07.png](#)
- [figuras/gh_pages_08.png](#)
- [figuras/gh_pages_09.png](#)

3.1.3 1.3. Enlaces

- <https://pages.github.com/>
- <https://docs.github.com/es/pages>

3.1.4 1.4. Anexos

Sitios de usuarios

- Debe crear un repositorio cuyo identificador sea su nombre de usuario en Github seguido de `.github.io`, algo como `<usuario>.github.io`
- La ubicación del sitio será `http://<usuario>.github.io`
- La forma de publicar es similar a la mostrada previamente.

1.4.1. Ejercicio GH Pages

Crear con GitHub Pages una página de repositorio (o de usuario) en su cuenta de GitHub. Añada estilos CSS y alguna imagen a la página HTML. Guarde la URL del repositorio

1.4.2. Usando directorio `docs` en `main`

GitHub da la posibilidad de publicar páginas HTML en la rama `main` usando el directorio `docs`. Para ello:

- Crear repo `test-demo-pages`
- Activar Pages en rama `main` usando la carpeta `docs`.
- Clonar repositorio en local.
- Crear directorio `docs` y añadir y editar fichero `index.html` con `estilos.css`.
- Subir los cambios a GitHub y probar la URL que nos indica Pages que será similar a `http://<usuario>.github.io/<repositorio>`

3.2 DNS

Objetivos:

1. Conseguir un nombre de dominio sin coste.
2. Asociar ese dominio en GitHub Pages.

3.2.1 1. Dominios gratuitos

Hay varias opciones, entre ellas:

- Dynu: <https://www.dynu.com/en-US/>
- *duckdns*: Admite wildcards xxx.midominio.duckdns.org y admite registros tipo TXT. <https://duckdns.org>
- *noip.com*: <https://www.noip.com/es-MX/free>
- *FreeDNS*: <http://freedns.afraid.org>
- Freenom: <https://www.freenom.com/>

De entre las opciones posible se muestra Dynu.

3.2.2 2. Dominio en Dynu

2.1. Alta

Puede darse de alta en <https://www.dynu.com/en-US/>

2.2. Dominio DDNS

Puede añadir un dominio en el menú *DDNS*, opción *SIGN UP*. Este debería ser el enlace:

<https://www.dynu.com/en-US/ControlPanel/AddDDNS>

Debe usar la opción *Option 1: Use Our Domain Name*, seleccionar uno de sus dominios en el desplegable y elegir un Nombre de Host. Luego pulsar el botón *Add*

Dynu {d}DNS
Working hard to empower you!

Search

DDNS ▾ DOMAINS EMAIL ▾ SERVICES

Logged in

Add Dynamic DNS

Home / Control Panel / D

Option 1: Use Our Domain Name

Host
daweb

Top Level
freeddns.org

Option 2: Use Your Own Domain Name

Domain Name
mydomain.com

+ Add **✕ Cancel**

[Bulk Add](#) | [Purchase Domain Name](#) | [Transfer Domain Name](#)

Una vez creado le muestra el dominio y observe que de forma automática le ha asignado una IP a ese dominio. Esa IP es la del equipo del que se está conectando, y lo habitual es que sea la IP pública del *router* de la red desde la que se esté conectando en ese momento.

Dynu {d}DNS
Working hard to empower you!

Search

DDNS ▾ DOMAINS EMAIL ▾ SERVICE

Logg

Manage Dynamic DNS Service

Home / Control Panel / Dynamic D

daweb.freeddns.org

Last Update 12/18/2025 1:09:59 PM

IPv4 Address ?

IPv6 Address ?

IPv6 Address

Group ?

TTL (seconds) ?

120

✓ Save ✕ Cancel + New 🗑 Remove

Wildcard IPv4 Alias ?

ON

Wildcard IPv6 Alias ?

ON

Enable IPv6 Address ?

ON

Email Notification ?

OFF

DNS Records

Current Status

Zone Import

Domain Registration

3.2.3 3. Configuración Dominio en GitHub Pages

Para que su sitio web sea accesible desde un dominio de Dynu (por ejemplo, `clases.ddnsgeek.com`) necesita:

1. En Dynu: Crear un registro de tipo **CNAME** que apunte su subdominio a la dirección por defecto de GitHub (`suusuario.github.io`).
2. En GitHub: Configurar el repositorio para que acepte las peticiones que lleguen desde ese dominio

3.1. Configuración en Dynu (DNS)

Debe decirle a Dynu dónde debe enviar a los usuarios cuando escriban tu nombre de dominio:

1. Iniciar sesión en **Dynu Control Panel**.
2. Ir a **DDNS Services** y añadir su dominio (ej. `clases.ddnsgeek.com`).
3. Buscar la sección de **DNS Records** (Registros DNS).
4. Crear un nuevo registro con los valores:
 - **Type (Tipo):** CNAME
 - **Node (Host):** `www` (o déjar vacío si quieres usar el dominio raíz).
 - **Hostname:** `su-usuario.github.io` (sustituye "su-usuario" por su usuario de GitHub).
 - **TTL:** Puede dejar el valor por defecto.

Dynu {d}DNS
Working hard to empower you!

DDNS ▾ DOMAINS EMAIL ▾ SERVICES ▾ SUPPORT ▾

Logged in as angioies fr

Manage DNS Records

Home / Control Panel / Dynamic DNS Services / Manage Dynamic DNS Services

clases.ddnsgeek.com

Node Name [?] **Type** [?] A - IPv4 ▾ **TTL*** [?]

IPv4 Address [?] **Group** [?]

[+ Add DNS Record](#) [✕ Cancel](#)

Show 25 ▾ entries

Hostname [?]	Type [?]	Data [?]	TTL [?]	Actions [?]
*.clases.ddnsgeek.com	A	[REDACTED]	120	✎ ✖
*.clases.ddnsgeek.com	AAAA	[REDACTED]	120	✎ ✖
clases.ddnsgeek.com	A	[REDACTED]	120	✎
clases.ddnsgeek.com	AAAA	[REDACTED]	120	✎
clases.ddnsgeek.com	CNAME	angioies.github.io	120	✎ ✖ 🗑

Showing 1 to 5 of 5 entries [← Previous](#)

3.2. Configuración en GitHub

Ahora debe configurar GitHub:

1. Entrar en repositorio de GitHub.
2. Ir a **Settings** (Configuración) > **Pages**.
3. En la sección **Custom domain**, escribir su dominio de Dynu (ej. `clases.ddnsgeek.com`).
4. Hacer clic en **Save**.

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <http://clases.ddnsgeek.com/>

Last [deployed](#) by [angoies](#) 2 hours ago

[Visit site](#)

[Unpublish site](#)

Build and deployment

Source

Deploy from a branch ▾

Branch

Your GitHub Pages site is currently being built from the `/docs` folder in the `main` branch. [Learn more about configuring the publishing source for your site.](#)

[main](#) ▾

[/docs](#) ▾

[Save](#)

Learn how to [add a Jekyll theme](#) to your site.

Your site was last deployed to the [github pages](#) environment by the [pages build and deployment](#) workflow. [Learn more about deploying to GitHub Pages using custom workflows](#)

Custom domain

Custom domains allow you to serve your site from a domain other than `angoies.github.io`. [Learn more about configuring custom domain](#)

[clases.ddnsgeek.com](#)

[Save](#)

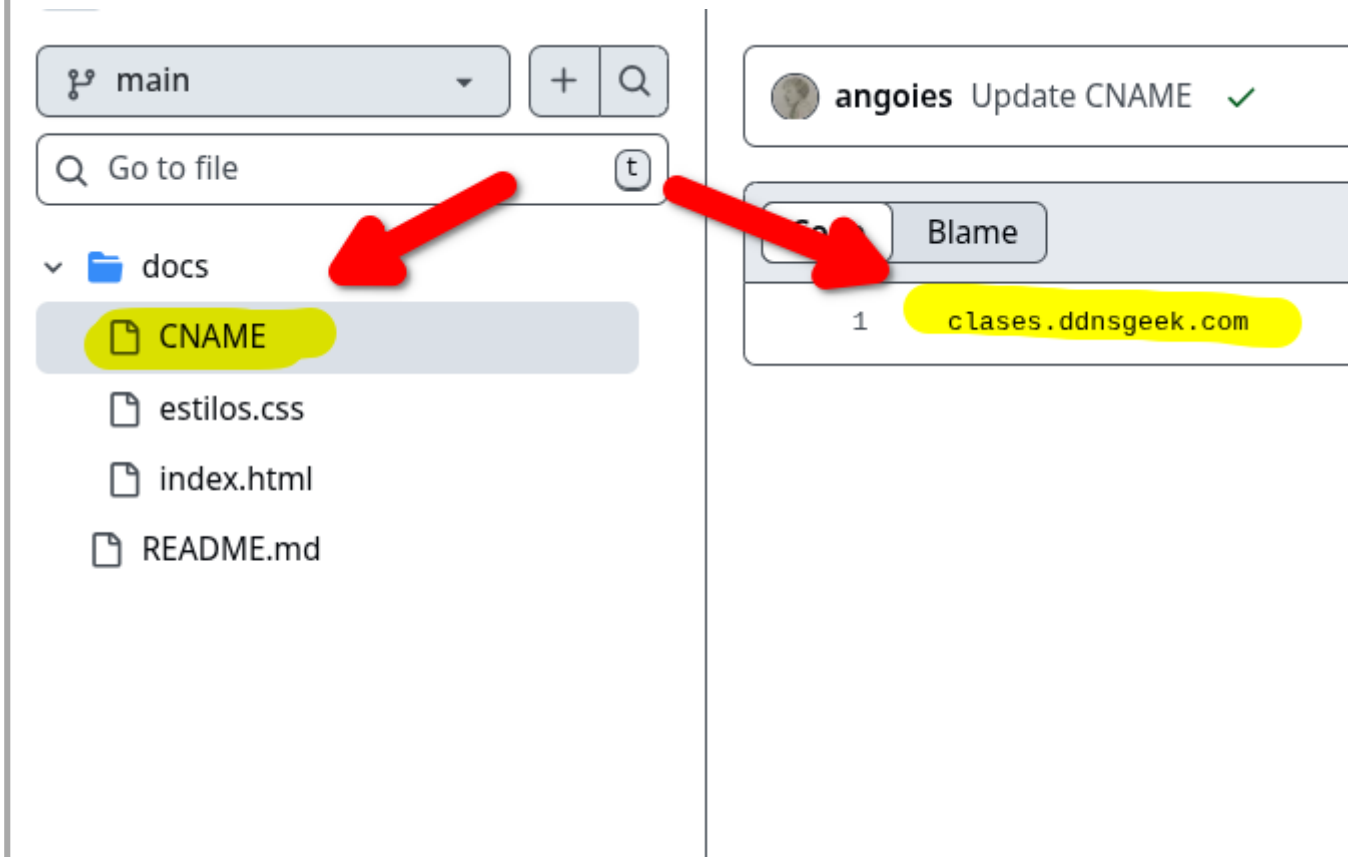
[Remove](#)

✓ DNS check successful

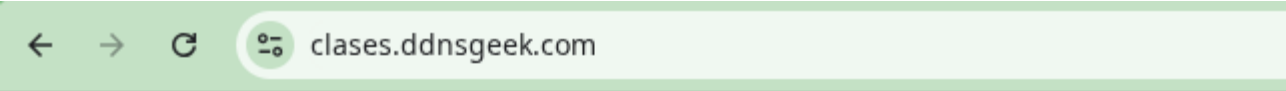
✓ [Enforce HTTPS](#) ✓

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site.

GitHub creará automáticamente un archivo llamado `CNAME` en el directorio o rama de Pages. Si prefiere hacerlo manualmente, el archivo `CNAME` es simplemente un archivo de texto sin extensión que contiene una sola línea:



Una vez guardados los cambios GitHub realiza un "DNS check". Esto puede tardar desde unos minutos hasta horas (propagación DNS). Cuando el check sea correcto podrá marcar la casilla *Enforce HTTPS*. Esto cifrará la conexión de tu sitio para que sea seguro.

clases.ddnsgeek.com

Ejemplo uso de Pages

Lorem ipsum dolor sit amet consectetur adipisicing elit. voluptatum vel sed consequatur natus ad aliquid modi praesentium vero expedita ipsa deleniti ex, maiores commodi vitae per illum!

Voluptate praesentium vero temporibus aliquam distincti voluptatibus iusto facilis, molestias, aperiam expedita re fuga laboriosam tenetur, consequatur dolores veritatis in Facere eaque ullam accusamus dignissimos officiis quas

3.3 Markdown

3.3.1 Markdown

1. Introducción

Markdown[9] is a lightweight markup language for creating formatted text using a plain-text editor. John Gruber and Aaron Swartz created Markdown in 2004 as a markup language that is appealing to human readers in its source code form.

Its key design goal was readability, that the language be readable as-is,

Ojo con las versiones. Por ejemplo la de GitHub nos dice:

1.1 What is GitHub Flavored Markdown?

GitHub Flavored Markdown, often shortened as GFM, is the dialect of Markdown that is currently supported for user content on GitHub.com and GitHub Enterprise.

This formal specification, based on the CommonMark Spec, defines the syntax and semantics of this dialect.

GFM is a strict superset of CommonMark. All the features which are supported in GitHub user content and that are not specified on the original CommonMark Spec are hence known as extensions, and highlighted as such.

While GFM supports a wide range of inputs, it's worth noting that GitHub.com and GitHub Enterprise perform additional post-processing and sanitization after GFM is converted to HTML to ensure security and consistency of the website.

2. Utilización

- Documentación (permite control de versiones)
- GitHub (publica directamente en formato Markdown) o en GitHub Pages usando `mkdocs`
- Mapas (markmap)
- Presentaciones (marp)

3. Básico

En general debe ser suficiente este conjunto de marcas:

- Cabeceras de nivel
- Párrafos
- Citas
- Listas
- Enlaces
- Imágenes
- Código
- Bloques de código
- Separadores
- strong / em

Y puede ampliar con otras marcas:

- Tablas
- Listas de tareas
- emoticonos

4. Tareas

- Crear documento Markdown. Incluir distintos tipos de marcas.
- Ampliar: añadir plugins VScode. Intentar numeración de secciones/cabeceras, añadir tabla de contenidos o imprimir a PDF.
- Ampliar: añadir plugin VScode. Crear esquema que se traduce a mapa (markmap)

5. Enlaces

5.1. TUTORIALES

- Markdown original: <https://daringfireball.net/projects/markdown/basics>
- Tutorial: <https://tutorialmarkdown.com/sintaxis>

5.2. ESPECIFICACIONES

- CommonMark Spec: <https://spec.commonmark.org/0.30/>
- Github Markdown: <https://github.github.com/gfm/#what-is-github-flavored-markdown>

5.3. UTILIDADES

- Markmap: <https://markmap.js.org/repl>
- Marp: <https://marp.app/>
- Typora: <https://typora.io/>

5.4. VSCODE PLUGINS

- Markmap: <https://marketplace.visualstudio.com/items?itemName=gera2ld.markmap-vscode>
- Marp: <https://marketplace.visualstudio.com/items?itemName=marp-team.marp-vscode>
- MD All in One: <https://marketplace.visualstudio.com/items?itemName=yizhang.markdown-all-in-one>
- Markdown to PDF: <https://marketplace.visualstudio.com/items?itemName=yizane.markdown-pdf>

3.4 mkdocs

MkDocs: Project documentation with Markdown.

MkDocs is a fast, simple and downright gorgeous static site generator that's geared towards building project documentation. Documentation source files are written in Markdown, and configured with a single YAML configuration file. <https://www.mkdocs.org>.

La idea es:

- simplificar el trabajo de documentar,
- poder hacerlo en Markdown.
- añadir de forma simple estilos, diseño adaptativo, barra de búsquedas, índices automáticos, etc.
- desplegar en Github

Pasos a dar:

1. Crear un repositorio en GitHub y clonarlo en local.
2. Instalar `mkdocs` (se usa un entorno virtual Python).
3. Usando como guía [Getting Started with MkDocs](#)
 - Crear proyecto dentro de directorio clonado
 - Lanzar mkdocs en local para ver cambios en vivo antes de subirlos a GitHub.
 - Añadir ficheros en formato Markdown
 - Modificar el fichero de configuración para que indexe los ficheros añadidos.
 - Modificar el tema por defecto
4. Una vez terminado desplegar en GitHub y ver el resultado: se activa Pages y crea una rama de nombre `gh-pages` donde sube el sitio web generado.

3.4.1 1. Crear repositorio y clonar

1. Acceda a GitHub y cree un repositorio público con Fichero `README.md`.
2. Clone el repositorio en local
3. Acceda al directorio creado con el comando `cd`

Aviso: a lo largo de estos ejemplos se ha usado el acceso HTTPS a Github mediante el uso de token.

La ejecución de esos tres pasos será similar a esta:

```
alu@xd13:~/varios$ git clone https://github.com/angoies/Demo-mkdocs.git
Clonando en 'Demo-mkdocs'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Recibiendo objetos: 100% (3/3), listo.
alu@xd13:~/varios$ cd Demo-mkdocs/
alu@xd13:~/varios/Demo-mkdocs$
```

3.4.2 2. Instalar mkdocs

Se utiliza un entorno virtual Python:

```
alu@xd13:~/varios/Demo-mkdocs$ python3 -m venv venv
alu@xd13:~/varios/Demo-mkdocs$ source venv/bin/activate
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

sobre el que se instala `mkdocs`:

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ pip install mkdocs
Collecting mkdocs
  Downloading mkdocs-1.6.1-py3-none-any.whl.metadata (6.0 kB)
...
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

Aviso: ver anexo si no está instalado `venv` o Python

3.4.3 3. Crear el proyecto

Para crear un proyecto de documentación usamos el comando `mkdocs new <dir>`, que crea un directorio para ese proyecto y dentro un fichero de configuración `mkdocs.yml`, un directorio `docs` y dentro de este un fichero `index.md` inicial. En `docs` es donde crearemos y editaremos nuestros ficheros en Markdown.

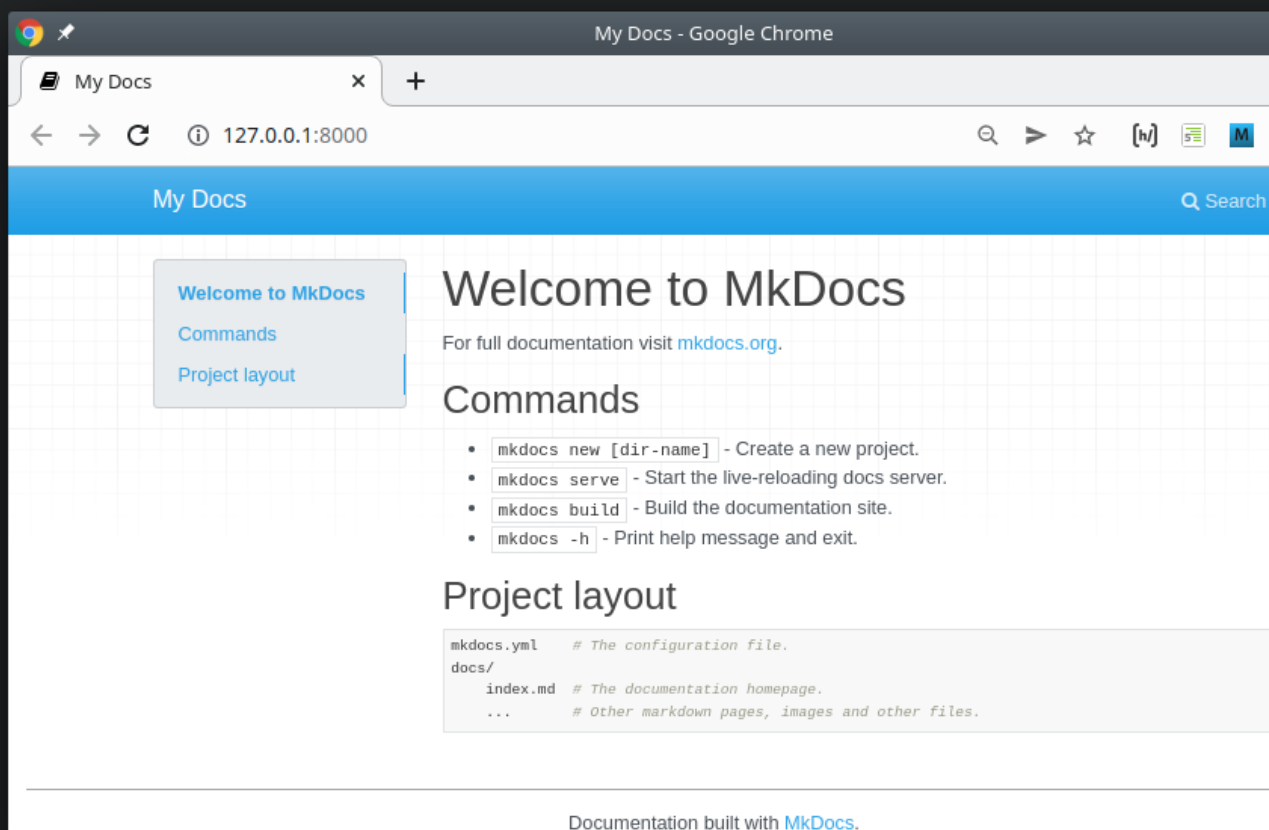
En este caso, al estar ya creado el directorio del proyecto, se usa el directorio actual como destino `mkdocs new .`:

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ mkdocs new .
INFO    - Writing config file: ./mkdocs.yml
INFO    - Writing initial docs: ./docs/index.md
(venv) alu@xd13:~/varios/Demo-mkdocs$ ls -al
total 28
drwxrwxr-x 5 alu alu 4096 mar 26 11:30 .
drwxrwxr-x 6 alu alu 4096 mar 26 11:21 ..
drwxrwxr-x 2 alu alu 4096 mar 26 11:30 docs
drwxrwxr-x 8 alu alu 4096 mar 26 11:21 .git
-rw-rw-r-- 1 alu alu  19 mar 26 11:30 mkdocs.yml
-rw-rw-r-- 1 alu alu  55 mar 26 11:21 README.md
drwxrwxr-x 5 alu alu 4096 mar 26 11:23 venv
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

3.4.4 4. Modo desarrollo

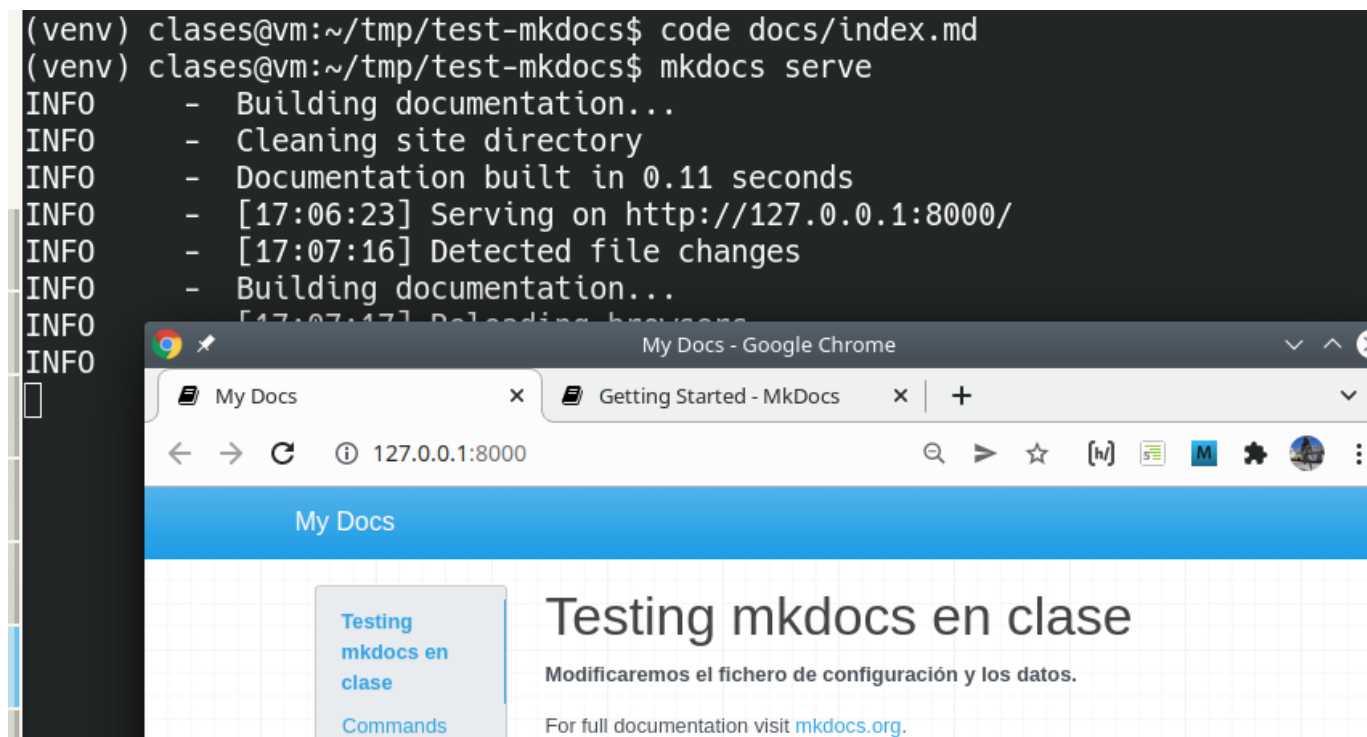
Con el comando `mkdocs serve` se inicia un servidor web de desarrollo (en la URL `http://127.0.0.1:8000/`) que muestra el resultado de mkdocs en formato web:

```
(venv) clases@vm:~/tmp/test-mkdocs$ mkdocs serve
INFO      - Building documentation...
INFO      - Cleaning site directory
INFO      - Documentation built in 0.06 seconds
INFO      - [16:53:50] Serving on http://127.0.0.1:8000/
INFO      - [16:53:54] Browser connected: http://127.0.0.1:8000/
```



El servidor soporta auto-reload, y reconstruye la documentación si hay cambios en el fichero de configuración o en el directorio de documentación.

Por ejemplo, si edita `index.md` y cambia el título de dicha página `mkdocs serve` lo detecta (`INFO - [17:07:16] Detected file changes`) y reconstruye la documentación (`INFO - Building documentation...`):



Ahora el proceso quedaría:

1. Preparar la documentación en ficheros Markdown, figuras, enlaces, etc.
2. Subirla a un hosting, en esta práctica GitHub.

3.4.5 5. Crear la documentación

`mkdocs` convertirá sus ficheros markdown en ficheros HTML y les añadirá estilos CSS y comportamiento dinámico con javascript. Crea índices con las cabeceras de nivel que encuentra en los ficheros fuente. Y genera una barra de navegación que permite navegar secuencialmente. Por si fuera poco incorpora un buscador y un comportamiento adaptativo.

Si tiene varios ficheros que desea referenciar en la barra de navegación hay que hacerlo en el fichero `mkdocs.yml`, apartado `nav`:

```
site_name: Ejemplo uso mkdocs
site_url: http://local.com
nav:
  - Home: index.md
  - Ejercicios:
    - E01: ejercicio_01.md
    - E02: ejercicio_02.md
  - Sobre nosotros: about.md
theme: readthedocs
```

5.1. Temas

Si quiere modificar el tema (por defecto utiliza uno llamado `mkdocs`) debe usar el fichero de configuración e indicar otro tema. En este caso se usa el tema `readthedocs` ya incluido en `mkdocs`.

Hay temas adicionales disponibles [en este enlace](#)

5.2. Build

Una vez haya terminado de trabajar con su documentación podría hacer un `build` de la misma. Esto genera un directorio `site` con todo el HTML, estilos, javascript, imágenes, fuentes, etc:

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ mkdocs build
INFO - Cleaning site directory
INFO - Building documentation to directory: /home/alu/varios/Demo-mkdocs/site
```

```
INFO - Documentation built in 0.04 seconds
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ ls -l site/
total 44
-rw-rw-r-- 1 alu alu 5305 mar 26 11:36 404.html
drwxrwxr-x 2 alu alu 4096 mar 26 11:36 css
drwxrwxr-x 2 alu alu 4096 mar 26 11:36 img
-rw-rw-r-- 1 alu alu 7058 mar 26 11:36 index.html
drwxrwxr-x 2 alu alu 4096 mar 26 11:36 js
drwxrwxr-x 2 alu alu 4096 mar 26 11:36 search
-rw-rw-r-- 1 alu alu 109 mar 26 11:36 sitemap.xml
-rw-rw-r-- 1 alu alu 127 mar 26 11:36 sitemap.xml.gz
drwxrwxr-x 2 alu alu 4096 mar 26 11:36 webfonts
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

3.4.6 6. Despliegue

Si quisiera desplegarlo en un hosting sólo tendría que subir el directorio `site`. Pero nuestro objetivo es desplegarlo sobre GitHub Pages, y eso lo hace de una forma muy sencilla `mkdocs`. Y es lo que se presenta en este apartado.

Antes de continuar y para evitar por error que el entorno virtual (el directorio `venv`) sea parte del repositorio se recomienda crear el fichero `.gitignore`. Este contendrá la lista de los ficheros y directorios que no queremos que formen parte del repositorio.

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ nano .gitignore
(venv) alu@xd13:~/varios/Demo-mkdocs$ cat .gitignore
venv/
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

Ahora puede hacer el *deploy* de la documentación con el comando `mkdocs gh-deploy`. Este comando hace el *build* de la documentación, la mueve a una rama `gh-pages` y hace un *push* al remoto:

```
(venv) alu@xd13:~/varios/Demo-mkdocs$ mkdocs gh-deploy
INFO - Cleaning site directory
INFO - Building documentation to directory: /home/alu/varios/Demo-mkdocs/site
INFO - Documentation built in 0.05 seconds
WARNING - Version check skipped: No version specified in previous deployment.
INFO - Copying '/home/alu/varios/Demo-mkdocs/site' to 'gh-pages' branch and pushing to GitHub.
Enumerando objetos: 37, listo.
Contando objetos: 100% (37/37), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (36/36), listo.
Escribiendo objetos: 100% (37/37), 827.52 KiB | 11.33 MiB/s, listo.
Total 37 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'gh-pages' on GitHub by visiting:
remote:   https://github.com/angoies/Demo-mkdocs/pull/new/gh-pages
remote:
To https://github.com/angoies/Demo-mkdocs.git
* [new branch]      gh-pages -> gh-pages
INFO - Your documentation should shortly be available at: https://angoies.github.io/Demo-mkdocs/
(venv) alu@xd13:~/varios/Demo-mkdocs$
```

Además nos informa que en breve (*shortly*) la documentación estará disponible en la URL `https://angoies.github.io/Demo-mkdocs/`

Nuevamente el proceso se realiza con Actions, puede comprobarlo en la pestaña Actions del repositorio:

The screenshot shows the GitHub Actions interface for the repository 'angoies / Demo-mkdocs'. The top navigation bar includes links for Code, Issues, Pull requests, Actions (highlighted), Projects, Wiki, and Security. On the left sidebar, under the 'Actions' section, there is a 'New workflow' button and a list of workflow categories: 'All workflows' (selected), 'pages-build-deployment', 'Management', 'Caches', 'Deployments', 'Attestations', and 'Runners'. The main content area is titled 'All workflows' and shows 'Showing runs from all workflows'. Below this, it indicates '1 workflow run'. A table-like structure displays the workflow run details: a green checkmark icon, the workflow name 'pages build and deployment' (highlighted in yellow), and the runner 'gh-pages'. Below the workflow name, it says 'pages-build-deployment #1: by angoies'.

angoies / Demo-mkdocs

<> Code Issues Pull requests **Actions** Projects Wiki Security

Actions New workflow

All workflows

pages-build-deployment

Management

Caches

Deployments

Attestations

Runners

All workflows Filter

Showing runs from all workflows

1 workflow run

Event	Status	Branch	Actor
✓	pages build and deployment	gh-pages	pages-build-deployment #1: by angoies

O en la de código:

The screenshot shows the GitHub interface for the repository 'Demo-mkdocs' by user 'angoies'. The repository is public and has 0 stars, 0 watches, and 0 forks. The main content area displays the README file, which contains the title 'Demo-mkdocs' and the description 'Repositorio de ejemplo del uso de mkdocs'. The right sidebar shows the 'About' section, 'Releases' (with a link to 'Create new release'), and 'Deployments' (with a deployment named 'github-pages' from 3 minutes ago). Three red arrows with numbers 1, 2, and 3 point to specific elements: arrow 1 points to the repository name 'Demo-mkdocs', arrow 2 points to the 'Create new release' link, and arrow 3 points to the 'github-pages' deployment entry.

angoies / Demo-mkdocs

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Public Pin Watch 0 Fork 0 Star 0

main Go to file + <> Code

angoies Initial commit 251a9ee · 27 minutes ago

README.md Initial commit 27 minutes ago

README

Demo-mkdocs

Repositorio de ejemplo del uso de mkdocs

About

Repositorio de ejemplo del uso de mkdocs

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published

[Create new release](#)

Deployments 1

github-pages 3 minutes ago

Y ver la URL

angoies / Demo-mkdocs

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Deployments

All deployments

Environments

github-pages

Manage environments

github-pages deployments

Latest deployments

✓ **github-pages**

Last deployed 6 minutes ago

<https://angoies.github.io/Demo-mkdocs/>

Filter Filter deployments

1 deployment

✓ **Deployed 251a9ee ...** Active

Deployed to **github-pages** by angoies via pages-build-deployment #1 gh-pages

3.4.7.7. Anexos

7.1. Enlaces

- Mkdocs: <https://www.mkdocs.org/>

7.2. Instalación mkdocs

Debe tener instalado previamente:

- python3
- python-venv

Si no los tiene instalados:

```
# apt install python3
# apt install python3-venv
```

Luego cree un entorno virtual en el repositorio:

```
$ python3 -m venv mivenv
```

Ese comando crea un directorio de nombre `mienv`. Añadir el entorno virtual al fichero `.gitignore`:

```
$ echo "mienv" >> .gitignore
$ echo ".gitignore" >> .gitignore
```

Activar el entorno virtual creado en `mienv`:

```
$ source mienv/bin/activate
(mienv) $
```

Observe como ahora el prompt muestra entre paréntesis el nombre del entorno virtual.

Ahora instale el modulo `mkdocs` con la utilidad `pip` en el entorno virtual:

```
(mienv) $ pip install mkdocs
...
(mienv) $
```

Y ya puede usar `mkdocs`:

```
(mienv) $ mkdocs
```

Al terminar de trabajar con el entorno:

```
(mienv) $ deactivate
$
```

Una vez fuera del entorno virtual no es posible acceder a `mkdocs`

```
$ mkdocs
bash: mkdocs: orden no encontrada
$
```

Si desea eliminar el entorno virtual basta con borrar el directorio `mienv`

```
$ rm -rf mienv
$
```

7.3. Instalación mkdocs Windows

Pasos a dar:

- Instalar python (y quizás haga falta pip, opcional venv)
- Descargar <https://www.python.org/>
- Instalar seleccionando en primera pantalla "Add Python.exe to PATH"
- Probar instalación:

```
User@WinDev2004Eval MINGW64 ~
$ python --version
Python 3.12.2

User@WinDev2004Eval MINGW64 ~
$ pip --version
pip 24.0 from C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip (python 3.12)

User@WinDev2004Eval MINGW64 ~
$
```

- Instalar mkdocs :

```
User@WinDev2004Eval MINGW64 ~
$ pip install mkdocs
Collecting mkdocs
Downloading mkdocs-1.5.3-py3-none-any.whl.metadata (6.2 kB)

...
Installing collected packages: ...
Successfully installed ....

User@WinDev2004Eval MINGW64 ~
$
```

Y probar la instalación

```
User@WinDev2004Eval MINGW64 ~
$ mkdocs.exe --version
mkdocs, version 1.5.3 from C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\mkdocs (Python 3.12)

User@WinDev2004Eval MINGW64 ~
$
```

4. Formatos

4.1 1. YAML. TOML y JSON

Aunque no son exactamente lo mismo, hemos agrupado en este documento estos tres formatos de datos.

4.1.1 1.1. JSON

JSON (JavaScript Object Notation - Notación de Objetos de JavaScript) es un formato ligero de intercambio de datos. Leerlo y escribirlo es simple para humanos, mientras que para las máquinas es simple interpretarlo y generarlo. Está basado en un subconjunto del Lenguaje de Programación JavaScript, Standard ECMA-262 3rd Edition - Diciembre 1999. JSON es un formato de texto que es completamente independiente del lenguaje pero utiliza convenciones que son ampliamente conocidos por los programadores de la familia de lenguajes C, incluyendo C, C++, C#, Java, JavaScript, Perl, Python, y muchos otros. Estas propiedades hacen que JSON sea un lenguaje ideal para el intercambio de datos.

JSON está constituido por dos estructuras:

- Una colección de pares de nombre/valor. En varios lenguajes esto es conocido como un objeto, registro, estructura, diccionario, tabla hash, lista de claves o un arreglo asociativo.
- Una lista ordenada de valores. En la mayoría de los lenguajes, esto se implementa como arreglos, vectores, listas o secuencias.

Estas son estructuras universales; virtualmente todos los lenguajes de programación las soportan de una forma u otra. Es razonable que un formato de intercambio de datos que es independiente del lenguaje de programación se base en estas estructuras.

1.1.1. Introducción

El ejemplo de uso más habitual de JSON es en el uso de APIs web. Estas APIs ofrecen su servicios en formato JSON o XML. Desde diferentes lenguajes de programación puede lanzar peticiones a estas APIs y procesar el resultado.

Si por ejemplo se recibe esta cadena desde un servidor web:

```
'{"name": "John", "age": 30, "city": "New York"}'
```

en JavaScript puede usar la función `JSON.parse` para convertirla en un objeto JavaScript y usar el objeto en su código:

```
const obj = JSON.parse('{"name": "John", "age": 30, "city": "New York"}');

dest = document.getElementById("demo")
dest.innerHTML = obj.name + ", " + obj.age;
```

1.1.2. Ejemplos de APIs

Observe como ejemplo una API relacionada con "Stars Wars": <https://swapi.dev/>. Puede probar a acceder a las siguientes URLs desde su navegador:

- <https://swapi.dev/api/people/1/>
- <https://swapi.dev/api/starships/9/>

Y si quiere la salida en formato JSON:

- <https://swapi.dev/api/people/1/?format=json>
- <https://swapi.dev/api/vehicles/4/?format=json>

Aviso: parece que en el momento de usar este documento la API genera una advertencia de seguridad debido a que el certificado SSL está caducado. Puede obviar este aviso en el navegador, o us

Puede invocar LA API desde línea de comandos con la utilidad curl (en este ejemplo la salida se procesa con la utilidad json_pp para hacerla más legible:

```
$ curl -sk https://swapi.dev/api/people/2/?format=json | json_pp
{
```

```

    "birth_year" : "112BBY",
    "created" : "2014-12-10T15:10:51.357000Z",
    "edited" : "2014-12-20T21:17:50.309000Z",
    "eye_color" : "yellow",
    ...
  }
}
$

```

Otro ejemplo de uso con la API de datos ficticios <https://jsonplaceholder.typicode.com>. Puede probar a acceder a las siguientes URLs desde su navegador:

- <https://jsonplaceholder.typicode.com/photos/2>
- <https://jsonplaceholder.typicode.com/photos/>
- <https://jsonplaceholder.typicode.com/users/1>

O puede probar esta otra API relacionada con "Stars Wars", <https://swapi.dev/>:

- <https://swapi.dev/api/people/1/?format=json>
- <https://swapi.dev/api/films/3/?format=json>

Pero también puede invocarlas desde línea de comandos con la utilidad `curl`. En el caso de la primera API:

```

User@WinDev2004Eval MINGW64 ~
$ curl -s https://jsonplaceholder.typicode.com/photos/2 -s
{
  "albumId": 1,
  "id": 2,
  "title": "reprehenderit est deserunt velit ipsam",
  "url": "https://via.placeholder.com/600/771796",
  "thumbnailUrl": "https://via.placeholder.com/150/771796"
}
User@WinDev2004Eval MINGW64 ~
$

```

O en el caso de la segunda API donde:

- hay que especificar el formato de salida en la query de la URL añadiendo `?format=json`.
- y la salida de curl se procesa para hacerla más legible con la utilidad `json_pp`.

1.1.3. Uso API JSON desde Javascript

En este apartado se muestra un ejemplo de uso cómo usar los datos JSON que devuelve la API en una página web.

Para ello se usa el método `fetch()` que permite realizar llamadas AJAX (*Asynchronous JavaScript and XML*) en JavaScript.

Puede encontrar el código del ejemplo en la plataforma. Observe un fragmento del mismo donde se hace la petición, la respuesta si es correcta se procesa como JSON y se recorre el objeto devuelto para construir una tabla HTML:

```

fetch("https://jsonplaceholder.typicode.com/users/")
  .then((response) => response.json())
  .then((response) => {
    let cad = "<tr><th>Nombre</th><th>Email</th><th>Teléfono</th></tr>";
    for (let usuario of response) {
      cad += `<tr><td>${usuario.name}</td>
              <td>${usuario.email}</td>
              <td>${usuario.phone}</td></tr>`;
    }
    document.getElementById("tabla1").innerHTML = cad;
  });

```

`fetch` permite obtener los datos de forma asíncrona y no recarga la página HTML. Puede comprobarlo viendo como la hora que se muestra en el pie de página no se modifica.

Nota: En este ejemplo se ha usado `fetch` con promesas (`.then`), pero también se puede usar `fetch + async/await` o incluso una biblioteca externa como `axios`.

4.1.2 1.2. YAML

Definición:

YAML (/ˈjæməɪ/) (see § History and name) is a human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted.

Su estructura básica:

A YAML document represents a computer program's native data structure in a human readable text form. A node in a YAML document can have three basic data types:

- Scalar: Atomic data types like strings, numbers, booleans and null
- Sequence: A list of nodes
- Mapping: A map of nodes to nodes. Also known as Hashes, Hash Maps, Dictionaries or Objects. Unlike in many programming languages, a key can be more than just a string. It can be a sequence or mapping itself.

Y en castellano:

YAML fue creado bajo la creencia de que todos los datos pueden ser representados adecuadamente como combinaciones de listas, hashes (mapeos) y datos escalares (valores simples). La sintaxis es relativamente sencilla y fue diseñada teniendo en cuenta que fuera muy legible pero que a la vez fuese fácilmente mapeable a los tipos de datos más comunes en la mayoría de los lenguajes de alto nivel.

1.2.1. Sintaxis YAML

Un mapa o *hash* asocia un valor con su clave separados por `:`. En el código del siguiente ejemplo serían `invoice` o `name`, y se le han asociado valores atómicos: un entero y una cadena.

Puede a su vez haber mapas anidados, como `address` que tiene dos mapas anidados, `street` y `city`.

Y puede tener listas o secuencias como la del mapa `order_items`: (estas listas pueden admitir también la notación: `order_items: [Sled, Wrapping Paper]`).

```
invoice: 314159
name: Santa Claus
address:
  street: Santa Claus Lane
  city: North Pole

order_items:
  - Sled
  - Wrapping Paper
# order_items: [ Sled , Wrapping Paper ]
```

1.2.2. Casos de uso

1.2.2.1. MKDOCS: FICHERO DE CONFIGURACIÓN

El fichero de configuración `mkdocs.yml` de la utilidad `mkdocs` usa yaml:

```
site_name: MkLorum
site_url: https://example.com/
nav:
  - Home: index.md
  - About: about.md
```

1.2.2.2. CONTENEDORES DOCKER

Es muy frecuente usar YAML en ficheros de configuración para despliegues de sistemas, contenedores, etc.

Para describir los servicios de un despliegue de contenedores Docker se usa un fichero llamado `docker-compose.yml`. En el siguiente ejemplo el fichero lanza dos contenedores, uno con la base de datos `mariadb` y otro con la aplicación `wordpress`.

```
services:
  db:
    image: mariadb:10.6.4-focal
    command: '--default-authentication-plugin=mysql_native_password'
    volumes:
```

```

- db_data: /var/lib/mysql
restart: always
environment:
- MYSQL_ROOT_PASSWORD=somewordpress
- MYSQL_DATABASE=wordpress
- MYSQL_USER=wordpress
- MYSQL_PASSWORD=wordpress
expose:
- 3306
- 33060
wordpress:
image: wordpress:latest
volumes:
- wp_data: /var/www/html
ports:
- 80:80
restart: always
environment:
- WORDPRESS_DB_HOST=db
- WORDPRESS_DB_USER=wordpress
- WORDPRESS_DB_PASSWORD=wordpress
- WORDPRESS_DB_NAME=wordpress
volumes:
db_data:
wp_data:

```

1.2.2.3. GITHUB ACTION

Observe el contenido de una *Action* en GitHub que puede ser similar:

```

name: CI/CD
on: [push] # Compact Flow Style Sequence
jobs:
flake8-lint:
name: Lint with flake8
runs-on: ubuntu-latest
steps:
- name: Check out source repository
uses: actions/checkout@v3
- name: Set up Python environment
uses: actions/setup-python@v4
with:
python-version: "3.11"
- name: flake8 Lint
uses: py-actions/flake8@v2

```

4.1.3 1.3. TOML

TOML (Tom's Obvious, Minimal Language) es un formato de configuración diseñado para ser:

- Obvio y fácil de leer para humanos,
- Fácil de mapear a estructuras de datos típicas como diccionarios, listas, números, cadenas, etc.

Fue creado como alternativa a JSON, YAML, etc. con un enfoque en configuración de aplicaciones. TOML nace con estos objetivos:

- Ser lo suficientemente expresivo para representar configuraciones complejas.
- Mantener una sintaxis estricta y coherente, evitando ambigüedades.
- Ofrecer tipos de datos explícitos (fechas, booleanos, números, listas, etc.).
- Ser más predecible y más fácil de parsear que YAML o INI.
- Configuración de proyectos: en Python (Poetry, Black,...), en Go, JavaScript (Deno), etc.
- Configuración de herramientas y librerías en Rust, como Cargo.

1.3.1. Sintaxis básica de TOML

Clave-valor y comentarios

```

nombre = "Laura"
edad = 30
activo = true

```

Tablas (o bloques de configuración). Se indican entre corchetes [], como si fueran secciones.

```

# Configuración de usuario
[usuario]

```

```

nombre = "Laura"
edad = 30

# Subtablas (anidadas). Se usan

[servidor]
host = "localhost"
puerto = 8000

[servidor.seguridad]
https = true

```

Listas

```
lenguajes = ["Python", "JavaScript", "Go"]lenguajes = ["Python", "JavaScript", "Go"]
```

1.3.2. Ejemplo

Vea un fichero de ejemplo. En este caso centraliza configuraciones de herramientas, y elimina la necesidad de archivos de para cada una de ellas como .flake8, mypy.ini, setup.py, etc. En Python es un estándar promovido por PEP 518 y PEP 621

```

[tool.black]
line-length = 88
target-version = ['py38']
skip-string-normalization = true

[tool.isort]
profile = "black"
line_length = 88
known_first_party = ["mi_paquete"]
known_third_party = ["requests", "pandas"]

[tool.flake8]
max-line-length = 88
[tool.black]
line-length = 88
target-version = ['py38']
skip-string-normalization = true

[tool.isort]
profile = "black"
line_length = 88
known_first_party = ["mi_paquete"]
known_third_party = ["requests", "pandas"]

[tool.flake8]
max-line-length = 88
extend-ignore = ["E203", "W503"]
exclude = ["build", "dist", ".venv"]

```

4.1.4 1.4. Enlaces

1.4.1. Enlaces YAML

- Sitio oficial de YAML: <https://yaml.org/>
- YAML en Wikipedia: <https://es.wikipedia.org/wiki/YAML>
- Aprende YAML:
- YAML tutorial: Get started in 5 minutes <https://www.educative.io/blog/yaml-tutorial>
- Yaml info: <https://www.yaml.info/learn/index.html>
- Aprende X en Y minutos: <https://learnxinyminutes.com/docs/es-es/yaml-es/>

1.4.2. Enlaces JSON

- JSON: <http://www.json.org/json-es.html>
- Wikipedia: <https://es.wikipedia.org/wiki/JSON>
- Tutoriales:
- W3Schools JSON: https://www.w3schools.com/js/js_json_intro.asp
- JSON tutorial: <https://beginnersbook.com/2015/04/json-tutorial/>
- APIs para pruebas:

- The Star Wars API: <https://swapi.dev/>
- API con datos ficticios: <https://jsonplaceholder.typicode.com>
- Fetch: Peticiones Asíncronas: <https://lenguajejs.com/javascript/peticiones-http/fetch/>

1.4.3. Enlaces TOML

- TOML.io – <https://toml.io/en/> Documentación oficial. Sitio oficial del lenguaje TOML. Incluye la especificación técnica, ejemplos y preguntas frecuentes.
- PEP 518 – pyproject.toml <https://peps.python.org/pep-0518/> Explica cómo pyproject.toml se ha convertido en el estándar para definir configuraciones en proyectos Python.

4.2 Linting

4.2.1 1. YAML

1.1. YAML (Genéricas)

Estas herramientas verifican que el archivo esté bien formado (llaves, corchetes, espacios y tipos de datos).

- **yamllint**: No solo busca errores de sintaxis, sino que comprueba reglas de estilo (longitud de línea, espacios sobrantes, etc.)
- *Instalación*: `pip install yamllint`
- *Uso*: `yamllint mi_fichero.yaml`
- **Online YAML Lint**: Sin instalar nada, yamllint.com es la opción clásica.

1.2. YAML Específica: Compose de docker

Puede usar el propio comando de Docker Compose para validar la sintaxis y la configuración del fichero. Dependiendo de la versión, sería:

```
docker compose config
```

O

```
docker compose -f docker-compose.yml config
```

Este comando analiza el fichero, muestra la configuración resultante y detecta errores de sintaxis o configuración. Aquí no solo importa que el YAML sea válido, sino que cumpla con la **especificación de Docker** (nombres de imágenes, volúmenes, redes, etc.).

1.3. YAML Específica: Action de GitHub

Una de las recomendadas es **actionlint**, que es un linter especializado en workflows de GitHub Actions. Detecta errores de sintaxis, problemas de configuración y errores comunes en ficheros YAML de GitHub Actions.

- GitHub: <https://github.com/rhysd/actionlint>
- Prueba online: <https://rhysd.github.io/actionlint/>

Además, puede combinarlo con herramientas generales de validación de YAML como **yamllint**, aunque estas no están especializadas en el esquema específico de GitHub Actions.

Otra opción es utilizar **act**, que te permite simular la ejecución de los workflows localmente y así detectar posibles errores en el comportamiento, aunque su enfoque es más de simulación que de simple validación de sintaxis. <https://nektosact.com/introduction.html>

1.4. Integración en el Editor (VS Code)

Si usas Visual Studio Code puedes usar entre otras:

1. **YAML (Red Hat)**: Proporciona autocompletado y validación en tiempo real comparando tu archivo con esquemas de Kubernetes, Docker Compose y más.
2. **Docker (Microsoft)**: Añade IntelliSense específico para archivos `.yaml` de Docker, reconociendo palabras clave como `services`, `build`, o `environment`.

4.2.2 2. TOML

Para los archivos **TOML** (Tom's Obvious Minimal Language), que se han vuelto muy populares gracias a herramientas como Rust (Cargo) y Python (pyproject.toml), también existen herramientas de validación.

2.1. Herramientas basadas en Python

Si ya tiene Python instalado, puedes aprovechar su soporte nativo o librerías específicas:

- **toml-lint**: A linter for TOML files <https://github.com/yaegassy/toml-lint>
- **toml-validator**: Una herramienta sencilla de línea de comandos.
- **Instalación**: `pip install toml-validator`
- **Uso**: `toml-validator archivo.toml`
- **tomli** (o el módulo nativo `tomllib` en Python 3.11+): Puede validar un archivo rápidamente con un "one-liner" en la terminal:

```
python3 -c "import tomllib; tomllib.loads(open('archivo.toml').read())"
```

Si no imprime nada, es válido. Si hay error, lanzará una excepción indicando la línea.

2.2. Integración en el Editor (VS Code)

Para no depender de la terminal:

1. *Even Better TOML*: Es la extensión más completa. Ofrece:
 - Validación de sintaxis en tiempo real.
 - Autocompletado.
 - Validación de esquemas (por ejemplo, te avisa si escribes mal una clave en un `pyproject.toml`).
2. *vscode-prettier-toml*: Si ya usas Prettier, puedes añadir soporte para TOML instalando el plugin `prettier-plugin-toml`.

2.3. online

- Validate TOML syntax, format, and structure with detailed error reporting <https://noobkit.cc/toml-linter/>
- CodeBeautify: [TOML Validator](#)

4.3 Ejemplos

EJEMPLOS

1. Markdown

- [Esquema](#)
- [Slides](#)

2. YAML

- [DevOps](#)
- [Docker](#)
- [Docker WordPress](#)
- [Ejemplo](#)
- [mkdocs](#)
- [Tutorial](#)

3. JSON

Ejemplo 01:

- [HTML](#)
- [JS](#)

Ejemplo 02:

- [HTML](#)
- [JS](#)

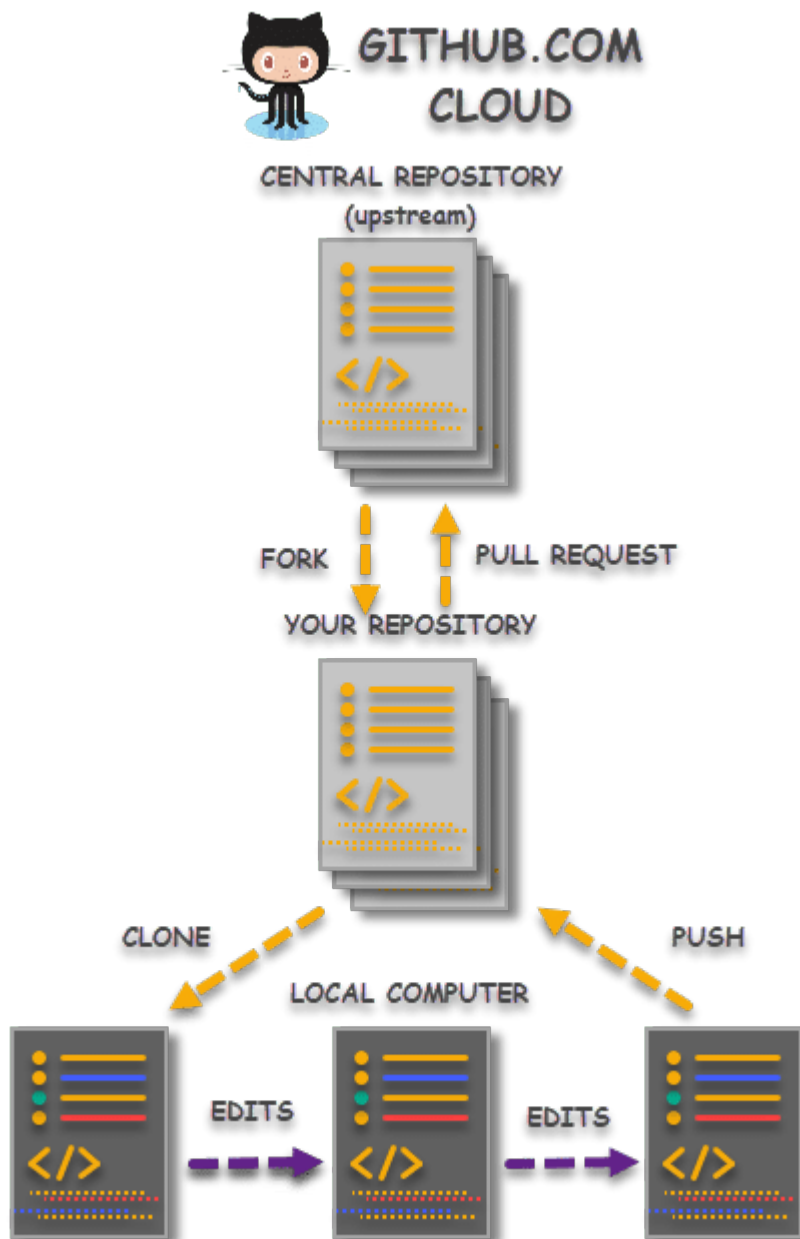
5. Amplia

5.1 Git. Trabajo en grupos: teams

5.1.1 1. Introducción al trabajo en equipo

Cuando se colabora en el desarrollo de algún proyecto de SW libre o al participar en un grupo de trabajo cerrado **se hace un fork** del repositorio para poder trabajar con él. Al hacerlo obtenemos una copia de todo el repositorio (con todo su historial) en nuestra cuenta de GitHub.

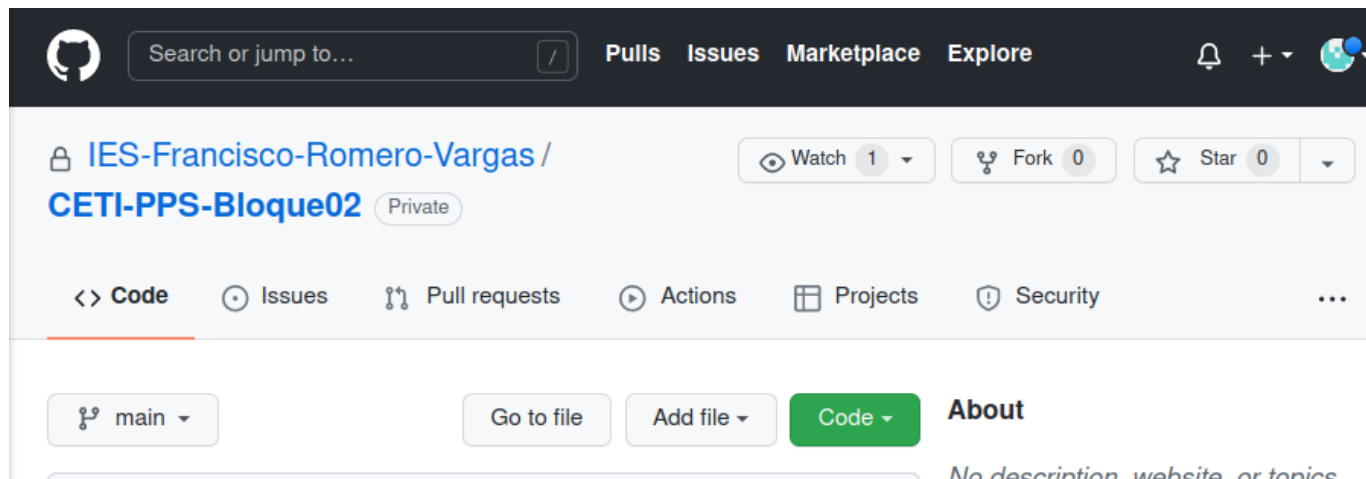
Antes de seguir con este documento es recomendable consultar este enlace: *Difference between Git Clone and Git Fork*: <https://www.toolsqa.com/git/difference-between-git-clone-and-git-fork/> donde se explica de forma gráfica la diferencia entre hacer un clone y un fork, y el proceso de colaboración a través de **Pulls Requests (PR)** que se puede resumir en esta figura:



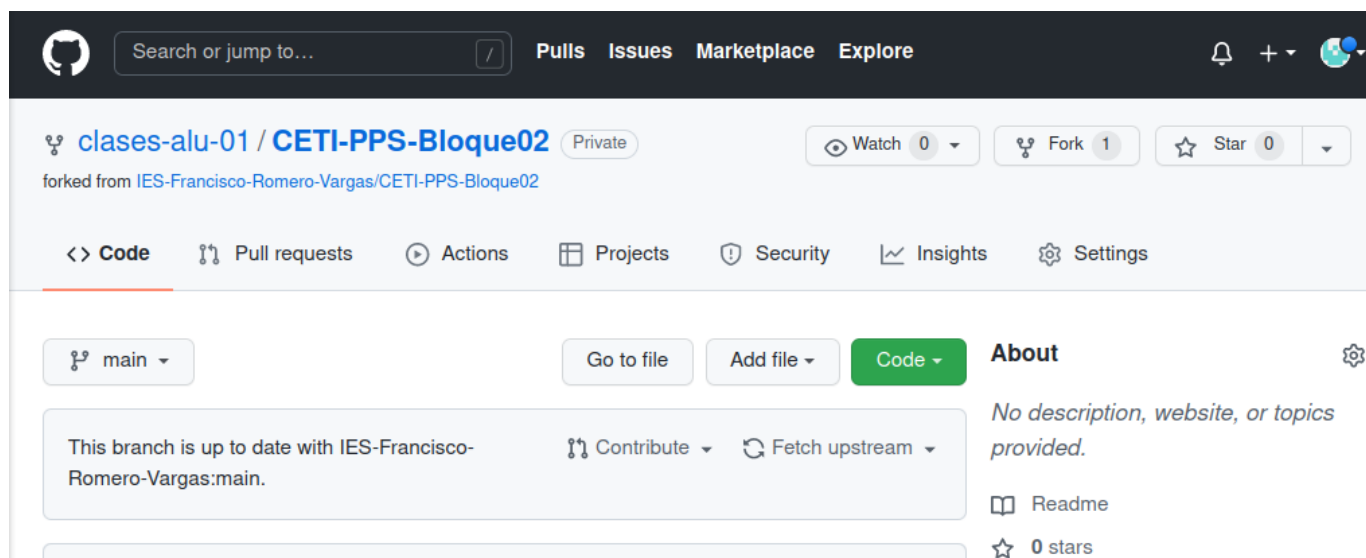
Veremos que el proceso de incorporar cambios al repositorio central o principal es a través de PR desde el directorio **forkeado**. El trabajo en local se hace tras clonar el repositorio *forkeado*.

1.1. Primer paso: fork del repositorio

Iniciamos el proceso de forma práctica. Para hacer un fork iniciamos sesión en GitHub, buscamos el repositorio principal y hacemos clic en el botón **fork** de la parte superior de la página. Esto creará una instancia del repositorio completo en nuestra cuenta.



Una vez creado el fork lo podemos ver en nuestra cuenta, con la leyenda **forked from**



1.2. Segundo paso: clonar el repositorio forkeado

Una vez que el repositorio esté en tu cuenta, puede clonarlo a su computador para trabajarlo en local. Ya conocemos esta parte:

```
$ git clone [<dir coderepo>
```

```
clases@vm:~/tmp$ git clone https://github.com/clases-alu-01/CETI-PPS-Bloque02.git
Clonando en 'CETI-PPS-Bloque02'...
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
remote: Enumerating objects: 30, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 30 (delta 3), reused 26 (delta 3), pack-reused 0
Desempaquetando objetos: 100% (30/30), 653.50 KiB | 2.42 MiB/s, listo.
clases@vm:~/tmp$ ls
CETI-PPS-Bloque02
clases@vm:~/tmp$ cd CETI-PPS-Bloque02/
clases@vm:~/tmp/CETI-PPS-Bloque02$ git log --oneline
a492dd0 (HEAD -> main, origin/main, origin/HEAD) update typos
f8cf187 test borrado
```

```
bdc2f28 Primera práctica Bloque 02 - Git
ef0ee73 Initial commit
clases@vm:~/tmp/CETI-PPS-Bloque02$
```

Si lo desea puede modificar la configuración o iniciar la configuración de trabajo en local. Por ejemplo

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ git config --local user.name "Clases Alu 01"
clases@vm:~/tmp/CETI-PPS-Bloque02$ git config --local user.email "clases.alu+01@gmail.com"
clases@vm:~/tmp/CETI-PPS-Bloque02$ git config --global credential.helper 'cache --timeout 7200'
```

1.3. Trabajar en local

En esta fase se modifica el código, se hacen pruebas, etc. Al final se añade al seguimiento y se hacen commits etc. Lo más correcto sería trabajar sobre una rama creada para ese nuevo código. mejora o corrección de errores:

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ git branch featureA
clases@vm:~/tmp/CETI-PPS-Bloque02$ git switch featureA
Cambiado a rama 'featureA'
clases@vm:~/tmp/CETI-PPS-Bloque02$ echo "para ejercicio de teams y PR" > test.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ git add test.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ git commit -m "añado fichero de test"
[featureA 84f935e] añado fichero de test
1 file changed, 1 insertion(+)
create mode 100644 test.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ echo "segundo fichero para que haya un par de commits" > leme.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ git add leme.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ git commit -m "añado segundo fichero"
[featureA f3fa288] añado segundo fichero
1 file changed, 1 insertion(+)
create mode 100644 leme.txt
clases@vm:~/tmp/CETI-PPS-Bloque02$ git log --oneline
f3fa288 (HEAD -> featureA) añado segundo fichero
84f935e añado fichero de test
a492dd0 (origin/main, origin/HEAD, main) update typos
f8cf187 test borrado
bdc2f28 Primera práctica Bloque 02 - Git
ef0ee73 Initial commit
clases@vm:~/tmp/CETI-PPS-Bloque02$
```

1.4. Subir los cambios a repo forkeado

Cuando ya tengamos el código terminado, podemos incorporarlo a nuestro repositorio "forkeado". Observe que `git push` especifica el destino `origin` (el directorio clonado) y la rama a subir, `featureA`

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ git push origin featureA
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
Enumerando objetos: 7, listo.
Contando objetos: 100% (7/7), listo.
Compresión delta usando hasta 8 hilos
Comprimiendo objetos: 100% (4/4), listo.
Escribiendo objetos: 100% (6/6), 653 bytes | 653.00 KiB/s, listo.
Total 6 (delta 1), reusado 0 (delta 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'featureA' on GitHub by visiting:
remote:   https://github.com/clases-alu-01/CETI-PPS-Bloque02/pull/new/featureA
remote:
To https://github.com/clases-alu-01/CETI-PPS-Bloque02.git
 * [new branch]   featureA -> featureA
clases@vm:~/tmp/CETI-PPS-Bloque02$
```

Si vamos a nuestro repositorio en GitHub se puede observar la rama `main`

The screenshot shows a GitHub repository page. At the top, there's a dark header with the GitHub logo, a search bar, and navigation links: Pull requests, Issues, Marketplace, and Explore. Below this, the repository name 'clases-alu-01 / CETI-PPS-Bloque02' is displayed, along with a 'Private' label and buttons for 'Watch' (0) and 'Fork' (1). A breadcrumb trail indicates it was forked from 'IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02'. A secondary navigation bar includes links for Code, Pull requests, Actions, Projects, Security, Insights, and Settings.

A yellow notification bar states: 'featureA had recent pushes 6 minutes ago' with a 'Compare & pull request' button. Below this, a branch selector shows 'main' with 2 branches and 0 tags. Buttons for 'Go to file', 'Add file', and 'Code' are present. A status bar indicates 'This branch is up to date with IES-Francisco-Romero-Vargas:main.' and options to 'Contribute' or 'Fetch upstream'.

The commit history section shows a commit by 'angoies' titled 'update typos' (hash a492dd0, 5 days ago, 4 commits). Below this is a file list:

File	Commit Message	Time
docs	update typos	5 days ago
.gitignore	Initial commit	5 days ago
README.md	Primera práctica Bloque 02 - Git	5 days ago

Below the file list, the 'README.md' file is highlighted. On the right side, the 'About' section notes 'No description, website provided.' and shows 0 stars, 0 watching, and 1 fork. The 'Releases' section states 'No releases published' with a link to 'Create a new release'. The 'Packages' section also states 'No packages published'.

Y la rama `featureA` con los nuevos ficheros:

Search or jump to... Pull requests Issues Marketplace Explore

clases-alu-01 / CETI-PPS-Bloque02 Private Watch 0 Fork 1

forked from IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02

<> Code Pull requests Actions Projects Security Insights Settings

featureA had recent pushes 5 minutes ago Compare & pull request

featureA 2 branches 0 tags Go to file Add file Code

This branch is 2 commits ahead of IES-Francisco-Romero-Vargas:main. Contribute Fetch upstream

File	Commit Message	Time
docs	update typos	5 days ago
.gitignore	Initial commit	5 days ago
README.md	Primera práctica Bloque 02 - Git	5 days ago
leme.txt	añado segundo fichero	2 hours ago
test.txt	añado fichero de test	2 hours ago

About
No description, website provided.
Readme
0 stars
0 watching
1 fork

Releases
No releases published
[Create a new release](#)

Packages
No packages published

Tengamos en cuenta que esta rama se ha incorporado al repositorio *forkeado*, no al inicial.

Puede observar al hacer `git push` que además de subir el código a nuestro repositorio, nos indica como iniciar un **Pull Request** a través de un enlace:

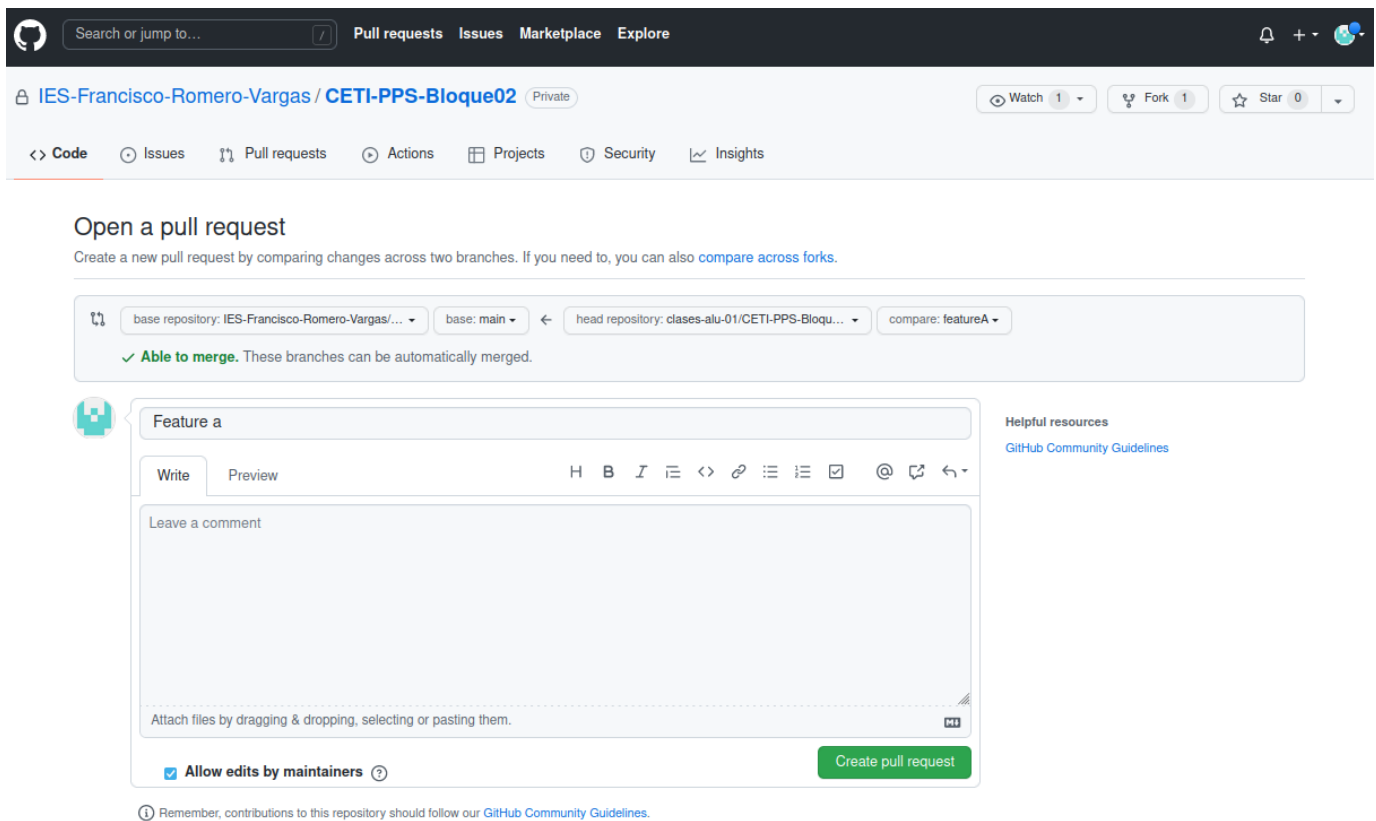
```
remote: Create a pull request for 'featureA' on GitHub by visiting:
remote:   https://github.com/clases-alu-01/CETI-PPS-Bloque02/pull/new/featureA
```

1.5. Hacer Pull Request

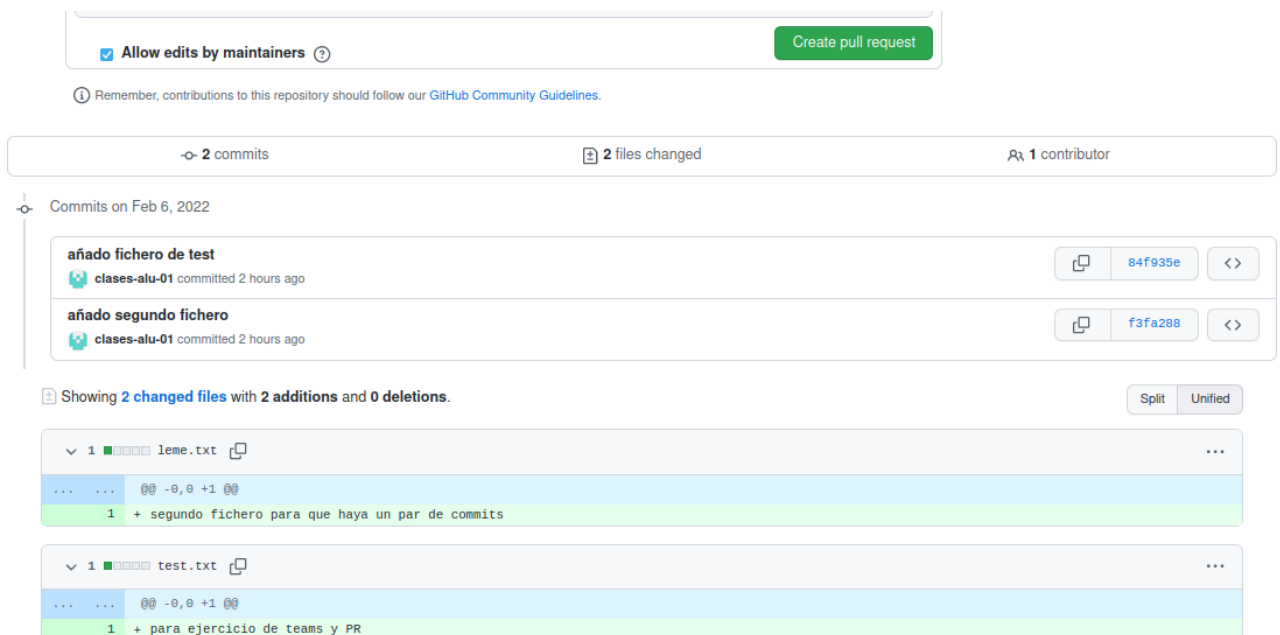
El objetivo es incorporar el código de la rama `featureA` al repositorio original (que no es nuestro). Para eso se debe hacer un **Pull Request** (PR), una petición para que los cambios se incorporen al código.

Se puede hacer el PR

- usando directamente el enlace que se ofrece al hacer `git push`
- o buscando en GitHub en el repositorio forkeado dicha opción:

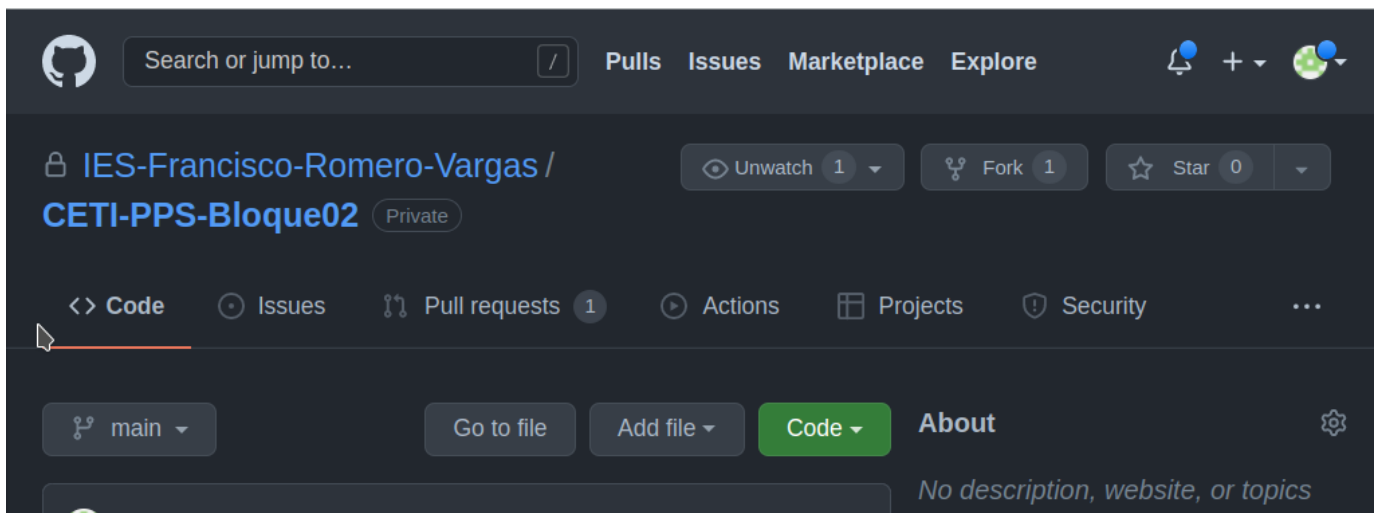


Observe que tiene la opción de añadir información en la PR: nombre de la PR, descripción, etc. Y en la siguiente figura vea que además puede revisar su propia PR ya que se muestran en la misma ventana los commits de la rama `featureA` y las diferencias entre `main` y la nueva rama:

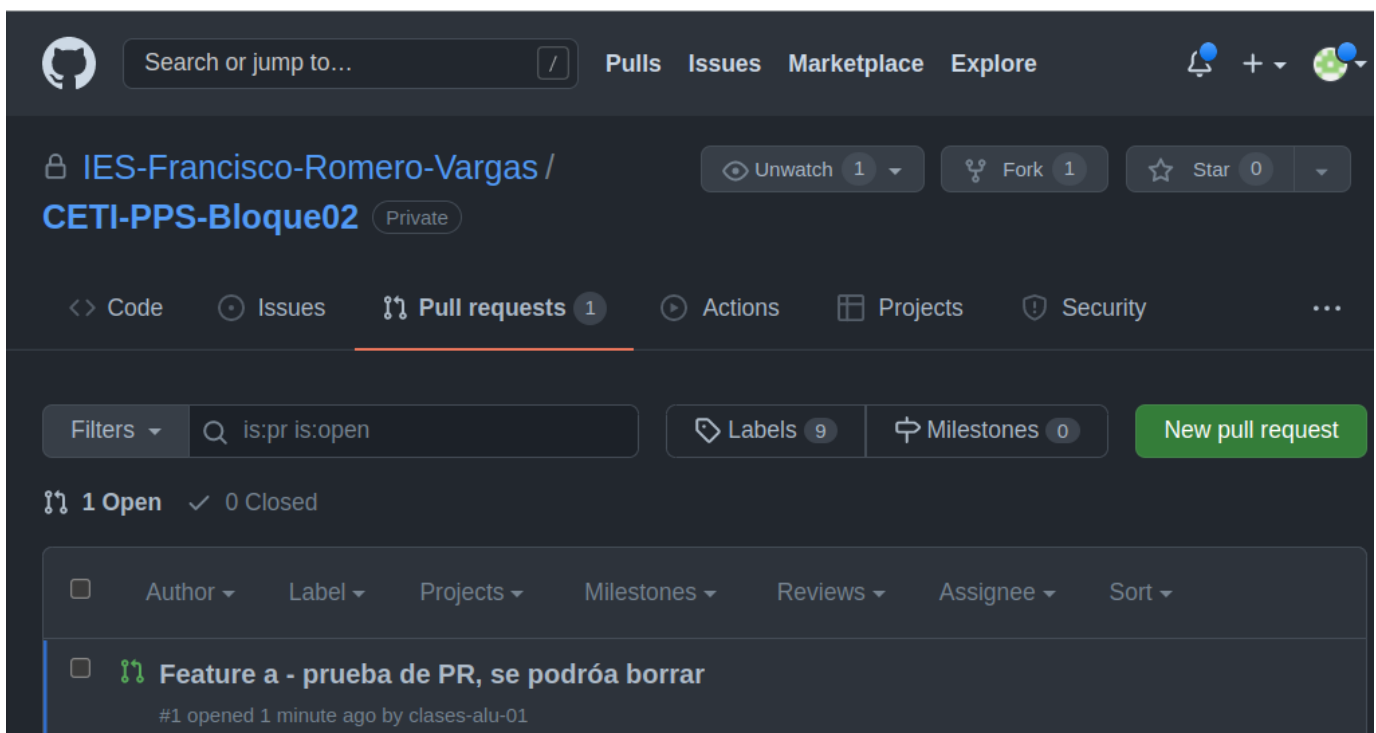


1.6. En el lado del repositorio original

Una vez realizada la PR en el directorio forkeado el propietario del repositorio original recibe la petición y puede acceder a ella a través de la pestaña **Pull Requests** que en este caso aparece con un 1 a su derecha indicando las PR pendientes:



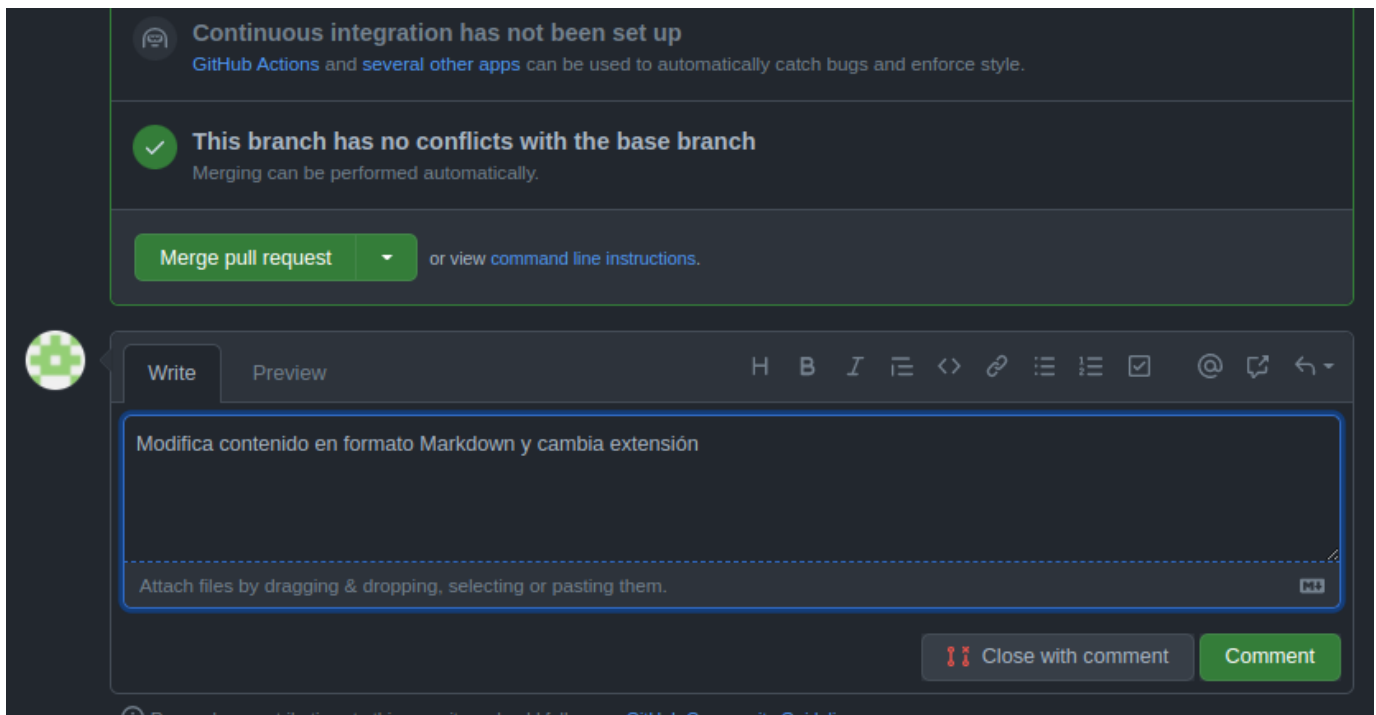
Si accedemos a las PR vemos que hay una pendiente (open):



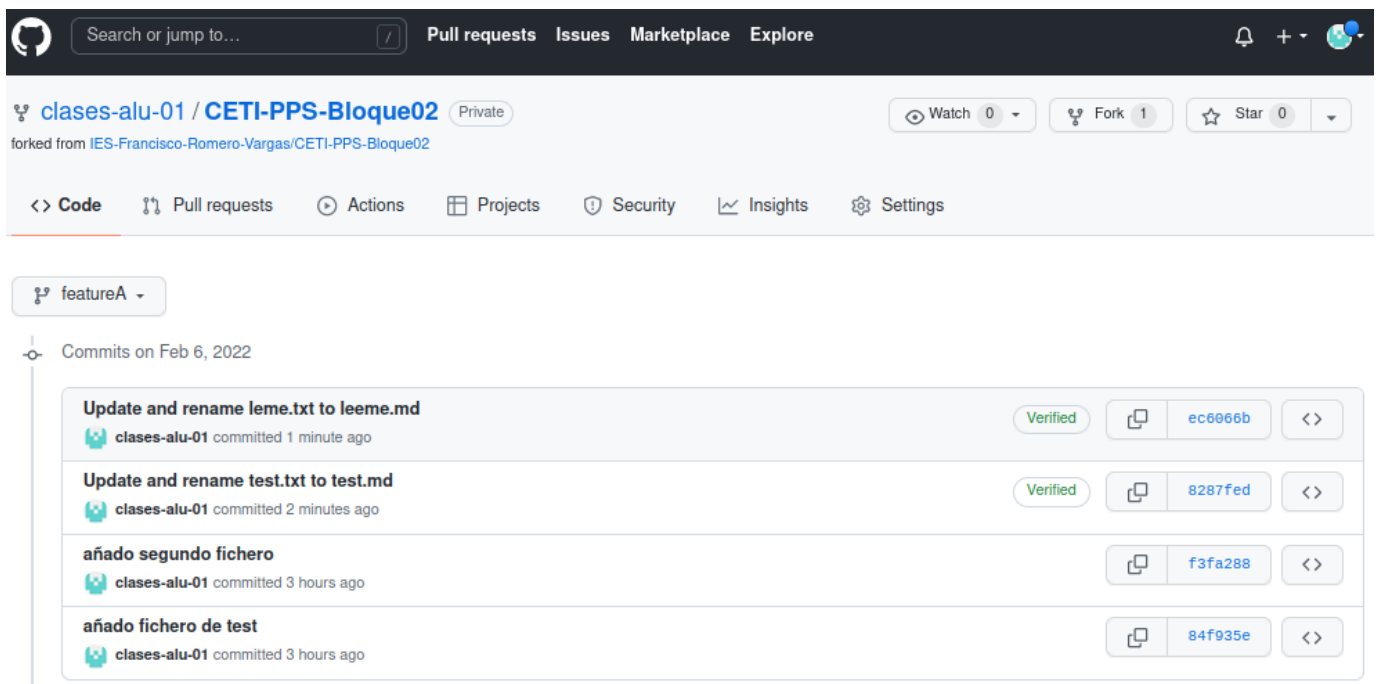
Y si accede a ella podrá ver los commits, el código, etc:

The screenshot shows a GitHub Pull Request (PR) interface. At the top, the repository is 'IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02'. The PR title is 'Feature a - prueba de PR, se podrá borrar #1'. The PR is from the 'clases-alu-01' repository, branch 'featureA', and targets the 'main' branch of the repository. The PR status is 'Open' and it shows '2 commits' and '2 files changed'. A comment from 'clases-alu-01' states: 'Añado dos ficheros txt. Con dos commits que se ven en el PR.' Below the comment, a commit history shows two commits: 'añado fichero de test' (84f935e) and 'añado segundo fichero' (f3fa288). On the right side, there are sections for 'Reviewers' (No reviews), 'Assignees' (No one—assign yourself), 'Labels' (None yet), 'Projects' (None yet), 'Milestone' (No milestone), 'Linked issues' (Successfully merging this pull request may close these issues), and 'Notifications' (Customize). At the bottom, there is a green box with a checkmark indicating 'This branch has no conflicts with the base branch' and a button to 'Merge pull request'.

En este caso en vez de aceptar directamente los cambios vamos a realizar un comentario solicitando cambios en el código.



En este caso el usuario `clases.alu+01` ve los comentarios a su PR y modifica el código y los nombres de los ficheros (directamente desde el interfaz de GitHub, creando dos nuevos commits a la rama.



Y vea que en este caso no tiene que hacer una nueva PR puesto que los cambios en la rama `featureA` son accesibles desde la pestaña de las PR:

Feature a - prueba de PR, se podrá borrar #1
 clases-alu-01 wants to merge 4 commits into IES-Francisco-Romero-Va...:main from clases-alu-01:featureA

añado segundo fichero f3fa288

angoies requested a review from informatica-iesromerovargas 17 minutes ago

angoies assigned informatica-iesromerovargas and unassigned informatica-iesromerovargas 17 minutes ago

angoies requested review from angoies and removed request for informatica-iesromerovargas 13 minutes ago

angoies commented 11 minutes ago

Modifica contenido en formato Markdown y cambia extensión

clases-alu-01 added 2 commits 8 minutes ago

Update and rename test.txt to test.md Verified 8287fed

Update and rename leme.txt to leeme.md Verified ec6066b

Add more commits by pushing to the **featureA** branch on **clases-alu-01/CETI-PPS-Bloque02**.

Review requested Show all reviewers
 Review has been requested on this pull request. It is not required to merge. [Learn more](#).

1 pending reviewer

Continuous integration has not been set up
 GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.

This branch has no conflicts with the base branch
 Merging can be performed automatically.

Merge pull request or view [command line instructions](#).

En ese momento si el propietario está de acuerdo puede hacer la fusión con **Merge Pull Request**. O puede cerrar la petición con un comentario y no aceptar los cambios.

Feature a - prueba de PR, se podrá borrar #1
 clases-alu-01 wants to merge 4 commits into IES-Francisco-Romero-Va... :main from clases-alu-01:featureA

Modifica contenido en formato Markdown y cambia extensión

clases-alu-01 added 2 commits 43 minutes ago

- Update and rename test.txt to test.md Verified 8287fed
- Update and rename leme.txt to leeme.md Verified ec6066b

angoies commented 1 minute ago

Se rechaza, era una prueba sobre repo de trabajo, mejor hacer en uno de pruebas

angoies closed this 1 minute ago

Closed with unmerged commits Delete branch

This pull request is closed, but the `clases-alu-01:featureA` branch has unmerged commits. You can delete this branch if you wish.

If you wish, you can also delete this fork of IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02 in the [settings](#).

1.7. PR que se acepta

Preparamos un escenario donde se crea una nueva rama `ejemplos_git_merge`

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ git switch main
Cambiado a rama 'main'
Tu rama está actualizada con 'origin/main'.
clases@vm:~/tmp/CETI-PPS-Bloque02$ git branch ejemplos_git_merge
clases@vm:~/tmp/CETI-PPS-Bloque02$ git switch ejemplos_git_merge
Cambiado a rama 'ejemplos_git_merge'
clases@vm:~/tmp/CETI-PPS-Bloque02$ ls
docs  README.md
clases@vm:~/tmp/CETI-PPS-Bloque02$ git log --oneline
a492dd0 (HEAD -> ejemplos_git_merge, origin/main, origin/HEAD, main) update typos
f8cf187 test borrado
bdc2f28 Primera práctica Bloque 02 - Git
ef0ee73 Initial commit
clases@vm:~/tmp/CETI-PPS-Bloque02$
```

y se incorporan tres ficheros a dicha rama: un fichero Markdown y un par de figuras:

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ code docs/01_git_intro_test.merge.md
clases@vm:~/tmp/CETI-PPS-Bloque02$ git status
En la rama ejemplos_git_merge
Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
docs/01_git_intro_test.merge.md
docs/figuras/git-merge-conflicto.png
docs/figuras/git-merge-recursive.png

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
clases@vm:~/tmp/CETI-PPS-Bloque02$ git add .
```

```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git status
En la rama ejemplos_git_merge
Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
nuevo archivo: docs/01_git_intro_test.merge.md
nuevo archivo: docs/figuras/git-merge-conflicto.png
nuevo archivo: docs/figuras/git-merge-recursive.png

clases@vm:~/tmp/CETI-PPS-Bloque02$ git commit -m "Ejemplos de casos git merge"
[ejemplos_git_merge fb2f3d8] Ejemplos de casos git merge
3 files changed, 334 insertions(+)
 create mode 100644 docs/01_git_intro_test.merge.md
 create mode 100644 docs/figuras/git-merge-conflicto.png
 create mode 100644 docs/figuras/git-merge-recursive.png
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Ahora hacemos `push` al repositorio *forkeado*:

```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git push origin ejemplos_git_merge
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
Enumerando objetos: 10, listo.
Contando objetos: 100% (10/10), listo.
Compresión delta usando hasta 8 hilos
Comprimiendo objetos: 100% (7/7), listo.
Escribiendo objetos: 100% (7/7), 222.22 KiB | 13.89 MiB/s, listo.
Total 7 (delta 2), reusado 0 (delta 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'ejemplos_git_merge' on GitHub by visiting:
remote:   https://github.com/clases-alu-01/CETI-PPS-Bloque02/pull/new/ejemplos_git_merge
remote:
To https://github.com/clases-alu-01/CETI-PPS-Bloque02.git
 * [new branch]      ejemplos_git_merge -> ejemplos_git_merge
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Ahora nos queda hacer el PR (tenemos el enlace en la salida del comando `git push`) y que el propietario lo acepte. De hecho sin usar el enlace al acceder a GitHub nos ofrece hacer el PR:

The screenshot shows the GitHub interface for the repository `clases-alu-01 / CETI-PPS-Bloque02`. The repository is private and was forked from `IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02`. The `Pull requests` tab is active. A yellow notification box indicates that the branch `ejemplos_git_merge` has recent pushes. A green button labeled `Compare & pull request` is visible. The `About` section is currently empty. At the bottom, the `main` branch is selected, and there are buttons for `Go to file`, `Add file`, and `Code`.

Accedemos a la ventana de creación de PR:

The screenshot shows the GitHub 'Open a pull request' page. At the top, the navigation bar includes the GitHub logo, a search bar, and links for 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. Below this, the repository path 'IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02' is displayed, along with 'Watch 1', 'Fork 1', and 'Star 0' buttons. A secondary navigation bar contains links for 'Code', 'Issues', 'Pull requests' (which is highlighted), 'Actions', 'Projects', 'Security', and 'Insights'.

The main heading is 'Open a pull request', followed by a subtext: 'Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).' Below this is a comparison bar with dropdowns for 'base repository: IES-Francisco-Romero-Vargas/...', 'base: main', 'head repository: clases-alu-01/CETI-PPS-Bloque02', and 'compare: ejemplos_git_merge'. A green checkmark indicates 'Able to merge. These branches can be automatically merged.'

The main content area is titled 'Ejemplos de casos git merge' and features a 'Write' tab and a 'Preview' tab. The 'Write' tab has a rich text editor with a toolbar containing icons for bold, italic, link, and other formatting options. Below the editor is a large text area for comments, with a placeholder 'Leave a comment'. At the bottom of the comment area, there is a note: 'Attach files by dragging & dropping, selecting or pasting them.' Below the comment area, there is a checkbox labeled 'Allow edits by maintainers' which is checked, and a green button labeled 'Create pull request'.

On the right side, under the heading 'Helpful resources', there is a link to 'GitHub Community Guidelines'.

Y tras crearla vemos como queda:

The screenshot shows a GitHub Pull Request (PR) titled "Ejemplos de casos git merge #2". The PR is from the repository "IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02" (Private) and is from the branch "clases-alu-01:ejemplos_git_merge" to the base branch "IES-Francisco-Romero-Vargas:main". The PR status is "Open".

The PR description is: "Añado doc con ejemplos de git merge fast-froward, recursive y con colisión. Lleva un par de figuras tomadas con git history".

The PR has 1 commit, 0 checks, and 3 files changed. The commit hash is fb2f3d8.

The PR has no reviews, assignees, labels, or projects.

A green box indicates: "This branch has no conflicts with the base branch. Only those with write access to this repository can merge pull requests."

Ahora vemos de nuevo como el propietario del repositorio vería el PR abierto:

The screenshot shows the GitHub repository page for "IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02" (Private). The "Pull requests" tab is selected, showing 1 Open pull request and 1 Closed pull request.

The pull request list shows the PR titled "Ejemplos de casos git merge" (#2 opened 2 minutes ago by clases-alu-01).

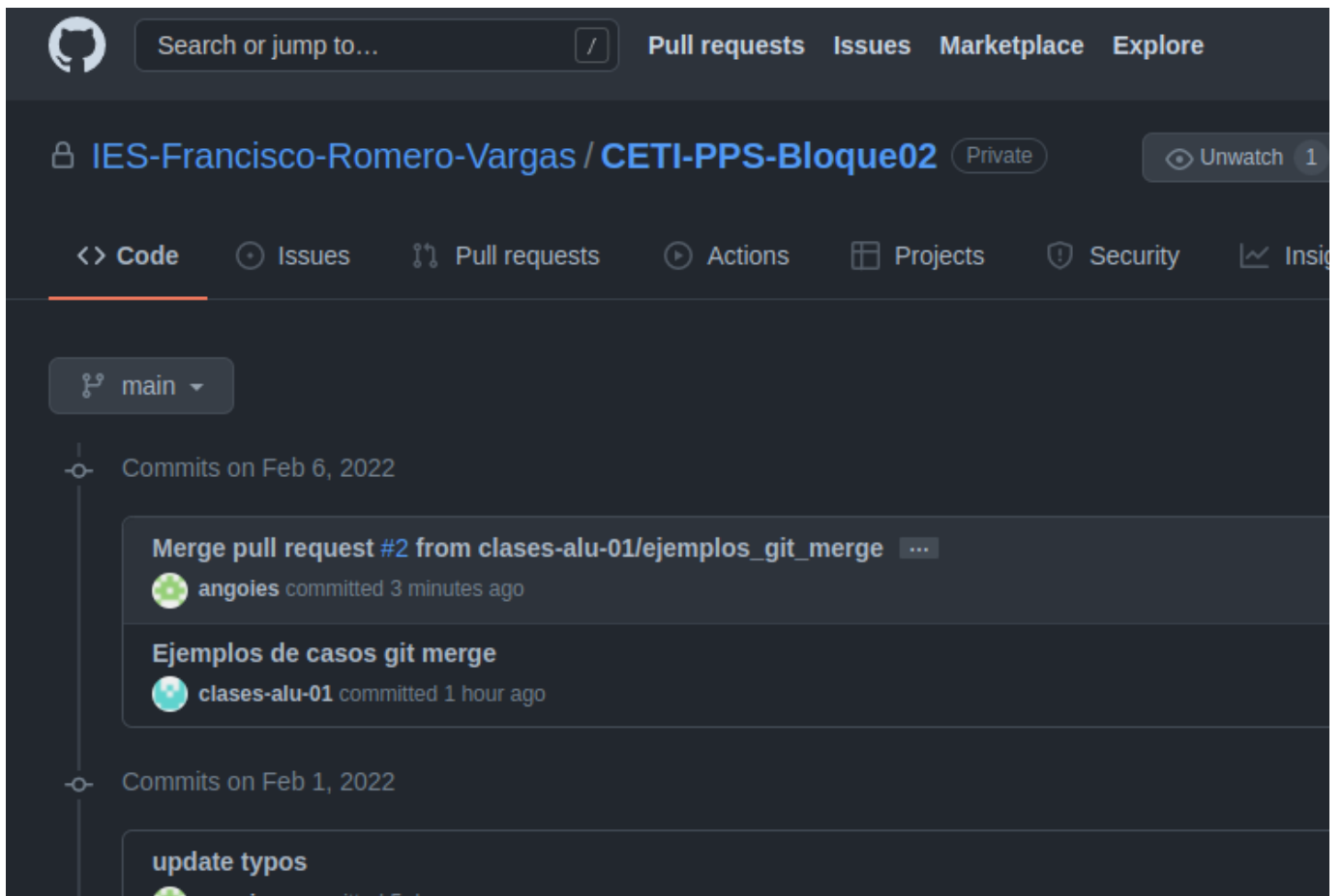
Lo abrimos si queremos ver los detalles. Y en este caso lo aceptamos haciendo **Merge Pull Request**

The screenshot shows a GitHub Pull Request (PR) interface. At the top, the repository is identified as 'IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02' with a 'Private' label. The 'Pull requests' tab is selected, showing 1 request. The PR title is 'Ejemplos de casos git merge #2'. It shows that 'clases-alu-01' wants to merge 1 commit into 'IES-Francisco-Romero-Vargas:main' from 'clases-alu-01:ejemplos_git_merge'. Below the title, there are tabs for 'Conversation' (0), 'Commits' (1), 'Checks' (0), and 'Files changed' (3). A comment from 'clases-alu-01' is visible, stating: 'Añado doc con ejemplos de git merge fast-froward, recursive y con colisión. Lleva un par de figuras tomadas con git history'. Below the comment, a commit is shown with the message 'Ejemplos de casos git merge' and hash 'fb2f3d8'. A message indicates: 'Add more commits by pushing to the ejemplos_git_merge branch on clases-alu-01/CETI-PPS-Bloque02.' At the bottom, a green box contains status messages: 'Continuous integration has not been set up' and 'This branch has no conflicts with the base branch'. A green button labeled 'Merge pull request' is visible.

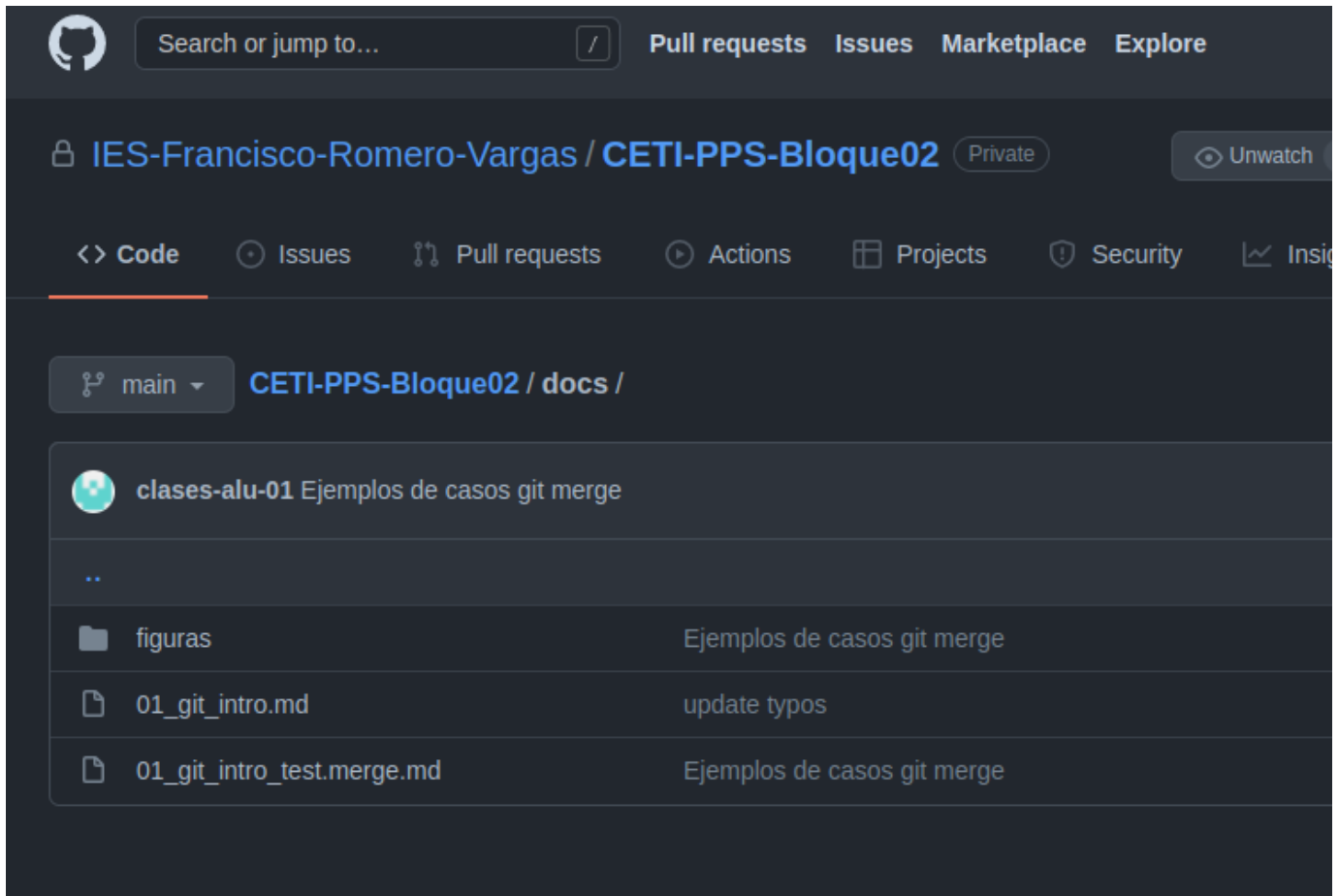
Ahora el PR está aceptado y cerrado, y los ficheros se han incorporado a la rama `main` del repositorio inicial.

The screenshot shows a GitHub pull request interface. At the top, the repository is 'IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02'. The pull request title is 'Ejemplos de casos git merge #2'. It shows that 'angoies' merged 1 commit into 'IES-Francisco-Romero-Vargas:main' from 'clases-alu-01:ejemplos_git_merge'. Below this, there's a section for 'Conversations' with one comment from 'clases-alu-01' stating: 'Añado doc con ejemplos de git merge fast-froward, recursive y con colisión. Lleva un par de figuras tomadas con git history'. The commit history shows a merge of commit '91b56f1' into 'main'. A success message states: 'Pull request successfully merged and closed. You're all set — the 'clases-alu-01:ejemplos_git_mer...' branch can be safely deleted. If you wish, you can also delete this fork of IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02 in the settings.' At the bottom, there's a 'Write' and 'Preview' section for adding a comment.

Puede apreciar dos commits nuevos en `main` : el original de la rama fusionada y uno más de la fusión realizada:



Y confirmamos que el fichero se ha incorporado a `main` :



5.1.2.2. Actualizar repositorios clone y forked (*upstream* del original)

Ahora tenemos algo que considerar: el repositorio principal está actualizado, pero el repositorio *forkeado* y/o el clonado no. Además es posible que haya incorporaciones de código de otras personas que no tengamos tampoco.

Los pasos a dar para conseguir que los repositorios clonados y forkeado estén actualizados serían:

1. Agregar en local el repositorio Git original como un **repositorio upstream**. Eso nos permite extraer los cambios desde el repositorio original a la versión local (`git remote add upstream <repo_original>`).
2. Descargar el repositorio original (`git fetch upstream`). Las confirmaciones (commits) del repositorio original serán almacenadas en una rama local llamada `upstream/main`.
3. Fusionar los cambios de la rama `upstream/main` a tu rama maestra local (`git merge upstream/main`). Esto hará que tu rama `main` se sincronice con el repositorio `upstream` sin perder tus cambios locales.
4. Enviar (`git push origin main`) los cambios al repositorio forkeado. Esto actualiza los cambios en el repositorio forkeado, y si es necesario genera la posibilidad de una PR.

Paso 1: observe los remotos originales (creados al hacer `git clone`) y los que se añaden como `upstream`

```
clases@vm:~/tmp/CETI-PPS-Bloque02$ git remote -v
origin  https://github.com/clases-alu-01/CETI-PPS-Bloque02.git (fetch)
origin  https://github.com/clases-alu-01/CETI-PPS-Bloque02.git (push)
### 1.
clases@vm:~/tmp/CETI-PPS-Bloque02$ git remote add upstream https://github.com/IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02.git
clases@vm:~/tmp/CETI-PPS-Bloque02$ git remote -v
origin  https://github.com/clases-alu-01/CETI-PPS-Bloque02.git (fetch)
origin  https://github.com/clases-alu-01/CETI-PPS-Bloque02.git (push)
upstream https://github.com/IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02.git (fetch)
upstream https://github.com/IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02.git (push)
clases@vm:~/tmp/CETI-PPS-Bloque02$
```

Paso 2: Descargamos del repositorio original (`git fetch upstream`):

```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git fetch upstream
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
Desempaquetando objetos: 100% (1/1), 669 bytes | 669.00 KiB/s, listo.
Desde https://github.com/IES-Francisco-Romero-Vargas/CETI-PPS-Bloque02
* [nueva rama]      main      -> upstream/main
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Se ha creado una rama `upstream/main` que podemos fusionar con nuestro `main` local

Paso 3: desde la rama `main` local hacemos un merge con la rama descargada

```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git switch main
Ya en 'main'
Tu rama está actualizada con 'origin/main'.
clases@vm:~/tmp/CETI-PPS-Bloque02$ git merge upstream/main
Actualizando a492dd0..91b56f1
Fast-forward
 docs/01_git_intro_test.merge.md      | 334 +++++
 docs/figuras/git-merge-conflicto.png | Bin 0 -> 118405 bytes
 docs/figuras/git-merge-recursive.png | Bin 0 -> 120730 bytes
 3 files changed, 334 insertions(+)
 create mode 100644 docs/01_git_intro_test.merge.md
 create mode 100644 docs/figuras/git-merge-conflicto.png
 create mode 100644 docs/figuras/git-merge-recursive.png
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Podemos comprobar como nuestro repositorio local está ahora adelantado respecto a `origin/main` (el repositorio forkeado)

```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git status
En la rama main
Tu rama está adelantada a 'origin/main' por 2 commits.
(usa "git push" para publicar tus commits locales)

nada para hacer commit, el árbol de trabajo está limpio
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Y ahora damos el último paso, actualizar origin, el repositorio *forkeado*

Paso 4:

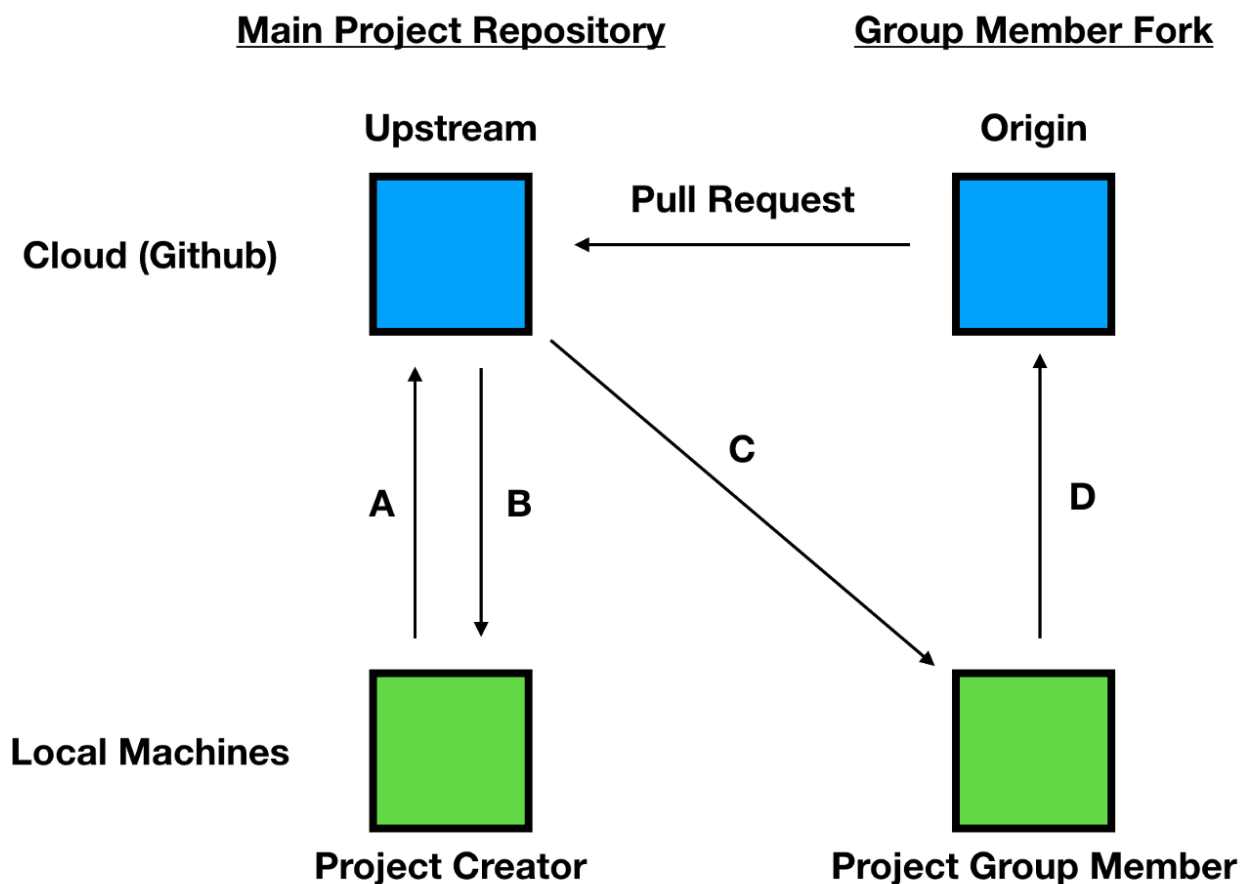
```

clases@vm:~/tmp/CETI-PPS-Bloque02$ git push origin main
Total 0 (delta 0), reusado 0 (delta 0)
To https://github.com/clases-alu-01/CETI-PPS-Bloque02.git
a492dd0..91b56f1  main -> main
clases@vm:~/tmp/CETI-PPS-Bloque02$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

nada para hacer commit, el árbol de trabajo está limpio
clases@vm:~/tmp/CETI-PPS-Bloque02$

```

Nota: es muy recomendable actualizar el repositorio local antes de hacer un PR ya que es más sencillo solucionar los posibles conflictos en local antes de incorporarlos a GitHub. Esta gráfica le puede servir de referencia: antes de hacer `push` y `pull request` actualizamos el repositorio en local a través de *upstream*.

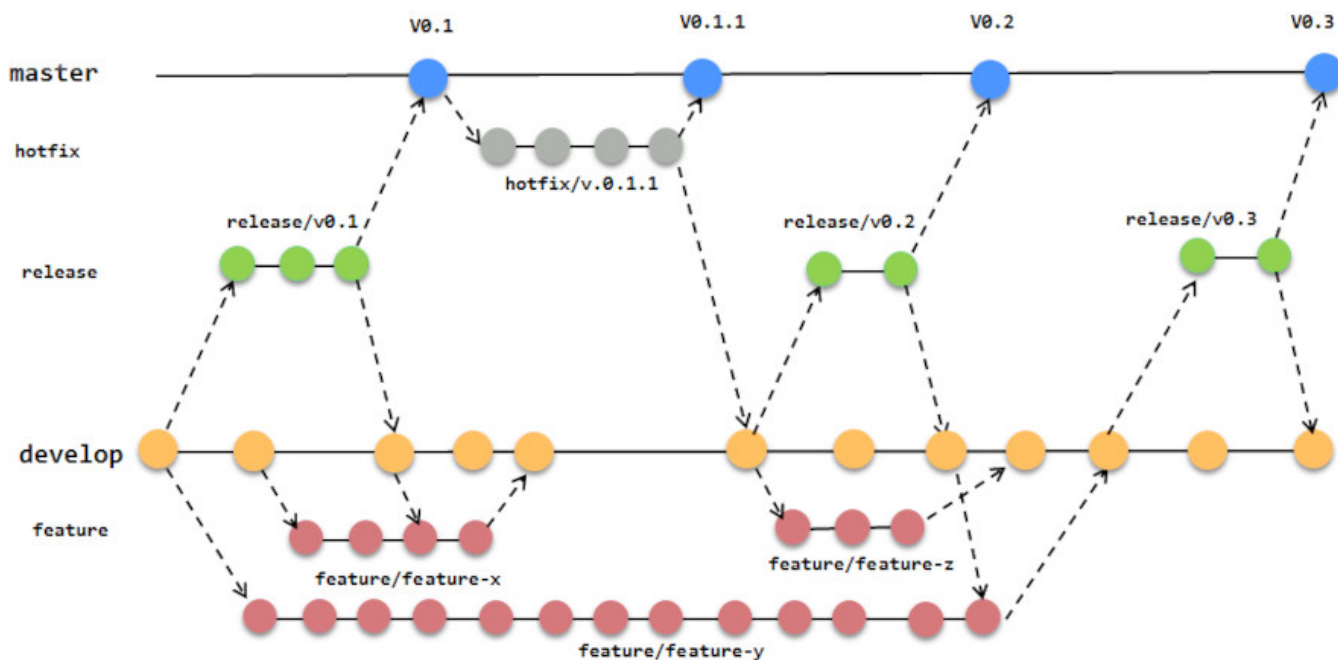


5.1.3 3. Git Flow

Hemos visto un ejemplo sencillo de cómo funciona el trabajo en equipo y las PR. Cuando la aplicación y el equipo crece es fundamental definir un flujo de trabajo.

Eso es lo que pretende git flow: sistematizar el uso (y nombres) de las ramas, cómo incorporar nuevas características, resolver fallos ,etc.

En este primer enlace <https://www.programoergosum.es/tutoriales/flujo-de-trabajo-en-git-con-gitflow/> se muestra una gráfica que resume todo el funcionamiento de git flow:



El modelo de GitFlow cuenta con dos ramas principales, master y develop, y varias de soporte para estas que podrán ser ramas de feature, release y hotfix.

- La rama master es la que contiene la última versión de nuestro proyecto y usaremos para desplegar en producción.
- La rama develop es la que contiene el último estado del desarrollo del mismo, es decir, hasta el último commit que hayamos hecho.
- Las ramas de feature se usarán para desarrollar nuevas funcionalidades, se crearán a partir de la rama develop y al terminar se fusionan con develop.
- Las ramas release se usarán para lanzar una nueva versión de nuestro proyecto y se fusionarán tanto con master como con develop.
- Las ramas hotfix se usarán para cambios rápidos sobre la rama master y se fusionan con master y develop.

Para seguir este modelo de trabajo se puede instalar el paquete git flow con el comando `apt install git-flow` y este paquete nos facilita comandos específicos que comienzan con `git flow` y simplifican el trabajo de crear ramas, fusionarlas, etc.

En este segundo enlace <https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow> se explica el uso de cada rama y el uso de los comandos `git flow` y sus comandos `git` equivalentes. Por ejemplo:

Al igual que al finalizar una rama release, una rama hotfix se fusiona tanto en main como en develop. con git tenemos que ejecutar

```
git checkout main
git merge hotfix_branch
git checkout develop
git merge hotfix_branch
git branch -D hotfix_branch
```

Con git flow sería equivalente a

```
git flow hotfix finish hotfix_branch
```

Si quiere profundizar más tiene un Curso en Openwebminar sobre `git flow`: <https://openwebinars.net/academia/aprende/gitflow/>

5.1.4.4. Buenas prácticas

Cada empresa tiene su método de trabajo, etc. Pero hay una serie de recomendaciones generales que se pueden aplicar siempre.

4.1. Buenas prácticas en los commits

En este artículo "Cómo escribir un buen mensaje de commit en Git" <https://geekytheory.com/como-escribir-buen-mensaje-commit-git/> tiene ejemplos. En particular hace referencia a la guía de estilo que se aplica en el desarrollo de angular, *Git Commit Guidelines* <https://github.com/angular/angular.js/blob/master/DEVELOPERS.md#commits>

4.2. Buenas prácticas en los PR

Algo similar nos encontramos con los PR en este artículo: *Buenas prácticas para crear una Pull Request* <https://geekytheory.com/como-crear-pull-request-buenas-practicas/>

- Revisa tu propia Pull Request
- Comenta tu propia Pull Request
- Haz Pull Requests pequeñas
- etc..

5.1.5 5. Enlaces

1. Artículos tipo *Paso a paso*

- [Cómo hacer tu primer PR](#)
- [Tutorial simple para hacer tu primer PR](#). *Añade sobre el anterior la configuración de remote y su uso tras la aceptación del PR final actualizar el repositorio local clonado*
- [Dos usuarios trabajando a través de PR](#). *Añade el uso de "issues" y asignaciones.**

2. [Difference between Git Clone and Git Fork](#) *Explicación del proceso fork/clone/push/PR*

3. [Github on Group Projects](#). *Buena explicación gráfica de clone/fork/upstream.*

4. Enlaces sobre **Git flow**:

- Gráfica resumen y comandos `git flow`: <https://www.programeergosum.es/tutoriales/flujo-de-trabajo-en-git-con-gitflow/>
- Git flow: explica uso cada rama y comandos git flow y comandos git equivalentes: <https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>
- Curso Openwebminar git flow (1h:15m) : <https://openwebinars.net/academia/aprende/gitflow/>

5. Buenas prácticas

- [en PR](#)
- [en commits](#)
- [en GitHub Issues](#)

6. About issues: [Learn how you can use GitHub Issues to track ideas, feedback, tasks, or bugs](#)

5.2 Git teams tarea

5.2.1 1. Ejercicio 01: Hacer proceso Pull Request

Pasos a dar:

- Hacer fork del repositorio `CETI-PPS-aficiones` en GitHub.
- Clonar en local
- Cambiar a rama `develop`
- Crear nueva rama (`feature-<user>`) y trabajar en ella
- Incluir un fichero en formato Markdown en el directorio `docs` con una breve reseña de su serie, libro o programa preferido y hacer commit del mismo.
- Modificar el fichero `indice.md` en la raíz y añadir una línea con su nombre, su serie, libro o programa preferido, y un enlace al nombre del fichero creado previamente en el directorio `docs`. El formato de un enlace en Markdown es `[Texto del enlace](ruta_al_documento)`. Vuelva a hacer commit para añadir este fichero al repositorio.
- Hacer `git push` al repositorio *forkeado* de la rama creada: `git push origin feature-<user>`
- Observe que en el propio mensaje de push indica la posibilidad de hacer PR (y muestra un enlace para hacerlo)
- Hacer el Pull Request desde el enlace (o desde el repositorio *forkeado*). Tenga en cuenta que debe hacer el PR **a la rama `develop` del repositorio principal**.
- Esperar que se acepte su PR (o se le propongan modificaciones)
- Esperar a que se acepten varios PR de otros miembros del equipo en el repositorio original. En ese momento el repositorio clonado y el forkeado no coinciden con original.
- Configurar un remote upstream y hacer `fetch & merge` del repositorio principal en su repositorio clonado y luego update del forkeado.

5.2.2 2. Ejercicio 02: Procesar Pull request

- Crear un repositorio en GitHub
- Añadir algunos ficheros y commits.
- Crear rama `develop`.
- Que un compañero intente añadir un documento y modificar un fichero existente sobre su repositorio y haga un PR
- Aceptar el PR sobre la rama `develop` o sugerir cambios antes de aceptar el PR.

5.3 Git seguro

Se exponen distintas formas de trabajar de una forma más segura con git y GitHub.

5.3.1 1. Fichero .gitignore

El fichero `.gitignore` permite especificar ficheros o carpetas del directorio de trabajo que no se añadirán al repositorio al hacer un `git add .` o similar.

Como ejemplo se crean algunos ficheros en un repositorio ya existente:

```
alu@tos:~/git-seguro$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

nada para hacer commit, el árbol de trabajo está limpio
alu@tos:~/git-seguro$ ls -a
.  ..  .git  README.md
alu@tos:~/git-seguro$
```

Cree el fichero `.env` con información de configuración (que no quiere añadir ni al repositorio local ni al remoto), el fichero `.gitignore` y el fichero `todo.md`:

```
alu@tos:~/git-seguro$ nano .env
alu@tos:~/git-seguro$ nano .gitignore
alu@tos:~/git-seguro$ echo "# Todo" > todo.md
alu@tos:~/git-seguro$ cat .env
USER=clases@locla.com
TOKEN=1234567890QWERTYASDFGHH
alu@tos:~/git-seguro$ cat .gitignore
.env
venv/
alu@tos:~/git-seguro$
```

Observe que en el fichero `.gitignore` se han añadido dos líneas: una para ignorar el fichero `.env` y otra para ignorar si existe el directorio `venv` (posible entorno virtual)

Si hace `git status` verá que aunque hay ficheros nuevos sólo "rastrea" los que no están incluido en `.gitignore`: `todo.md` y el propio `.gitignore`, pero no `.env`:

```
alu@tos:~/git-seguro$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que se será confirmado)
    .gitignore
    todo.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
alu@tos:~/git-seguro$
```

Observe que incluso haciendo un `git add .` el fichero `.env` no se añade al stage:

```
alu@tos:~/git-seguro$ git add .
alu@tos:~/git-seguro$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    nuevo archivo: .gitignore
    nuevo archivo: todo.md

alu@tos:~/git-seguro$
```

Haga *push* al repositorio remoto

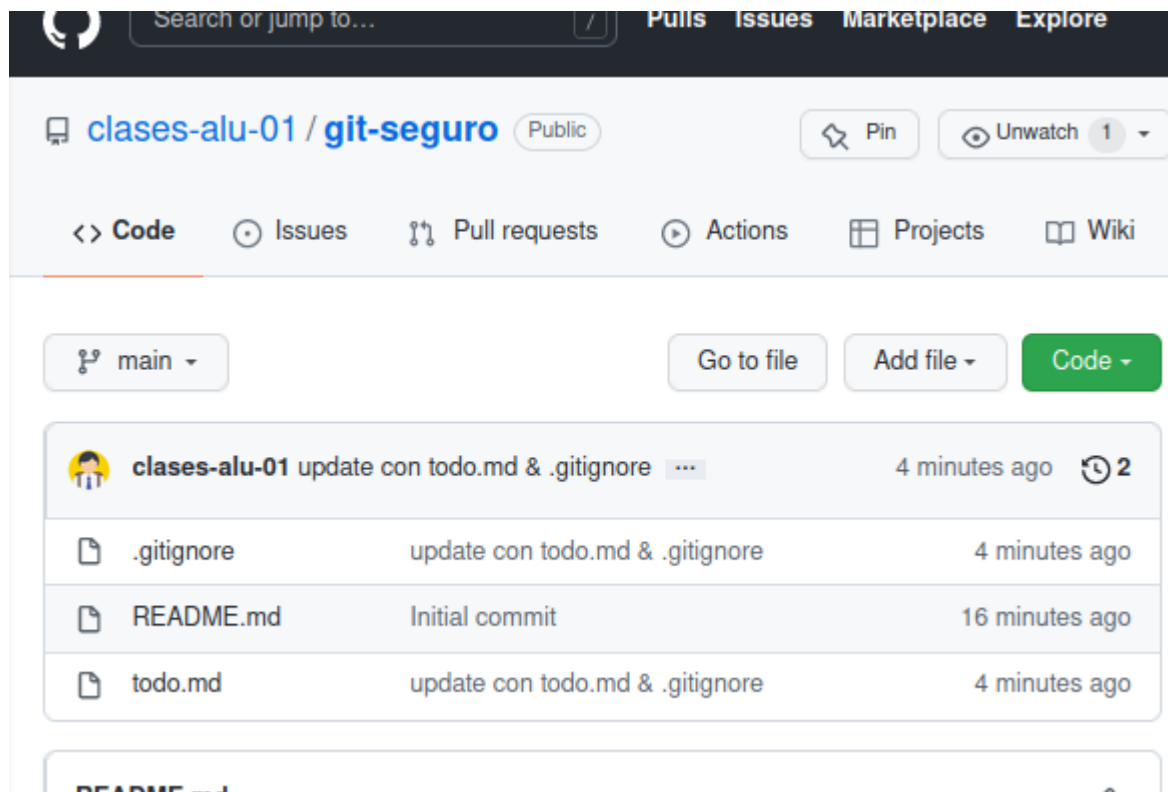
```
alu@tos:~/git-seguro$ git push origin main
Username for 'https://github.com': clases.alu+01@gmail.com
Password for 'https://clases.alu+01@gmail.com@github.com':
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 8 hilos
```

```

Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (4/4), 351 bytes | 351.00 KiB/s, listo.
Total 4 (delta 0), reusado 0 (delta 0)
To https://github.com/clases-alu-01/git-seguro.git
c3b190d..968a431 main -> main
alu@tos:~/git-seguro$

```

y puede comprobar en GitHub que se han añadido `.gitignore` y `todo.md`, pero no el fichero `.env` (ni el directorio `venv` si existiera).



Se pueden ignorar archivos individuales, directorios completos, o utilizar patrones. Tiene más información en este enlace: <https://aprendegit.com/tag/gitignore/>

Nota: existe la posibilidad al crear el repositorio en GitHub de marcar la opción de añadir un fichero `.gitignore`, seleccionar el tipo de *framework* o lenguaje a utilizar y GitHub crea un fichero `.gitignore` muy completo. A pesar de usar `.gitignore` debe ser precavido con el uso de `git add .`: a veces es preferible añadir los ficheros explícitamente.

5.3.2.2. Git leaks

<https://github.com/zricethezav/gitleaks>

Gitleaks is a tool for detecting secrets like passwords, API keys, and tokens in git repos, files, and whatever else you wanna throw at it via stdin.

1. Primero descargue de (<https://github.com/zricethezav/gitleaks/releases>) el fichero adecuado a su sistema. En el caso de estas prácticas algo similar a `gitleaks_X.YY.Z.linux_x64.tar.gz`.
2. Descomprimir el fichero y
3. copiar el fichero `gitleaks` al directorio `/usr/local/bin` (o incluir el directorio del ejecutable en el PATH).

Las reglas que se aplican las puede consultar en GitHub. Se aplican expresiones regulares para buscar tokens, API keys, etc. Para acceder a la ayuda:

```

$ gitleaks -h
Gitleaks scans code, past or present, for secrets

Usage:
  gitleaks [command]

...

```



```
"Commit": "",
"Entropy": 3.7683827,
"Fingerprint": "clave.txt:generic-api-key:1"
},
{
"RuleID": "generic-api-key",
...

```

Si quiere analizar sólo la zona de seguimiento o staged antes de un *commit* debe usar el comando `gitleaks git --staged`. Aquí tiene un ejemplo: añade sólo uno de los ficheros a la zona de seguimiento

```
$ git add discord.txt
$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    nuevos archivos: discord.txt

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    clave.txt
$
```

y observe cómo `gitleaks git --staged` detecta una fuga y `gitleaks dir` detecta dos.

```
$ gitleaks git --staged --no-banner
11:59AM INF 1 commits scanned.
11:59AM INF scanned ~88 bytes (88 bytes) in 117ms
11:59AM WRN leaks found: 1
$ gitleaks dir --no-banner
11:59AM INF scanned ~22948 bytes (22.95 KB) in 12.5ms
11:59AM WRN leaks found: 2
$
```

5.3.3 3. Mejorando el proceso: pre-commit

Ahora parece buena idea que antes de hacer un commit en local debe comprobar los ficheros a confirmar con `gitleaks` (y sobre todo antes de hacer un push al remoto).

Incluso sería deseable condicionar la realización del commit a un resultado sin fugas. Tenga en cuenta que el comando `gitleaks` devuelve el valor 1 si hay fugas y cero si no. Por otra parte la variable de entorno `$?` almacena el valor de la última ejecución de un comando en CLI.

```
$ gitleaks git --no-banner
12:05PM INF 2 commits scanned.
12:05PM INF scanned ~92 bytes (92 bytes) in 128ms
12:05PM INF no leaks found
$ echo $?
0
$ gitleaks git --staged --no-banner
12:05PM INF 1 commits scanned.
12:05PM INF scanned ~88 bytes (88 bytes) in 123ms
12:05PM WRN leaks found: 1
$ echo $?
1
$
```

En el siguiente apartado se automatiza este proceso con pre-commit.

3.1. pre-commit

El proceso de buscar fugas y detener el commit se puede automatizar usando los *hooks* de git, en este caso el del tipo *pre-commit*.

Git cuenta con mecanismos para lanzar scripts de usuario cuando suceden ciertas acciones importantes, llamados puntos de enganche (hooks). Hay dos grupos de esos puntos de lanzamiento: los del lado cliente y los del lado servidor. Los puntos de enganche del lado cliente están relacionados con operaciones tales como la confirmación de cambios (commit) o la fusión (merge). ...

Los puntos de enganche se guardan en la subcarpeta *hooks* de la carpeta Git. En la mayoría de proyectos, estará en `.git/hooks`. Por defecto, esta carpeta contiene unos cuantos scripts de ejemplo.

<https://git-scm.com/book/es/v2/Personalizaci%C3%B3n-de-Git-Puntos-de-enganche-en-Git>

El objetivo es que cada vez que se invoque un commit se ejecute antes un script que *comprueba* cosas, y si todo es correcto se ejecuta el commit. En nuestro caso la comprobación será que `gitleaks` no encuentre fugas.

Para ello se crea un fichero `.git/hooks/pre-commit` y se le añade el script que se muestra a continuación:

```
$ nano .git/hooks/pre-commit
## SE edita y añade el script
$ cat .git/hooks/pre-commit
#!/bin/sh
# This is an example of what adding gitleaks to a pre-commit ...

gitleaks git --staged -v
if [ $? -eq 1 ]; then
    echo "\n>> pre-commit: Error: gitleaks ha detectado fugas de información en tus cambios."
    echo ">> pre-commit: No se realizará el commit"
    exit 1
fi
```

El script invoca `gitleaks` y comprueba el valor de retorno (`$?`). Si es 1 indica que hay fugas, y el script devuelve como resultado el valor 1 provocando que el commit no se realice.

Tenga en cuenta que debe dar permisos de ejecución al fichero `pre-commit`:

```
$ chmod +x .git/hooks/pre-commit
$
```

Y sólo queda probar el commit. Al ejecutarlo se muestra la salida de `gitleaks`, el mensaje de error del script, y el commit no se realiza:

```
$ git commit -m "testing pre-commit: debe dar error pro clave.txt"
...
9:42PM WRN leaks found: 1
9:42PM INF scan duration: 51.923786ms

>> Error: gitleaks ha detectado fugas de información en tus cambios.
>> pre-commit: No se realizará el commit"
$
```

5.3.4 4. Otras consideraciones

4.1. Información sensible

Debe evitar respaldar información sensible en el repositorio. Para evitarlo hasta ahora:

- El primer paso es usar `.gitignore`. Ficheros del tipo `.pem` o `.key` deberían estar incluidos. Pero no olvide por ejemplo si tiene una aplicación PHP las credenciales de la BD, etc.
- El segundo, usar `git add <files>` en vez de `git add .` para controlar con exactitud que se añade a la zona de seguimiento.
- Usar una herramienta como `gitleaks` para buscar fugas
- Configurar `pre-commit` para automatizar esta tarea.

Otras herramientas adicionales que puede usar:

- **git-seekret**: *inspect on commits and/or staged (and uncommitted yet) files, to prevent adding sensitive information into the git repository. It can be easily integrated with git hooks, forcing it to analyse all staged files before they are included into a commit.* Sus reglas se basan en [seekret](#).
- **BFG**: *The BFG is a simpler, faster alternative to git-filter-branch for cleaning bad data out of your Git repository history, removing Crazy Big Files and removing Passwords, Credentials & other Private data*

Si la información sensible se ha añadido al repositorio remoto en GitHub se recomienda leer este enlace [Eliminación de datos confidenciales de un repositorio](#).

Cuando has subido datos sensibles (por error) a un repo de GitHub, no basta con borrar el archivo y hacer un commit, porque Git guarda todo el historial. El dato seguirá allí en la historia del repositorio. Por eso necesitas reescribir el historial para eliminarlo de verdad. Las dos herramientas más usadas son `git filter-branch` y `BFG Repo-Cleaner`

4.2. Proteger el acceso al repositorio git

- Hay que proteger el directorio `.git` : *These internal files contain the whole history of your committed changes. In other words, the .git folder is your Git repository and consequently should never be exposed to people not authorized to clone it.*
- Si se usan varios equipos de trabajo (en la oficina, en casa, portátil, etc) se recomienda crear pares de claves SSH distintas para cada equipo. De esa forma se puede revocar la clave del equipo que pueda estar comprometido.
- Si da acceso a terceras aplicaciones, usar tokens de acceso que son fácilmente revocables.
- Si utiliza GitHub o GitLab debería usar *Two-Factor Authentication (2FA)*.

4.3. Mantener git (y herramientas) actualizadas

Y por supuesto, una vez más: git y todo el conjunto de utilidades alrededor del mismo pueden tener vulnerabilidades, así que todos deberían estar siempre actualizados.

4.4. Ejemplos de fugas por git

- [Data of 16 million Brazilian COVID-19 patients exposed online:](#)

The personal and healthcare records of 16 million Brazilian COVID-19 patients, including government representatives, exposed online.

The leak came to limelight after a GitHub user noticed the spreadsheet containing the passwords on the personal GitHub account of an employee working with Albert Einstein Hospital in the city of Sao Paulo.

The spreadsheet included the login credentials for systems including E-SUS-VE and Sivep-Gripe, two government databases used to store information on COVID-19 patients.

- [Nissan source code leaked online after Git repo misconfiguration](#)

Nissan source code leaked online after Git repo misconfiguration Nissan was allegedly running a Bitbucket Git server with the default credentials of admin/admin.

4.5. GitHub seguro: Recomendaciones

En este apartado se enumeran algunas recomendaciones específicas para GitHub tomadas de [Security Best Practices for Github](#).

Entre ellas:

- Autenticación:
- Usar [doble factor de autenticación](#)
- limitar la IP de acceso a GitHub
- Configuración de los repositorios
- fijar al mínimo los permisos base para los repositorio de la organización a ninguno. Esto obkliga a especificar los permisos en cada repositorio
- Controlar el acceso de loscolaboradores externos, fijando sus permisos al mínimo posible.
- Revisar permisos de acceso en la configuración del repositorio:

IES-Francisco-Romero-Vargas / CETI-PPS-Bloque02 (Private)

Unwatch (1) Fork (4) Star (0)

<> Code Issues Pull requests Actions Projects Security Insights Settings

General

Access

Collaborators and teams

Team and member roles

Code and automation

Branches

Actions

Webhooks

Pages

Security

Code security and analysis

Deploy keys

Secrets

Integrations

Who has access

PRIVATE REPOSITORY (locked)

Only those with access to this repository can view it.

[Manage](#)

BASE ROLE (None)

No base role set. Only Owners and those with direct access can access this repository.

[Set base role](#)

DIRECT ACCESS

1 has access to this repository. 1 team.

Manage access

Create team Add people Add team

☐ Select all Type Role

Find a team, organization member or outside collaborator...

☐ **CETI-Alumnado**
@ceti-alumnado • 10 members Role: Read

- Controlar el forking de un repositorio:
 - Es difícil manejar la seguridad de cada fork.
 - Puede crearse una copia del fork en la cuenta personal del usuario lo cual puede ser otro problema de seguridad.
- No permitir cambios en la visibilidad de los repositorios (público/privado)
- Elegir si los miembros pueden crear repositorios en tu organización o no. Si se permite que los miembros creen repositorios, elegir si pueden crear tanto repositorios públicos como privados o solo públicos.
- Habilitar las reglas de protección de ramas (*branch protection rules*)
 - Revisar el código antes de hacer un merge
 - Forzar un número de revisores para permitir un commit
 - Obligar a que los commits estén firmados
- Seguridad en el código fuente y
 - Habilitar chequeo de dependencias
 - Revisar los logs
 - Revisar el acceso de terceros y de aplicaciones a GitHub

5.3.5 5. Prácticas

5.1. Ejercicio

Entregar memoria que refleje con enlaces y capturas lo siguiente:

1. Crear un repositorio en local con al menos dos commit.
2. Crear fichero `.gitignore` que impida añadir al repositorio ficheros con extensión `.log` y el directorio del entorno virtual (pero sí el fichero `.gitignore`). Cree algún fichero de log en el directorio de trabajo para comprobar que son ignorados.
3. Deberá conectar ese repositorio con un repositorio remoto en GitHub. Este deberá ser público y deberá indicar su URL en la práctica.
4. Verificar que su repositorio no tiene fugas con la utilidad `gitleaks`. Añada un fichero de entorno con alguna clave como la del tema para comprobar que si hay fugas las detecta. Finalmente incluya el fichero de variables de entorno al `.gitignore` para evitar que se añada al repositorio.
5. Crear hook pre-commit en repositorio local para comprobar las fugas con `gitleaks` antes de hacer el commit.
6. Comprimir el directorio del repositorio y subir a la plataforma

5.2. Ejercicio Opcional

Crear un pre-commit para una aplicación en Python que verifique:

- que el código Python supera el linting (`flake8` o `pylint`).
- que el código Python supere un test unitario.

5.3.6 6. Tools: pre-commit

pre-commit (<https://pre-commit.com/>) es un gestor de hooks. En vez de mantener scripts manuales dentro de `.git/hooks`, se declaran los hooks en `.pre-commit-config.yaml`. Después se activa con `pre-commit install`, y desde ese momento se lanza en cada `git commit`.

Además el framework pre-commit gestiona versiones y actualizaciones automáticamente. Otra ventaja de este framework es que el fichero de configuración se distribuye junto con el repositorio, y por lo tanto es "versionable".

6.1. Ejemplo práctico

Se instala el módulo

```
(venv) alu@xd13:~/git
$ pip install pre-commit
...
(venv) alu@xd13:~/git
```

Se crea el fichero de configuración `.pre-commit-config.yaml` en la raíz del repositorio:

```
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.2 # Usa la última versión estable
  hooks:
  - id: gitleaks
```

Y se activa para que git lo utilice:

```
(venv) alu@xd13:~/git
$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
(venv) alu@xd13:~/git
```

Para probarlo añadir el fichero de configuración `config.py` con una clave de prueba:

```
# Archivo con un secreto expuesto
AWS_SECRET_KEY = "AKIAIMNO7VWG6EXAMPLE"
DATABASE_URL = "postgres://user:password123@localhost:5432/mydb"
```

Añadir al repo:

```
alu@xd13:~/git
$ git add config.py
```

e intentar hacer commit:

```
alu@xd13:~/git
$ git commit -m "test pre-commit"
[INFO] Initializing environment for https://github.com/gitleaks/gitleaks.
[INFO] Installing environment for https://github.com/gitleaks/gitleaks.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
Detect hardcoded secrets.....Failed
- hook id: gitleaks
- exit code: 1
```



```
Finding:      AWS_SECRET_KEY = "REDACTED"
Secret:       REDACTED
RuleID:       aws-access-token
Entropy:      3.784184
File:         config.py
Line:         2
Fingerprint:  config.py:aws-access-token:2

1:06PM INF 1 commits scanned.
1:06PM INF scan completed in 4.23ms
1:06PM WRN leaks found: 1

(venv)
alu@xd13:~/git
$
```

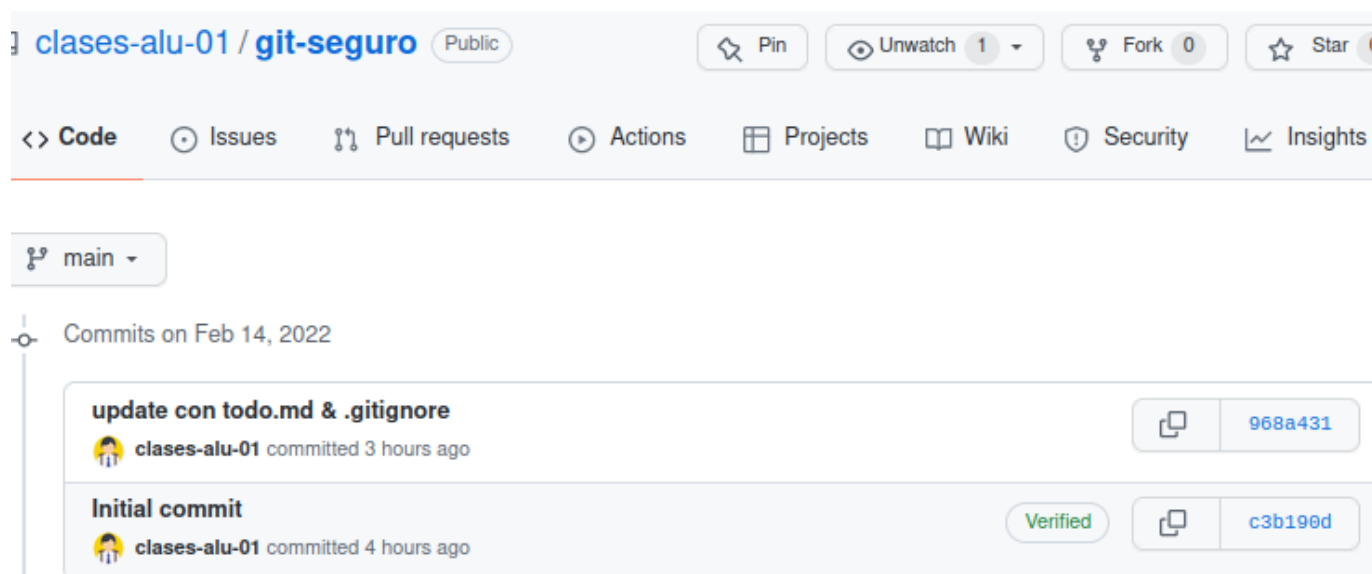
Otras posibilidades:

- `pre-commit run`: ejecuta los hooks sobre los ficheros staged. Es el comportamiento por defecto durante el commit
- `pre-commit run --all-files`: ejecuta los hooks sobre todos los archivos del repositorio

5.3.7.7. Anexos

7.1. Commits verificados con GPG

En GitHub puede ver la información de los commit realizados justo debajo del botón "Code" (el de color verde). Justo a la izquierda del *hash* aparece una marca indicando si el *commit* tiene un autor "**verificado**";



clases-alu-01 / git-seguro Public

<> Code Issues Pull requests Actions Projects Wiki Security Insights

main

Commits on Feb 14, 2022

update con todo.md & .gitignore 968a431
 clases-alu-01 committed 3 hours ago

Initial commit c3b190d
 clases-alu-01 committed 4 hours ago Verified

Aparecen como verificados los *commits* realizados en GitHub (por ejemplo el de creación inicial del repositorio) y los generados a partir de un Merge en GitHub.

Si el *commit* proviene del repositorio local no aparece como verificado excepto si el *commit* en local se han firmado con clave GPG.

Para poder firmar necesita generar claves GPG con `gpg --gen-key`. El comando solicitará nombre, email y una frase de contraseña de protección para la clave GPG (se pedirá cada vez que vaya a firmar con ella).

```
alu@tos:~$ gpg --full-gen-key
gpg (GnuPG) 2.2.19; Copyright (C) 2019 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Por favor seleccione tipo de clave deseado:
(1) RSA y RSA (por defecto)
(2) DSA y ElGamal
(3) DSA (sólo firmar)
(4) RSA (sólo firmar)
(14) Existing key from card
Su elección:
las claves RSA pueden tener entre 1024 y 4096 bits de longitud.
¿De qué tamaño quiere la clave? (3072) 4096
El tamaño requerido es de 4096 bits
Por favor, especifique el periodo de validez de la clave.
0 = la clave nunca caduca
<n> = la clave caduca en n días
<n>w = la clave caduca en n semanas
<n>m = la clave caduca en n meses
<n>y = la clave caduca en n años
¿Validez de la clave (0)?
La clave nunca caduca
¿Es correcto? (s/n) s

GnuPG debe construir un ID de usuario para identificar su clave.

Nombre y apellidos: Clases Alu 01
Dirección de correo electrónico: clases.alu+01@gmail.com
Comentario:
Ha seleccionado este ID de usuario:
"Clases Alu 01 <clases.alu+01@gmail.com>"

¿Cambia (N)ombre, (C)omentario, (D)irección o (V)ale/(S)alir? v
Es necesario generar muchos bytes aleatorios. Es una buena idea realizar
....
gpg: clave 1D4EC5FC181F659E marcada como de confianza absoluta
gpg: certificado de revocación guardado como '/home/alu/.gnupg/openpgp-revocs.d/6E1C43C8C0C42ED71084ECE61D4EC5FC181F659E.rev'
claves pública y secreta creadas y firmadas.

pub   rsa4096 2022-02-14 [SC]
       6E1C43C8C0C42ED71084ECE61D4EC5FC181F659E
uid           [absoluta] Clases Alu 01 <clases.alu+01@gmail.com>
sub   rsa4096 2022-02-14 [E]

alu@tos:~$
```

Debe anotar el id de la clave (en este ejemplo `1D4EC5FC181F659E`) que se puede obtener también con `gpg --list-secret-keys --keyid-format LONG`

```
alu@tos:~$ gpg --list-secret-keys --keyid-format LONG
/home/alu/.gnupg/pubring.kbx
-----
sec   rsa4096/1D4EC5FC181F659E 2022-02-14 [SC]
       6E1C43C8C0C42ED71084ECE61D4EC5FC181F659E
uid           [absoluta] Clases Alu 01 <clases.alu+01@gmail.com>
ssb   rsa4096/6047FF0F707B2B66 2022-02-14 [E]

alu@tos:~$
```

En este ejemplo es `1D4EC5FC181F659E`, y es el id que se utiliza con el comando `gpg --armor --export <id>` para extraer la parte publica de la clave que luego debe añadir en GitHub:

```
alu@tos:~$ gpg --armor --export 1D4EC5FC181F659E
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGIKg1QBEACdugTGBYdTQR6e+y08y1XW/+A7wbM/50/UDjf0c/ptrjcN2+B5
...
...
+l6uxkkZ4HyWfYdd4rPVptYrBeF2zJe5ci0RL0JBcuHn8qXl27iGxyIiq9UGi/jW
8uNn
=gU2J
-----END PGP PUBLIC KEY BLOCK-----
```

Copie el bloque incluyendo las dos líneas que delimitan `-----BEGIN PGP PUBLIC KEY BLOCK-----` y `-----END PGP PUBLIC KEY BLOCK-----` y nos vamos a GitHub, Menú principal, *Settings*. opción *SSH and GPG keys*. Haga clic en el botón "New GPG Key", añada el texto copiado y pulse "Add GPG Key".

**clases-alu-01**

Your personal account

[Go to your personal profile](#)

-  Profile
-  Account
-  Appearance
-  Accessibility
-  Notifications

Access

-  Billing and plans
-  Emails
-  Password and authentication

SSH and GPG keys

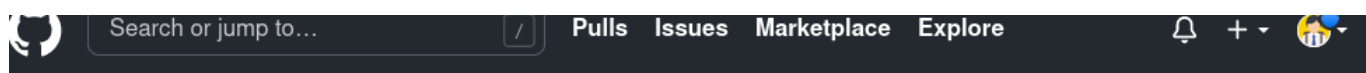
-  Organizations

GPG keys / Add new

Key

Begins with '-----BEGIN PGP PUBLIC KEY BLOCK-----'

[Add GPG key](#)

**clases-alu-01**

Your personal account

[Go to your personal profile](#)

- Profile
- Account
- Appearance
- Accessibility
- Notifications

Access

- Billing and plans
- Emails
- Password and authentication
- SSH and GPG keys**
- Organizations
- Moderation

SSH keys

[New SSH key](#)

There are no SSH keys associated with your account.

Check out our guide to [generating SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

[New GPG key](#)

This is a list of GPG keys associated with your account. Remove any keys that you do not recognize.

Email address: clases.alu+01@gmail.com

Key ID: 1D4EC5FC181F659E

Subkeys: 6047FF0F707B2B66

Added on 14 Feb 2022

[Delete](#)

Learn how to [generate a GPG key and add it to your account](#).

Tras copiar la clave pública en GitHub debes configurar git en local para que use la firma GPG segun estas instrucciones: <https://docs.github.com/en/authentication/managing-commit-signature-verification/telling-git-about-your-signing-key>

Se le indica a git (a nivel local o global) que `id` de firma va a usar con `git config user.signingKey <id>`:

```
~/git-module$ git config --global user.signingKey 1D4EC5FC181F659E
~/git-module$
```

Y ya puede firmar sus commits. Al hacer `commit` use la opción `-S` y pedirá la frase de paso de la clave GPG. Para probar modifique algún fichero y haga `git add`, `git commit -S` y `git push`.

```
alu@tos:~/git-seguro$ nano README.md
alu@tos:~/git-seguro$ git add README.md
alu@tos:~/git-seguro$ git commit -S -m "test sin -S desactivada commit.gpgsign"
[main 0444a7c] test sin -S desactivada commit.gpgsign
1 file changed, 2 insertions(+)
alu@tos:~/git-seguro$ git push
....
alu@tos:~/git-seguro$
```

En local puede ver el `commit` firmado con `git log --show-signature`:

```
alu@tos:~/git-seguro$ git log --show-signature
commit 0444a7cc7506c5975c56e92e8735cd6632a93aeb (HEAD -> main)
gpg: Firmado el mar 15 feb 2022 16:51:39 CET
gpg: usando RSA clave 6E1C43C8C0C42ED71084ECE61D4EC5FC181F659E
gpg: Firma correcta de "Clases Alu 01 <clases.alu+01@gmail.com>" [absoluta]
Author: Clases alu 01 <clases.alu+01@gmail.com>
Date: Tue Feb 15 16:51:39 2022 +0100

    test sin -S desactivada commit.gpgsign
...
```

Si accede a GitHub puede comprobar que el commit aparece "Verified" y que puede visualizar el ID de la clave GPG, etc:

clases-alu-01 / git-seguro Public

Code Issues Pull requests Actions Projects Wiki Security

main

Commits on Feb 14, 2022

test GPG Verified 9e6a045

clases-alu-01 committed 3 hours ago

update todo.md to check GPG

clases-alu-01 committed 3 hours ago

update con todo.md & .gitignore

clases-alu-01 committed 8 hours ago

Initial commit

clases-alu-01 committed 8 hours ago

This commit was signed with the committer's **verified signature**.

clases-alu-01

GPG key ID: 1D4EC5FC181F659E

[Learn about vigilant mode.](#)

Nota: Si quiere indicarle a git que por defecto firme los commits ejecute el comando `git config commit.gpgsign true` (para firmar los commit en cualquier repositorio `git config --global commit.gpgsign true`).

```
alu@tos:~/git-seguro$ git config commit.gpgsign true
alu@tos:~/git-seguro$ nano README.md
alu@tos:~/git-seguro$ git commit -m "test sin opción -S"
[main f81e10d] test sin opción -S
1 file changed, 3 insertions(+)
alu@tos:~/git-seguro$
/* Ha pedido la frase de paso y ha firmado */
alu@tos:~/git-seguro$ git log --show-signature
commit f81e10d2e444a6ebc9fd30c7458b9b994ed9d70 (HEAD -> main)
gpg: Firmado el mar 15 feb 2022 16:45:28 CET
gpg: usando RSA clave 6E1C43C8C0C42ED71084ECE61D4EC5FC181F659E
gpg: Firma correcta de "Clases Alu 01 <clases.alu+01@gmail.com>" [absoluta]
Author: Clases alu 01 <clases.alu+01@gmail.com>
Date: Tue Feb 15 16:45:28 2022 +0100

    test sin opción -S
...
alu@tos:~/git-seguro$
```

Si es necesario borrar una clave GPG:

```
# Listar las claves de tu anillo de claves públicas
$ gpg --list-keys
# Eliminar una clave pública (de tu anillo de claves públicas):
$ gpg --delete-key "ID|Nombre de Usuario"
# Listar las claves de tu anillo de claves secretas
$ gpg --list-secret-keys
# Esto elimina la clave secreta de tu anillo de claves secretas.
$ gpg --delete-secret-key "ID|Nombre de Usuario"
```

7.2. Uso del agente SSH

Se puede añadir la clave SSH al agente SSH para que no pida la palabra de paso reiteradamente. Para ello:

1. Primero compruebe si está iniciado el agente ssh con el comando `ssh-add -l`

```
alu@tos:~$ ssh-add -l
Could not open a connection to your authentication agent.
alu@tos:~$
```

2. Si indica que no puede abrir la conexión es necesario iniciar el agente SSH. Si no pasar al paso 3.

```
alu@tos:~$ eval `ssh-agent -s`
Agent pid 916589
alu@tos:~$ ssh-add -l
The agent has no identities.
alu@tos:~$
```

3. Añadir la clave con `ssh-add -t <time> <file>` (si se omite `-t time` se añade de forma permanente). Si se omite `file` toma la clave por defecto. Puede comprobar que se ha añadido con `ssh-add -l`:

```
alu@tos:~$ ssh-add -t 2h
Enter passphrase for /home/alu/.ssh/id_ed25519:
Identity added: /home/alu/.ssh/id_ed25519 (clases.alu+01@gmail.com)
Lifetime set to 7200 seconds
alu@tos:~$ ssh-add -l
256 SHA256:W9Ka18JIVwQfuC68EUUp1wpv7acGwroNdzucFnG4wr4 clases.alu+01@gmail.com (ED25519)
alu@tos:~$
```

Ya puede usar la clave SSH durante ese periodo sin introducir la frase de paso. Se pueden borrar las claves del agente con `ssh-add -D`.

7.3. Mejorando el pre-commit

Podemos mejorar el script, y controlar la ejecución de gitleaks con la variable lógica `gitleaksEnabled` (que se obtiene de la variables `hooks.gitleaks` de la configuración de git).

Para ello:

1. Modificamos el script

```
alu@tos:~/git-seguro$ cat .git/hooks/pre-commit
#!/bin/sh
# This is an example of what adding gitleaks to a pre-commit...

# Buscamos valor asignado: <> true no se ejecuta gitleaks
gitleaksEnabled=$(git config --bool hooks.gitleaks)

if [ "$gitleaksEnabled" = "true" ]; then
    gitleaks protect --staged -v
    if [ $? -eq 1 ]; then
        echo "\n>> pre-commit: Error: gitleaks protect has detected sensitive information in your changes."
        echo ">> pre-commit: No se realizará el commit"
        exit 1
    fi
fi
alu@tos:~/git-seguro$
```

1. Definimos en la configuración de git la variable `hooks.gitleaks` a `true` para poder activar (o desactivar) este *hook* desde línea de comandos:

```
alu@tos:~/git-seguro$ git config --bool hooks.gitleaks true
alu@tos:~/git-seguro$ git config --bool hooks.gitleaks
true
alu@tos:~/git-seguro$
```

Comprobamos que el nuevo script funciona de forma similar:

```
alu@tos:~/git-seguro$ git commit -m "test"
...
9:52PM WRN leaks found: 1
9:52PM INF scan duration: 54.537883ms

>> pre-commit: Error: gitleaks protect has detected sensitive information in your changes.
>> pre-commit: No se realizará el commit
alu@tos:~/git-seguro$
```

Y si estás seguros de lo que hace y considera que es un "falso positivo" puede desactivar `hooks.gitleaks` con el valor `false` para hacer el pre-commit sin ejecutar gitleaks:

```
alu@tos:~/git-seguro$ git config --bool hooks.gitleaks false
alu@tos:~/git-seguro$ git commit -m "testing pre-commit: me salto validación clave.txt"
[main e864dcf] testing pre-commit: me salto validación clave.txt
1 file changed, 1 insertion(+)
create mode 100644 clave.txt
alu@tos:~/git-seguro$
```

7.4. Pre-commit en Python

Es posible usar en vez de un *shell script* un *script en Python* como el que proponen los autores de gitleaks:

```
#!/usr/bin/env python3
import os, sys
import subprocess

def gitleaksEnabled():
    out = subprocess.getoutput("git config --bool hooks.gitleaks")
    if out == "false":
        return False
    return True

if gitleaksEnabled():
    exitCode = os.WEXITSTATUS(os.system('gitleaks protect -v --staged'))
    if exitCode == 1:
        print('Warning: gitleaks has detected sensitive information in your changes. To disable git config hooks.gitleaks false ')
        sys.exit(1)
else:
    print('gitleaks disabled (enable `git config hooks.gitleaks true`')
```

7.5. Dudas tras la clase

- **Cómo deshacer el último commit:** Si no quieres mantener los cambios y no ha publicado en GitHub: `git reset --hard HEAD~1`. Si quieres deshacer el commit pero mantener los cambios `--soft` en vez de `--hard`.
- Como borrar información sensible ya incluida en el repositorio remoto:
- Si hay token y claves implicados revocarlos.
- Cambiar los datos sensibles o comprometidos.
- Borrar los ficheros afectados con la utilidad `git filter-repo` o BFG.

6. Utiles

6.1 Git y VSCode

- Ayuda en [Using Git source control in VS Code](#)
- Tutorial Git: [Git con Visual Studio Code](#)

6.2 1. Anexo dudas git

6.2.1 1.1. Actualizar el main local

Si desea actualizar su repositorio local con los cambios en GitHub (suyos o de terceros) debe hacer un `git pull` (o `git fetch` && `git merge`) en la rama `main`:

```
clases@vm:~/23-demo-pr$ git switch main
Cambiado a rama 'main'
Tu rama está actualizada con 'origin/main'.
clases@vm:~/23-demo-pr$ ls -al
total 16
drwxr-xr-x  3 clases clases 4096 feb 13 13:34 .
drwx----- 23 clases clases 4096 feb 13 12:19 ..
drwxr-xr-x  8 clases clases 4096 feb 13 13:34 .git
-rw-r--r--  1 clases clases  12 feb 13 12:19 README.md
clases@vm:~/23-demo-pr$ git pull
Enter passphrase for key '/home/clases/.ssh/id_ed25519':
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
Desempaquetando objetos: 100% (7/7), 3.13 KiB | 3.13 MiB/s, listo.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
Desde github.com:angoies/23-demo-pr
 451b44c..b7ffb5f  main      -> origin/main
* [nueva rama]      angoies-patch-1 -> origin/angoies-patch-1
Actualizando 451b44c..b7ffb5f
Fast-forward
 .github/workflows/lint.yml | 27 ++++++
 doc.txt                    |  1 +
 2 files changed, 28 insertions(+)
 create mode 100644 .github/workflows/lint.yml
 create mode 100644 doc.txt
clases@vm:~/23-demo-pr$ ls -al
total 24
drwxr-xr-x  4 clases clases 4096 feb 13 13:35 .
drwx----- 23 clases clases 4096 feb 13 12:19 ..
-rw-r--r--  1 clases clases  32 feb 13 13:35 doc.txt
drwxr-xr-x  8 clases clases 4096 feb 13 13:35 .git
drwxr-xr-x  3 clases clases 4096 feb 13 13:35 .github
-rw-r--r--  1 clases clases  12 feb 13 12:19 README.md
clases@vm:~/23-demo-pr$
```

Otros comandos relacionados:

```
$ git fetch origin
$ git log main..origin/main
$ git diff main origin/main
$ git diff --name-only main origin/main
```

6.2.2 1.2. Ver cambios

Si quiere ver los cambios entre commits de un repositorio puede usar `git diff`

```
clases@vm:~/23-demo-pr$ git log --oneline
89a7343 (HEAD -> miapp) segunda prueba del Action: no debe dejar merge
c02ded4 (origin/miapp) añadimos fichero .py para probar Action
b7ffb5f (origin/main, origin/HEAD, main) Merge pull request #2 from angoies/angoies-patch-1
59e321d (origin/angoies-patch-1) Create lint.yml
147fead Merge pull request #1 from angoies/feature-xyz
c974611 (origin/feature-xyz, feature-xyz) update para prueba de PR
451b44c Initial commit
clases@vm:~/23-demo-pr$ git diff c02ded4 89a7343
diff --git a/app.py b/app.py
index 01c76a6..767efee 100644
--- a/app.py
+++ b/app.py
@@ -3,7 +3,9 @@

def suma(a: int, b: int) -> int:
    """ docstring function
+
+ # debe dar errores en linter
+
    """
-   x = "no se usa"
-   return a + b
+   x = "no se usa esta variable"
+   return a + b
clases@vm:~/23-demo-pr$
```

6.2.3 1.3. Listar configuración remoto

```
$ git remote -v
> origin https://github.com/OWNER/REPOSITORY.git (fetch)
> origin https://github.com/OWNER/REPOSITORY.git (push)
```

6.2.4 1.4. Pasar remoto de https a ssh

Modificar el `origin` y pasar de https a ssh:

```
$ git remote set-url origin git@github.com:OWNER/REPOSITORY.git
```

- Verificar el cambio:

```
$ git remote -v
# Verify new remote URL
> origin git@github.com:OWNER/REPOSITORY.git (fetch)
> origin git@github.com:OWNER/REPOSITORY.git (push)
```

- En sentido contrario sería similar:

```
git remote set-url origin https://github.com/OWNER/REPOSITORY.git
```

6.2.5 1.5. Subir varias ramas a GitHub

Sponga que desea subir varias ramas al mismo tiempo y una de ellas no existe aún en GitHub. El comando más directo es:

```
git push origin main dev
```

- `origin`: Indica el nombre del servidor remoto (GitHub por defecto).
- `main dev`: Lista las ramas locales que quieres enviar. Git detectará que `main` ya existe y la actualizará, y verá que `dev` es nueva, por lo que la creará en el servidor.

6.2.6 1.6. Squash and Merge (Aplastar y Fusionar)

Esta opción toma todos los commits por ejemplo que hiciste en `dev` y los combina en uno solo antes de ponerlos en `main`. El resultado es Un historial de `main` limpio, con un solo commit por cada funcionalidad terminada (sin ver los commits intermedios de esa rama).

En este caso el proceso de fusión sería en dos pasos:

```
git merge --squash dev
git commit -m "Descripción de la funcionalidad".
```

Duda: ¿Por qué `git merge --squash` no hace el commit automáticamente? Por diseño Git detiene el proceso después de "aplastar" los cambios y los deja en el Staging Area (el área de preparación). Esto permite:

- Revisar que todo esté correcto antes de finalizar.
- Escribir un mensaje de commit personalizado que resuma todos los cambios que acabas de fusionar.

6.2.7 1.7. Ir hacia atrás

Detectamos algo que no nos gusta en el último commit y queremos "retorceder" en el tiempo..

1.7.1. La opción "dura": `git reset --hard`

Si lo que quieres es borrar el commit y que los archivos vuelvan a estar como antes (eliminando los cambios del área de preparación), el comando es:

```
git reset --hard HEAD~1
```

- `HEAD~1` : Significa "un paso atrás del estado actual".
- `--hard` : Borra los cambios de los archivos. Tu carpeta de proyecto quedará limpia, como si nunca hubieras hecho el merge.*

1.7.2. La opción precavida: `git reset --soft`

Si quieres deshacer el commit, pero mantener los cambios en tus archivos (por ejemplo, para revisar algo antes de volver a intentar el commit):

```
git reset --soft HEAD~1
```

El commit desaparece del historial, pero todos los archivos modificados aparecerán de nuevo en el Staging Area (listos para ser confirmados otra vez).

1.7.3. `git reflog`

Si por error ejecutas un reset y sientes que perdiste el rastro de dónde estabas, Git guarda un historial de todos tus movimientos de puntero llamado Reflog.

Ese caso puedes:

- Ejecuta el comando `git reflog`
- Busca el commit al que quieras regresar (buenos mensajes de commit harán este apso más sencillo)
- Copia el hash del commit (el código alfanumérico (ej: abc1234))
- Ejecuta el comando: `git reset --hard abc1234.`

6.2.8 1.8. Modificar el ultimo commit

[Atlassian: Reescritura del historial](#)

1.8.1. el mensaje

Para modificar solamente el mensaje del último commit en Git ejecuta

```
git commit --amend -m "Nuevo mensaje".
```

1.8.2. Modificación de archivos confirmados

El ejemplo siguiente muestra una situación habitual en el desarrollo basado en Git. Supongamos que hemos editado algunos archivos que queremos confirmar en una sola instantánea, pero que nos olvidamos de añadir uno. Arreglar este error tan solo es cuestión de preparar el otro archivo y confirmar con la marca `--amend`:

```
# Edit hello.py and main.py
git add hello.py
git commit
# Realize you forgot to add the changes from main.py
git add main.py
git commit --amend --no-edit
```

6.3 Borrador: uso GitHub como "Drive" en LMSGI

6.3.1 1. Introducción

Se pretende tener sincronizado el contenido de sus ejemplos de clase con un repositorio en GitHub. Las instrucciones suponen que se está usando la máquina virtual del módulo y que el editor de trabajo es VsCode. No se usa la autenticación SSH en este ejemplo.

6.3.2 2. Paso inicial: Crear repositorio local

1. Abrir con Vscodex el directorio que quiere mantener (el nombre de la carpeta será el nombre del repositorio en Github)
2. Iniciar repositorio: Source Control / Iniciar repo
3. Añadir ficheros a la zona de *stage*: en *Source Control* se muestran los ficheros modificados bajo la etiqueta "Changes". Puede añadir todos los ficheros de golpe o ir seleccionando uno a uno pulsando el símbolo +.
4. Los ficheros ahora se muestran en "Staged Changes"
5. Antes de hacer commit debe configurar el `user.name` y `user.email` de git (se puede usar la opción `--global`)

```
git config user.name Ana
git config user.email ana.perez@miempresa.internal
```

6. Hacer commit especificando un mensaje

6.3.3 3. Subir/crear repositorio en Github

(Github <= Repositorio local)

1. Crear/subir repositorio a Github: **Publish Branch**
 - a. Vscodex pide autenticarse en Github (Allow)
 - b. Se abre navegador: introducir datos, etc.
 - c. Permitir abrir el enlace desde Vscodex, etc.
 - d. Aparecen en VScodex dos opciones: (privado o publico)
2. Una vez "publicado" tiene la opción de abrir el repo en Github para comprobar, etc

6.3.4 4. Actualizar repositorio en GitHub

Tras una sesión de trabajo en el aula la cosa es más sencilla

1. Añadir ficheros modificados (pasar de *Changes* a *Staged Changes*)
2. Hacer Commit
3. Sincronizar cambios con Github

Eso mismo en un terminal sería

```
git add <directorio> <files>...
git commit -m "mensaje descriptivo"
git push
```

6.3.5 5. Trabajar en casa

La primera vez en casa:

- Clonar el repositorio en Vscodex utilizando la URL que proporciona Github en el botón "Code" del repositorio

A partir de ese momento: 1. Actualizar el repositorio local (GitHub => Repositorio local) (git pull) 2. Si hace cambios en casa el proceso sería:

1. Añadir ficheros modificados o nuevos a Staged
2. hacer commit
3. publicar los cambios en GitHub

6.3.6 6. Si modifica el repositorio en casa y regresa al aula

Si quiere tener los cambios en el aula es necesario sincronizar el directorio de clase con el directorio de Github (GitHub => Repositorio local aula). Para ello basta hacer un `git pull` (o buscar la misma opción desde los menús de VSCode)

6.3.7 7. Almacenamiento de las credenciales

- En Github aparece en Integración / Aplicaciones / Authorized OAuth Apps
- En local el editor VSCode las almacena en el anillo de claves del Sistema Operativo. Si no está disponible guarda el token de autenticación en su propio almacenamiento interno.